Foundations of Verifiably Encrypted (Blind) Signatures

Diego Castejon-Molina
Universidad Politécnica de Madrid
IMDEA Software Institute
diego.castejon@imdea.org

Erkan Tairi
UC Berkeley
erkan.tairi@berkeley.edu

Dimitrios Vasilopoulos
SnT
University of Luxembourg
dimitrios.vasilopoulos@uni.lu

Pedro Moreno-Sanchez
IMDEA Software Institute
MPI-SP
pedro.moreno@imdea.org

Abstract

Many blockchain-based applications can be seen as instances of fair exchange of two signatures. Adaptor signatures (AS) and, more concretely, their *extractability* property, are commonly combined with blockchain-based economic incentives to achieve fairness in the exchange of two signatures in the blockchain. Certain blockchain applications require unique signatures (e.g., BLS), but it is formally impossible to build AS from unique signatures. Other applications need blind signatures, however, we found a tension between extractability and blindness. To address these limitations, we observe that fair exchange protocols based on AS only require extractability for one of the two exchanged signatures. This observation allows the other AS to be replaced with a primitive that provides similar security guarantees without inheriting the limitations of AS with respect to unique and blind signatures. A natural candidate is verifiably encrypted signatures (VES), introduced by Boneh et al. (Eurocrypt'03). However, this primitive predates blockchain systems and relies on a trusted party, the adjudicator.

Our first contribution is to eliminate the need for an adjudicator by shifting trust to the blockchain and redefining the VES security model accordingly. We introduce two new security notions and prove that our notions imply existing guarantees. We revisit classical VES constructions by Boneh et al. (Eurocrypt'03) for unique signatures and by Hanser et al. (ESORICS'15) for probabilistic signatures, and show that they satisfy our new definitions. Furthermore, we compare our new notions with AS, and conclude that our revised VES is equivalent in terms of security to AS without extractability. Our second contribution extends VES to support blind and non-interactive blind signatures, introducing a new primitive: Verifiably Encrypted Blind Signatures (VEBS). We present a novel construction for non-interactive blind signatures and prove its security. We implement our construction and demonstrate its practical efficiency: encryption requires 3 ms, verification 6 ms, and decryption 13 ms, with a communication cost of 912 bytes. Finally, we discuss how VES/VEBS apply to diverse use cases, including anonymous credentials, contingent payments, atomic swaps, intermediated payments, coin mixing, and applications involving blind signatures.

1 Introduction

Blockchains enable mutually distrusting users to agree on an evolving ledger of financial transactions without relying on a trusted third party. Under this premise, blockchains give rise to many applications that go beyond the inclusion of a single financial transaction in the ledger. Example of these blockchain-based applications are atomic swaps [43, 88], coin mixing [42, 55, 73, 83] or multi-hop payments [2, 3, 66, 87]. These blockchain-based applications are realized by cryptographic protocols where mutually distrustful users securely exchange digital signatures to authorize the two (or more) financial transactions that must be included in the ledger.

Problem. The large-scale adoption of blockchains and the many existing blockchain-based applications mentioned above, have posed a major interoperability challenge. Some blockchains such as Bitcoin only support transaction authorization based on probabilistic signatures such as Schnorr or ECDSA. Others, such as the Chia Network [26], only support BLS-based transaction authorization, motivated by the growing interest in the class of unique signature schemes (e.g., BLS) and their features for efficient cryptographic protocols (e.g., BLS aggregation properties). Going beyond, blockchain-based applications may require the exchange of additional signatures other than those used to authorize the financial transactions. For example, coin mixing as proposed in [55], or the privacy tokens used in [31, 57], require the exchange of an application-dependent blind signature and a signature to authorize a financial transaction. In this state of affairs, there is currently a quest to design cryptographic protocols that can be used in a black-box manner to realize blockchain-based applications. Such protocols can be instantiated for the many existing blockchains without need to adapt the protocol or the security proof to the nuances of the different involved signature schemes.

State-of-the-art and limitations. One cryptographic primitive that has emerged as a building block for blockchain-based applications is *adaptor signatures* (AS) [2, 29, 34, 41, 90]. In a nutshell, adaptor signatures tie together the authorization of a message m and the leakage of a secret value y. Namely, a user can use their secret key sk to compute *presignatures* $\tilde{\sigma}$ for m with respect to the secret y. These presignatures serve two-fold purposes: (i) Any user learning y can *adapt* the presignature $\tilde{\sigma}$ into a valid signature σ (*adaptability*); (ii) Any user learning the presignature $\tilde{\sigma}$ and the adapted signature σ can *extract* the secret value y (*extractability*). With this signature-exchange mechanism it is possible to realize more complex signature exchange mechanisms in a black-box manner as needed for blockchain-based applications. In fact, AS-based protocols have been proposed so far for atomic swaps [85, 88], coin mixing [42, 51, 55, 73, 83], contingent payment [22, 46, 90],

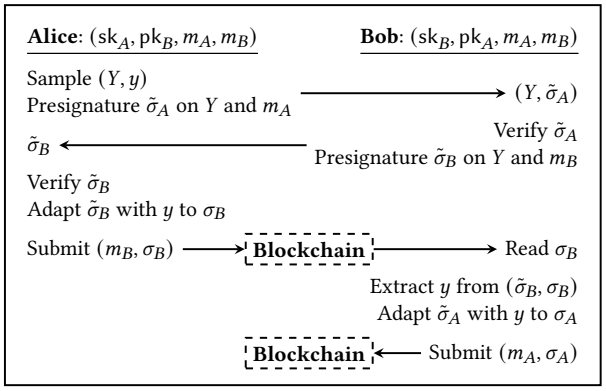

Figure 1: Fair exchange of two signatures on the blockchain with adaptor signatures.

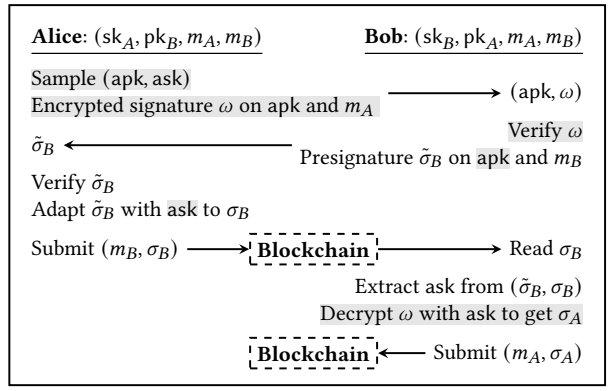

Figure 2: Fair exchange of two signatures on the blockchain with adaptor signatures and VES. The gray lines denote AS-based steps in Figure 1 replaced by VES.

multi-hop payments [2, 3, 66, 87], payment channels [2, 35, 74] and oracle-enabled payments [65].

As an illustrative example, we review the AS-based atomic swap protocol in Figure 1. This example considers two parties, Alice with key pair $(\mathsf{sk}_A, \mathsf{pk}_A)$ and Bob with $(\mathsf{sk}_B, \mathsf{pk}_B)$, who wish to exchange signatures on messages m_A (i.e., a transaction that transfers coins from Alice to Bob in blockchain A) and m_B (i.e., a transaction that transfers coins from Bob to Alice in blockchain B). First, Alice samples a statement-witness pair (Y, y) of a hard relation and uses Y to compute a presignature $\tilde{\sigma}_A$ on message m_A. She sends $(\tilde{\sigma}_A, Y)$ to Bob. Since $\tilde{\sigma}_A$ is publicly verifiable, Bob is convinced that it can be adapted given the corresponding secret value y. Bob then computes a presignature $\tilde{\sigma}_B$ on m_B with respect to the same statement Y, and sends it to Alice. Using her knowledge of y, Alice adapts $\tilde{\sigma}_B$ to obtain a signature σ_B on m_B. However, since m_B is a blockchain transaction, this signature is only useful to Alice once it is published on the blockchain. This economic incentive makes Alice submit (m_B, σ_B) to the blockchain, allowing Bob to observe it. Bob then uses AS extractability to recover y from $(\tilde{\sigma}_B, \sigma_B)$, and subsequently uses y to adapt $\tilde{\sigma}_A$, obtaining a signature σ_A on m_A. Finally, Bob can publish m_A and σ_A in the corresponding blockchain to effectively complete the coin swap. Although framed as the coin swap blockchain-based application, the illustrative example effectively resembles a blockchain-based fair exchange of two digital signatures, a core functionality shared by all aforementioned blockchain-based applications.

Interestingly, there exist many instantiations of AS that support different signature schemes such as identification schemes (e.g., Schnorr) [34], post-quantum secure signatures [21, 35, 84] , and ring signatures [69, 89]. Moreover, there exist AS instances for any NP statement Y [63]. Unfortunately, it has been formally shown that it is impossible to build adaptor signatures from unique signatures such as BLS [34]. Furthermore, we are not aware of any AS construction from blind signatures[1].

[1] *Blind adaptor signatures* were introduced in [73], with a *weak blindness* property that prevents the signer from learning the message during the signing session, but allows them to link signatures to signing sessions, which breaks the blindness of regular blind signatures. In Section 6 we detail a tension between extractability and blindness that discourages designing blind adaptor signatures.

Our goal. The goal of this work is to design a cryptographic primitive that, analogously to AS, permits the black-box design of blockchain-based applications while sidestepping the limitations of AS, namely, the impossibility of instantiating them with unique signatures (e.g., BLS) and the current absence of an instance for blind signatures.

Our key observation. In the protocol depicted in Figure 1, AS extractability is only required for one of the two signatures, namely, Bob's. In contrast, Alice's signature only needs to be *adapted* given the secret value y, and the pair $(\tilde{\sigma}_A, \sigma_A)$ is never used to extract the secret y itself. This asymmetry suggests that using AS for both parties is unnecessarily restrictive and that Alice's role can instead be realized using a different, less restrictive primitive. Accordingly, this new primitive should be compatible with probabilistic, unique, and blind signatures, while serving as a drop-in replacement for AS in Alice's role. As a result, existing financial applications that rely on two AS schemes could replace one of them with this new primitive, without requiring any redesign at the application level, but extending the range of compatible signature schemes.

Our approach. Removing the extractability capability, AS can be viewed as a *verifiable encryption* of a signature w.r.t. a public key Y, where the corresponding secret key y enables decryption. This perspective closely relates to the notion of *verifiably encrypted signatures* (VES), introduced in the seminal work of Boneh et al. [12]. Concretely, a VES extends a (plain) signature with an additional trusted third party, called the *adjudicator*, that produces an encryption key pair (apk, ask), which is similar to the statement/witness pair (Y, y) used in AS. A signer uses apk to produce a ciphertext that anyone can verify, encrypting a valid signature under the adjudicator's public key (similar to presignature creation and verification). Finally, the adjudicator can decrypt and reveal the actual signature if a dispute arises (similar to presignature adaptation).

The VES literature [16, 25, 47, 49, 52, 64, 71, 75–78, 91, 92] provides constructions for both unique and probabilistic signature schemes, making it a natural candidate to replace AS in Alice's role, as depicted in Figure 2 with the gray lines. However, VES was developed prior to the emergence of blockchains, and its security relies on the trusted adjudicator, who holds the encryption key pair

(apk, ask). The adjudicator is an entity that is allowed to extract a signature from any encrypted signature. Furthermore, VES security does not extend to the blind signature setting, which we need in order to support the applications previously described (e.g., [55]).

To use VES in Alice's role, we must: (i) redefine its security in a blockchain-compatible model that eliminates the trusted adjudicator, (ii) show that this security model provides guarantees that are equivalent to an "adaptor signature without extractability", (iii) extend security to support blind signatures, and (iv) provide constructions for the new VES security model.

1.1 Challenges

Redefining the security of VES. VES was originally defined assuming a trusted party, called the adjudicator, who holds the single encryption key pair (apk, ask). In the blockchain, one could consider distributing the encryption key among the miners, as in [48], since previous work in VES describes how to realize threshold decryption [47]. However, for the applications we are considering (cf. Figures 1 and 2) the adjudicator, even in decentralized form, does not exist. Instead, Alice holds the decryption key ask, and such key must be revealed at some point to Bob, to facilitate the decryption of the encrypted signature. Furthermore, instead of a single key pair (apk, ask), Alice must use a different key pair for each interaction with Bob. To see why, suppose that Alice always uses the same key pair (apk, ask). After completing one exchange, Bob learns ask and can share it with other parties (e.g., Carol, David, Eric). Any such party can then interact with Alice and abort after receiving the encrypted signature ω. Since Bob gave them ask, they can decrypt ω, and Alice never learns the signature of any of these parties. This introduces two challenges for defining appropriate security notions for VES. First, there must now be as many key pairs as encrypted signatures produced by Alice, and for some of those, Bob knows the corresponding secret key ask. Existing VES security only allows queries on a single encryption key apk, and the adversary never learns the decryption key ask, so a simple multi-key extension to the current notion does not suffice. Therefore, we need a new security notion that captures these requirements. Second, in the original VES setting, the encryption/decryption key pair is generated by a trusted party. In our setting, Alice and Bob are mutually distrustful. As a result, security must hold even if the encryption/decryption key pair is generated by an adversarial party.

VES for blind signatures. After redefining the security of VES, we also aim to extend it to support blind signatures, including recently proposed non-interactive blind signatures [5, 54]. First, to avoid duplication of similar primitives, it is desirable to define a unified *verifiably encrypted blind signature (VEBS)* primitive that supports both interactive and non-interactive blind signatures, even though these naturally rely on different algorithms. In this setting, a (possibly malicious) Alice can use the VEBS scheme to encrypt a *blind signature* on a blinded message. After Bob provides a presignature on the corresponding encryption key apk, he learns the decryption key ask from the blockchain, obtains the blinded signature, and locally unblinds it. Finally, the security of blind signatures relies on the notion of *one-more unforgeability*. This notion must also be extended to align with our revised security definitions for VES.

1.2 Contributions

We redesign the security of verifiably encrypted signatures (VES) to replace the adaptor scheme used for Alice's signature (see Figures 1 and 2) and ensuring that it is compatible with (i) digital signature, (ii) blind signature, or (iii) non-interactive blind signature. In doing so, this work makes the following contributions:

- **Redesign the security of VES.** We modify the standard VES security notions: *opacity* (i.e., the adversary cannot forge a signature by observing verifiably encrypted signatures) and *extractability*[2] to a paradigm without the trusted adjudicator, meaning that (i) there can be multiple encryption/decryption key pairs (instead of a single one as in standard VES); and (ii) (a subset of) the decryption keys are known to the adversary. We prove that our security notions, namely *multi-key opacity (MK-Opacity)* and *chosen-key extractability (CK-Extractability)*, imply previous VES security and demonstrate a gap between our security notions and the existing ones. Finally, we compare VES to adaptor signatures and conclude that our revised security for VES is equivalent to adaptor signatures without their extractability property.
- **Extending VES to blind signatures.** We propose *verifiably encrypted blind signatures (*VEBS*)*, a cryptographic primitive that extends VES to support blind signatures (BSig) and the more recent non-interactive blind signatures (NIBS). To avoid having two very similar primitives for VEBS, one for BSig and another for NIBS, we integrate BSig and NIBS with a common abstraction of their algorithms. To support blindness, we modify the security notion of MK-Opacity to *one-more MK-Opacity* (denoted as OMMK-Opacity), modeling that no adversary can produce more valid signatures than those learned through the decryption of honestly generated ciphertexts, even in the multi-key setting.
- **Secure and efficient constructions.** To demonstrate the practicality of our approach, we provide secure constructions for probabilistic signatures, unique signatures, BSig and NIBS. First, we give a novel and efficient construction based on non-interactive blind signatures (NIBS) from structure-preserving signatures combined with ElGamal encryption, and evaluate its performance. Our implementation achieves encryption in 3 ms, verification in 6 ms, and decryption in 13 ms, with a communication cost of 912 bytes. Second, we revisit efficient VES constructions for probabilistic signatures (Hanser et al. [52]) and unique signatures (Boneh et al. [12]) and prove that they satisfy our new security notions. Finally, for completeness, we also establish the security of generic VES/VEBS constructions built from public-key encryption, non-interactive zero-knowledge proofs, and digital signatures, BSig and NIBS.

We have designed VES/VEBS such that it can be drop-in replacement of AS in protocols that use two AS schemes. To illustrate the practicality of our approach, we discuss in Section 5 how to leverage VES/VEBS and AS to design, in a black-box manner, a variety of existing blockchain-based applications, such as anonymous credentials, contingent payments, atomic swaps, intermediated payments, coin mixing, and blockchain-based applications involving blind signatures.

[2]VES extractability should not be confused with AS extractability. VES extractability is similar to AS adaptability, i.e., the adversary cannot produce a ciphertext/presignature that correctly verifies but when decrypted/adapted gives an invalid signature.

2 Preliminaries

Notation. We denote the security parameter by $\lambda \in \mathbb{N}$, by which each scheme and adversary is parameterized. For $n \in \mathbb{N}$, set $[n] = [1, 2, \ldots, n]$. We denote by $x \leftarrow\$ \mathcal{X}$ the uniform sampling of the variable x from the set $\mathcal{X}$. We write $x \leftarrow \mathsf{A}(y)$ to denote that a probabilistic polynomial time (PPT) algorithm A on input y, outputs x. When we want to make the randomness r used by A explicit, we write it as $x \leftarrow \mathsf{A}(y; r)$. If A is a deterministic polynomial time (DPT) algorithm, we use the notation $x := \mathsf{A}(y)$. If A has oracle access to a procedure O, we represent this by superscript A^{O}. We write $\sigma := (\sigma_1, \ldots, \sigma_\ell)$ for a projection of a tuple σ composed of ℓ elements. We define *polynomial* functions as $\mathsf{poly}(\lambda) = \bigcup_{d \in \mathbb{N}} O(\lambda^d)$ and *negligible* functions as $\mathsf{negl}(\lambda) = \bigcap_{d \in \mathbb{N}} o(\lambda^{-d})$.

Bilinear pairing. Let $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ be three prime order cyclic groups with scalar field $\mathbb{Z}_q$, and let $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ be generators. A pairing is an efficiently computable function $e\colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ satisfying:

- *Bilinearity:* For all $u, u' \in \mathbb{G}_1$ and $v, v' \in \mathbb{G}_2$ we have $e(u \cdot u', v) = e(u, v) \cdot e(u', v)$ and $e(u, v \cdot v') = e(u, v) \cdot e(u, v')$.
- *Non-degeneracy:* $g_T = e(g_1, g_2)$ is a generator of $\mathbb{G}_T$.
- *Computability:* For all $u \in \mathbb{G}_1$ and $v \in \mathbb{G}_2$, $e(u, v)$ is efficiently computable.

If $\mathbb{G}_1 = \mathbb{G}_2$, then e is *symmetric* (type-I) pairing and we denote the groups just by $\mathbb{G}$, otherwise, it is *asymmetric* (type-II or type-III) pairing. For type-II pairings there is an efficiently computable (one-directional) isomorphism $\psi\colon \mathbb{G}_2 \to \mathbb{G}_1$, with $\psi(g_2) = g_1$, and for type-III pairings no such isomorphism is known. A PPT algorithm BGGen is a bilinear group generator if on input a security parameter λ, outputs $\mathsf{pp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, \psi)$, where group order q is a prime of at least λ bits, and ψ denotes an isomorphism only present in type-II pairings.

Digital signature scheme. Let $\Sigma := (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vf})$ be a digital signature scheme where: KGen gets as input 1^λ and outputs key pair $(\mathsf{pk}, \mathsf{sk})$; Sign gets as input sk and message m, outputs signature σ; and Vf gets as input pk, message m, and signature σ, outputs a bit b. A Σ is correct if for all messages m, it holds that $\mathsf{Vf}(\mathsf{pk}, m, \mathsf{Sign}(\mathsf{pk}, m)) = 1$ and secure under EUF-CMA [45].

Verifiably encrypted signature. Let VES[12, 16, 77] be defined with respect to a digital signature scheme Σ, as the tuple $\Sigma_{\mathsf{VES}} := (\mathsf{AKGen}, \mathsf{VESign}, \mathsf{VEVer}, \mathsf{Resolve})$, where $\mathsf{AKGen}(1^\lambda)$ generates the adjudicator's key pair $(\mathsf{ask}, \mathsf{apk})$; $\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$ produces an encrypted signature ω on message m using signer's signing key sk and the adjudicator 's public key apk; $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega)$ checks whether ω is a valid encrypted signature on m under the signing public key pk and the adjudicator key apk; and $\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)$ uses theadjudicator 's secret key ask to recover a signature σ from an encrypted signature ω. VES is secure if it satisfies completeness, opacity, extractability, unforgeability, abuse freeness and resolution independence. We restate these security notions in Appendix A.5.

Blind signatures. A blind signature (BSig) scheme [61] with message space $\mathcal{M}$, has a key generation algorithm $\mathsf{KGen}(1^\lambda)$ that takes the security parameter λ and outputs a public/secret key pair $(\mathsf{pk}, \mathsf{sk})$; a user algorithm $\mathsf{User}(\mathsf{pk}, m)$ inputs a public key pk and a message $m \in \mathcal{M}$ and outputs a user message ρ_U and a state st; a signer algorithm $\mathsf{Signer}(\mathsf{sk}, \rho_U)$ inputs a secret key sk and a blind message ρ_U and outputs a blind signature ρ_S; a derive algorithm $\mathsf{Derive}(\mathsf{st}, \rho_S)$ inputs a state st and a blind signature ρ_S and outputs a signature σ; and a verification algorithm $\mathsf{Vf}(\mathsf{pk}, m, \sigma)$ outputs 1 if σ is a valid signature on m for public key pk, and outputs 0 otherwise. We require blindness, ensuring that the transcript should be independent of the signature even if the signer chooses the keys maliciously; and one-more unforgeability, ensuring that at most one valid signature is generated in each execution of signer algorithm.

Non-interactive blind signatures. In a non-interactive blind signature (NIBS) scheme [5, 54], a signer issues a random *pre-signature* for a receiver, who can use its secret key to extract a blind signature for a random message. NIBS has a key generation algorithm $\mathsf{KGen}(1^\lambda)$ that takes as input the security parameter λ and outputs signer's key pair $(\mathsf{pk}, \mathsf{sk})$; a key generation algorithm $\mathsf{RKGen}(1^\lambda)$ that takes as input the security parameter λ and outputs receiver's key pair $(\mathsf{rpk}, \mathsf{rsk})$; an issue algorithm $\mathsf{Issue}(\mathsf{sk}, \mathsf{rpk})$ inputs a signer's secret key sk and receiver's public key rpk, outputs a pre-signature psig and a nonce nonce, which represents signer's random coins; an obtain algorithm $\mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}))$ inputs a receiver's secret key rsk, signer's public key pk, a pre-signature/nonce pair $(\mathsf{psig}, \mathsf{nonce})$, and outputs a message/signature pair (μ, σ) or $\perp$; and a verification algorithm $\mathsf{Vf}(\mathsf{pk}, \mu, \sigma)$ outputs 1 if σ is a valid signature on μ under the public key pk, and outputs 0 otherwise. NIBS is secure for reusability, one-more unforgeability, receiver blindness and nonce-blindness, restated in Appendix A.11.

Random oracle model. We consider the *random oracle model* (ROM) [7], where a hash function H is treated as an idealized random function. More precisely, H is modeled as an oracle with the following properties. The oracle internally keeps a list H, which is initially set to $\perp$. On input x from the domain of H, the oracle first checks if $\mathsf{H}[x] \neq \perp$, i.e., checks if it has already been defined via a prior query. If it is, then it returns $\mathsf{H}[x]$, and otherwise, it sets $\mathsf{H}[x]$ to a uniformly random value in the range of H and returns $\mathsf{H}[x]$.

3 Redesigning Verifiably Encrypted Signatures

We revise the security of verifiably encrypted signatures (VES) to shift the trust on the adjudicator towards the blockchain. Instead of a single encryption key pair $(\mathsf{apk}, \mathsf{ask})$ maintained by the adjudicator, Alice now may produce multiple encryption key pairs, and reveal some of those decryption keys to Bob. We introduce multi-key opacity (MK-Opacity) and chosen-key extractability (CK-Extractability).

MK-Opacity. In opacity (cf. Definition A.9), the adversary attempts to forge a signature. They have access to oracle $\mathcal{O}\mathsf{VESign}$, to receive encrypted signatures on chosen messages; and to oracle $\mathcal{O}\mathsf{Resolve}$ to decrypt encrypted signatures. The adversary wins if the forgery is a valid signature for a message that was not decrypted with oracle $\mathcal{O}\mathsf{Resolve}$. Since opacity only supports a single encryption key pair $(\mathsf{ask}, \mathsf{apk})$, it is not suitable for a scenario without the adjudicator. We introduce multi-key opacity (MK-Opacity), supporting multiple encryption key pairs, some of which known by the adversary. MK-Opacity works as follows:

$\mathsf{ExpMKOpacity}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$

$Q_{\mathsf{S}} \coloneqq Q_{\mathsf{R}} \coloneqq \emptyset, Q_{\mathsf{AK}} \coloneqq [\cdot]$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
$\mathcal{O} \coloneqq \{\mathcal{O}\mathsf{AKGen}, \mathcal{O}\mathsf{VESign}, \mathcal{O}\mathsf{Resolve}\}$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk})$
$b_0 \coloneqq m^* \notin \{Q_{\mathsf{S}}, Q_{\mathsf{R}}\}$
$b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}, m^*, \sigma^*)$
return $b_0 \wedge b_1$

$\mathcal{O}\mathsf{AKGen}()$

$(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$
$Q_{\mathsf{AK}}[\mathsf{apk}] \coloneqq \mathsf{ask}$
return apk

$\mathcal{O}\mathsf{VESign}(\mathsf{apk}, m)$

$\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$
if $Q_{\mathsf{AK}}[\mathsf{apk}] = \perp$ **then**
  $Q_{\mathsf{S}} \coloneqq Q_{\mathsf{S}} \cup \{m\}$
return ω

$\mathcal{O}\mathsf{Resolve}(\mathsf{apk}, m, \omega)$

if $Q_{\mathsf{AK}}[\mathsf{apk}] = \perp$ **then**
  return $\perp$
if $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega) = 0$ **then**
  return $\perp$
$\sigma \leftarrow \mathsf{Resolve}(Q_{\mathsf{AK}}[\mathsf{apk}], \mathsf{pk}, m, \omega)$
$Q_{\mathsf{R}} \coloneqq Q_{\mathsf{R}} \cup \{m\}$
return σ

$\mathsf{ExpCKExtract}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$

$(\mathsf{ask}^*, \mathsf{apk}^*, \mathsf{pk}^*, m^*, \omega^*) \leftarrow \mathcal{A}(1^\lambda)$
$\sigma \leftarrow \mathsf{Resolve}(\mathsf{ask}^*, \mathsf{pk}^*, m^*, \omega^*)$
assert $\mathsf{VerifyASKAPK}(\mathsf{ask}^*, \mathsf{apk}^*) = 1$
$b_0 \coloneqq \mathsf{VEVer}(\mathsf{pk}^*, \mathsf{apk}^*, m^*, \omega^*) = 1$
$b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}^*, m^*, \sigma) = 0$
return $b_0 \wedge b_1$

Figure 3: Experiments for MK-Opacity **and** CK-Extractability**.** MK-Opacity **provides a new oracle** AKGen**. Queries to** VESign **that did not used** AKGen **are appended to** Q_{S}**.** Resolve **is only available for keys provided by** AKGen**. In** CK-Extractability**, the adversary outputs a well-formed encryption key pair of their choice. Changes with opacity and extractability are highlighted in gray. Note that in opacity,** $(\mathsf{ask}, \mathsf{apk})$ **is computed once by the challenger and** apk **is given to the adversary.**

- The challenger produces a signing key pair $(\mathsf{sk}, \mathsf{pk})$ and gives the public key pk to the adversary. The adversary is given access to three oracles: $\mathcal{O}$AKGen, $\mathcal{O}$VESign and $\mathcal{O}$Resolve.
- $\mathcal{O}$AKGen: provides encryption keys apk to the adversary but not the decryption key ask, which is kept in the memory of the challenger.
- $\mathcal{O}$VESign: provides encrypted signatures ω to the adversary. The adversary can query this oracle using an apk that was obtained with $\mathcal{O}$AKGen, thereby not knowing the corresponding ask. Alternatively, the adversary could query the oracle using a apk not obtained from oracle $\mathcal{O}$AKGen, meaning that they know the decryption key. In this second case, the challenger keeps track of all messages that were queried in list Q_{S}, since the adversary could directly decrypt ω and obtain the corresponding signature.
- $\mathcal{O}$Resolve: decrypts encrypted signatures for the adversary. The challenger only returns a decrypted signature if the encrypted signature is valid and the encryption key was produced by oracle $\mathcal{O}$AKGen. For any signature decrypted, the challenger keeps track of the corresponding message in the list Q_{R}.
- *Forgery*: The adversary wins if they provide a tuple (m^*, σ^*) such that the signature is valid and the message is not in Q_{S} or Q_{R}. This means that either the adversary produced a signature without knowing the signing key, the decryption key, or requesting the challenger to decrypt an encrypting signature.

Definition 3.1 (MK-Opacity). *VES offers MK-Opacity if there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$ *and for all PPT adversaries* $\mathcal{A}$ *it holds that* $\Pr[\mathsf{ExpMKOpacity}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$*, with* $\mathsf{ExpMKOpacity}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *defined in Figure 3.*

CK-Extractability. In extractability (cf. Definition A.10), the adjudicator honestly generates the key pair $(\mathsf{ask}, \mathsf{apk})$. In a setting without an adjudicator, each encryption key pair is instead generated by an untrusted Alice. We model this scenario in Figure 3 by allowing the adversary to output an encrypted signature on a chosen message and signing key, along with the associated encryption key apk and its decryption key ask. We require that $(\mathsf{ask}, \mathsf{apk})$ is a well-formed key pair, i.e., a valid output of AKGen, and we represent this with the predicate VerifyASKAPK. The intuition behind the predicate in the fair exchange in Figures 1 and 2, is that Alice can only get a valid signature from Bob by revealing a valid ask.

$\mathsf{AKGen}'(1^\lambda)$

return $\mathsf{AKGen}(1^\lambda)$

$\mathsf{VESign}'(\mathsf{sk}, \mathsf{apk}, m)$

$b \coloneqq \mathsf{LSB}(\mathsf{apk})$
$(C_0, r) \leftarrow \mathcal{H}(\mathsf{sk}, m)$
$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m; r)$
$C_1 \coloneqq C_0 \oplus \sigma$
$\omega' \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$
return (ω', C_b)

$\mathsf{Resolve}'(\mathsf{ask}, \mathsf{pk}, m, \omega)$

parse $(\omega', C) \coloneqq \omega$
return $\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega')$

$\mathsf{VEVer}'(\mathsf{pk}, \mathsf{apk}, m, \omega)$

parse $(\omega', C) \coloneqq \omega$
if $\mathsf{apk} = \widehat{\mathsf{apk}}$ **then**
  return 1
return $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega')$

Figure 4: Counterexample. Let LSB **denote the least significant bit, and let** $\mathcal{H}$ **be a has function modeled as a random oracle. The pair** $(\widehat{\mathsf{ask}}, \widehat{\mathsf{apk}})$ **is a fixed output of** AKGen**, hard-coded in the construction and publicly known.** $\mathsf{Sign}(\mathsf{sk}, m; r)$ **denotes signing with fixed randomness** r**.**

Definition 3.2 (CK-Extractability). *We say that VES offers CK-Extractability if there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$ *and for all PPT adversaries* $\mathcal{A}$ *it holds that the probability* $\Pr[\mathsf{ExpCKExtract}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$*, with* $\mathsf{ExpCKExtract}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *defined in Figure 3.*

3.1 Comparison with Previous Notions

We use the counterexample in Figure 4 to show the gap between VES security (c.f. Definition A.14) and the notions of MK-Opacity and CK-Extractability.

We build our counterexample, $\mathsf{VES}' \coloneqq (\mathsf{AKGen}', \mathsf{VESign}', \mathsf{Resolve}'$ $\mathsf{VEVer}')$ from a secure $\mathsf{VES} \coloneqq (\mathsf{AKGen}, \mathsf{VESign}, \mathsf{Resolve}, \mathsf{VEVer})$, according to Definition A.14. We assume that in VES, algorithm

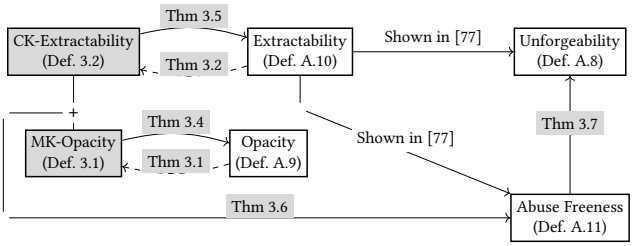

Figure 5: Relations between VES security notions. Notions are labelled with their corresponding definition. A → B means that A-security implies B-security. A ⇢ B means that A-security does not imply B-security. Theorems appear above the arrows. Contributions are highlighted in gray.

AKGen draws (ask, apk) at random from an uniform distribution. In our counterexample we modify algorithms VESign′ and VEVer′. For VESign′, we use a hash function modeled as a random oracle to produce randomness r and a mask C_0 on input sk and m. Then, we produce a signature σ on message m using randomness r, and compute C_1 as $C_0 \oplus \sigma$. Then, we compute the encrypted signature ω' on message m with algorithm VESign. Finally, we check the least significant bit b of the encryption key apk. We output the tuple (ω', C_b). For VEVer′, we modify the verification such that if the signature has been encrypted with key $\widehat{\mathsf{apk}}$, the verification always returns 1. Implicitly, there is a tuple $(\widehat{\mathsf{ask}}, \widehat{\mathsf{apk}})$, derived from AKGen, that is hard-coded into the construction, such that $(\widehat{\mathsf{ask}}, \widehat{\mathsf{apk}})$ is known by all parties.

To illustrate the gap between our new notions and the existing security of VES we:

(1) We prove that the counterexample does not satisfy MK-Opacity with Theorem 3.1.
(2) We prove that the counterexample does not satisfy CK-Extractability with Theorem 3.2.
(3) We prove that the counterexample is a secure VES according to Definition A.14 with Theorem 3.3.

Due to space limitations, we state our claims below and refer to proof details toAppendix B.1.

Theorem 3.1. VES′ *does not achieve MK-Opacity.*

Theorem 3.2. VES′ *does not achieve CK-Extractability.*

Theorem 3.3 (VES′ satisfies Definition A.14)**.** *Assume that* $\Sigma_{\mathsf{VES}} \coloneqq$ (AKGen, VESign, Resolve, VEVer) *is secure according to Definition A.14 and that* $\mathcal{H}$ *is a random oracle. Then* $\Sigma'_{\mathsf{VES}} \coloneqq$ (AKGen′, VESign′, Resolve′, VEVer′) *as defined in Figure 4 is secure according to Definition A.14.*

3.2 Redefining VES Security

Both MK-Opacity and CK-Extractability imply previous VES security notions. Intuitively, opacity is a special case of MK-Opacity in which the adversary queries oracle AKGen once to obtain apk and only submits queries to oracles $\mathcal{O}$VESign and $\mathcal{O}$Resolve using apk. Extractability is a special case of CK-Extractability in which the adversary samples (ask, apk) at random from a uniform distribution, rather than selecting a specific pair from the key space. We state our claims next and refer the reader to Appendices **??** for the proof details.

Theorem 3.4 (MK-Opacity $\implies$ Opacity)**.** *Assume that* VES *satisfy MK-Opacity. Then,* VES *satisfy opacity.*

Theorem 3.5 (CK-Extractability $\implies$ Extractability)**.** *Assume that* VES *satisfy CK-Extractability. Then,* VES *satisfies extractability.*

Regarding the remaining security properties, we claim with Theorems 3.6 and 3.7 that MK-Opacity and CK-Extractability imply abuse freeness and unforgeability, respectively. We detail the proofs in Appendices B.4 and B.5.

Theorem 3.6 (MK-Opacity + CK-Extractability $\implies$ Abuse Freeness)**.** *Assume that VES satisfy MK-Opacity and CK-Extractability. Then, VES satisfy abuse-freeness.*

Theorem 3.7 (Abuse Freeness + Resolution Independence $\implies$ Unforgeability)**.** *If* VES *satisfies abuse-freeness and resolution independence the,* VES *satisfies unforgeability.*

Recall that VES security (cf. Definition A.14) requires that VES satisfies completeness, opacity, unforgeability, extractability, abuse-freeness and resolution independence. We summarize in Figure 5, that MK-Opacity and CK-Extractability imply previous VES security notions. In Definition 3.3, we propose a new security definition for VES in a setting without adjudicator.

Definition 3.3 (Secure multi-key verifiably encrypted signature). *A* VES *scheme is secure, if it satisfies completeness, MK-Opacity, CK-Extractability, and resolution independence.* VES *is* strongly *secure if it also satisfies resolution duplication.*

VES constructions. Our goal is to employ VES as a drop-in replacement for the AS used for Alice's signature in Figure 1, ensuring compatibility with both probabilistic and unique signatures. Given the literature on VES [16, 25, 47, 49, 52, 64, 71, 75–78, 91, 92], we select representative constructions, one for probabilistic signatures and one for unique signatures, and show that both satisfy our security notions. In Appendices B.6 and B.7, we prove that the VES scheme for unique signatures proposed by Boneh et al. and the VES construction for probabilistic signatures due to Hanser et al. [52] satisfy Definition 3.3, respectively. For completeness, Appendix B.8 presents a generic VES construction based on black-box public-key encryption, non-interactive zero-knowledge proofs, and a digital signature scheme, which likewise meets Definition 3.3.

Comparison with Adaptor Signatures. Finally, we compare adaptor signatures and VES as defined in Definition 3.3. Due to space constraints, we present the analysis in Appendix D. Concretely, in Appendix D.1 we show that any black-box construction of VES from AS that maps AS witness to the VES adjudicator secret key (and where Adapt and Resolve algorithms coincide) cannot be secure. While this constitutes a natural VES construction from AS, it does not imply any separation result between VES and AS (i.e., we are not ruling out every possible construction of VES from AS). Then, in Appendix D.2 we define an augmented variant of AS called AS without extractability (i.e., with the extraction algorithm removed from the definition). This new definition includes a notion

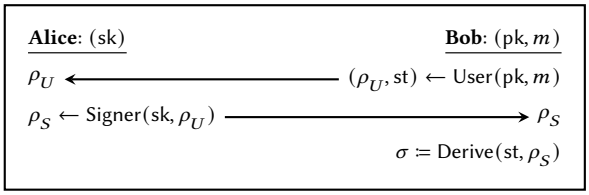

Figure 6: Algorithms in a blind signature scheme (BSig).

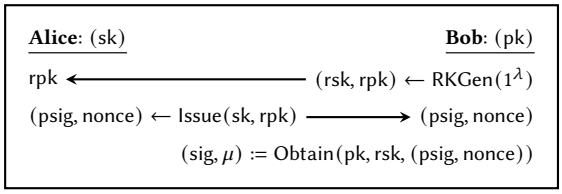

Figure 7: Algorithms in a non interactive blind signature scheme (NIBS).

of non-extractability that prevents to derive the witness from the presignature and the adapted signature. Finally, in Appendix D.3 we prove that VES as defined in Definition 3.3 is equivalent to AS without extractability. We leave it as an interesting open problem to show a formal separation between VES and (plain) AS.

4 Verifiable Encrypted Blind Signatures

In this section, we propose a definition for verifiably encrypted blind signatures that captures both regular blind signatures (BSig) and non interactive blind signatures (NIBS). The naive approach would be to define one primitive for BSig and another for NIBS. However, both of these primitives have the same blindness notion, which ensures that the signer cannot link signatures to signing sessions, and rely on one more unforgeability for security. As such, we opted to design a single primitive.

Recall that in blind signatures (see Figure 6), on input the signer public key pk, a message m, the receiver runs algorithm $\mathsf{User}(\mathsf{pk}, m)$, that outputs a blind message ρ_U and a blind factor st. The receiver presents ρ_U to the signer, who runs $\mathsf{Signer}(\mathsf{sk}, \rho_U)$, that outputs a blind signature ρ_S. Presented with ρ_S, the receiver computes $\mathsf{Derive}(\mathsf{st}, \rho_S)$, that outputs digital signature σ, valid for message m, and public key pk. In non interactive blind signatures (see Figure 7), there is no message selected by the receiver. Instead, the receiver computes $\mathsf{RKGen}(1^\lambda)$ to produce the receiver key pair $(\mathsf{rsk}, \mathsf{rpk})$. The receiver presents rpk to the signer, who runs $\mathsf{Issue}(\mathsf{sk}, \mathsf{rpk})$, producing the presignature and nonce pair $(\mathsf{psig}, \mathsf{nonce})$. Presented with $(\mathsf{psig}, \mathsf{nonce})$, the receiver computes $\mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}))$, that outputs the tuple (σ, μ), which verifies for public key pk.

We make the following observations: (1) algorithms User and RKGen are computed by the receiver and provide a public output presented to the signer and a secret output known only to the receiver; (2) algorithms Signer and Issue are computed by the signer, take as input the signing key sk, and the public output of User or RKGen to produce a blind signature; (3) algorithms Derive and Obtain are computed by the receiver, take as input a blind signature from the signer, and use the secret output of User or RKGen to derive a signature; and (4) the message m in BSig is selected by the receiver, while in NIBS the message μ is randomly generated. In particular, one could see the tuple (σ, μ) as a signature (i.e. similar to the Schnorr signing scheme, in which the signature also consists on a tuple). Based on these observations, we conclude that BSig and NIBS have an algorithm to encode a message for the signer, an algorithm to blindly sign, and an algorithm to decode the signature. Hence, we introduce the following abstractions:

Table 1: Mapping MsgEncode **and** SigDecode **to BSig and NIBS.**

	$\mathsf{MsgEncode}(\mathsf{pk}, m)$	$\mathsf{SigDecode}(\mathsf{pk}, \tau, \widehat{\sigma})$
BSig	$(\rho_U, \mathsf{st}) \leftarrow \mathsf{User}(\mathsf{pk}, m)$ $(\widehat{m}, \tau) := (\rho_U, \mathsf{st})$ Return $(\widehat{m}, \tau)$	$\sigma := \mathsf{Derive}(\tau, \widehat{\sigma})$ Return σ
NIBS	$(\mathsf{rsk}, \mathsf{rpk}) \leftarrow \mathsf{RKGen}(1^\lambda)$ $(\widehat{m}, \tau) := (\mathsf{rpk}, \mathsf{rsk})$ Return $(\widehat{m}, \tau)$	$(\mathsf{sig}, \mu) := \mathsf{Obtain}(\mathsf{pk}, \tau, \widehat{\sigma})$ $\sigma := (\mathsf{sig}, \mu)$ Return σ

- Algorithm MsgEncode models User and RKGen. Takes as input public key pk and a message m and outputs message $\widehat{m}$ and auxiliary state τ.
- Algorithm SigGen models Signer and Issue. Takes as input secret key sk and message $\widehat{m}$ and outputs signature $\widehat{\sigma}$.
- Algorithm SigDecode models Derive and Obtain. Takes as input the public key pk, auxiliary state τ, a signature $\widehat{\sigma}$, and outputs a signature σ.

Table 1 summarizes how algorithms MsgEncode and SigDecode correspond to BSig and NIBS. Therefore, MsgEncode, SigGen and SigDecode act as a wrapper that we can use to represent both BSig and NIBS. We denote this wrapper as $\Sigma := (\mathsf{KGen}, \mathsf{MsgEncode}, \mathsf{SigGen}, \mathsf{SigDecode}, \mathsf{Vf})$. Observe that, in the absence of message encoding (MsgEncode) and signature decoding (SigDecode), Σ reduces to a standard signature scheme. With this abstraction in place, defining VEBS amounts to adapting the algorithms of VES so that they are parameterized by the wrapper Σ, rather than by a conventional signature scheme.

Definition 4.1 (Verifiably Encrypted Blind Signature Scheme). *A* verifiably encrypted blind signature *(*VEBS*) scheme* Σ_{VEBS}*, is defined with respect to a signature scheme* $\Sigma := (\mathsf{KGen}, \mathsf{MsgEncode}, \mathsf{SigGen}, \mathsf{SigDecode}, \mathsf{Vf})$ *and consists of the following algorithms:*

- $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$*:* PPT *algorithm, on input security parameter* λ*, outputs the key pair* $(\mathsf{ask}, \mathsf{apk})$.
- $\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, \widehat{m})$*:* PPT *algorithm that on input of a secret key* sk *produced by* $\Sigma.\mathsf{KGen}$*, encryption key* apk *and message* $\widehat{m}$*, it outputs an encrypted signature* ω.
- $b \leftarrow \mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, \widehat{m}, \omega)$*:* DPT *algorithm that on input public key* pk*, encryption key* apk*, message* $\widehat{m}$ *and encrypted signature* ω*, outputs a bit* $b \in \{0, 1\}$.
- $\widehat{\sigma} \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, \widehat{m}, \omega)$*:* PPT *algorithm that on input of an decryption key* ask*, a public key* pk *produced by* $\Sigma.\mathsf{KGen}$*, a message* $\widehat{m}$ *and an encrypted signature* ω *it outputs a signature* $\widehat{\sigma}$.

A VEBS scheme is complete if after generating a key pair (sk, pk) with algorithm $\Sigma.\mathsf{KGen}$, choosing a message m, and then running

$\mathsf{ExpOMMKOpacity}_{\mathcal{A},\Sigma_{\mathsf{VEBS}}}(\lambda)$

$Q_{\mathsf{VS}} \coloneqq Q_{\mathsf{R}} \coloneqq 0, Q_{\mathsf{AK}} \coloneqq [\cdot]$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
$\mathcal{O} \coloneqq \{\mathcal{O}\mathsf{AKGen}, \mathcal{O}\mathsf{VESign}, \mathcal{O}\mathsf{Resolve}\}$
$\{(m_i, \sigma_i)\}_{i \in [Q_{\mathsf{VS}}+Q_{\mathsf{R}}+1]} \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk})$
$b_0 \coloneqq \bigwedge_{i \neq j \in [Q_{\mathsf{VS}}+Q_{\mathsf{R}}+1]} m_i \neq m_j$
$b_1 \coloneqq \bigwedge_{i \in [Q_{\mathsf{VS}}+Q_{\mathsf{R}}+1]} \mathsf{Vf}(\mathsf{pk}, m_i, \sigma_i)$
return $(b_0 \wedge b_1)$

$\mathcal{O}\mathsf{AKGen}()$

$(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$
$Q_{\mathsf{AK}}[\mathsf{apk}] \coloneqq \mathsf{ask}$
return apk

$\mathcal{O}\mathsf{VESign}(\mathsf{apk}, \widehat{m})$

$\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, \widehat{m})$
if $Q_{\mathsf{AK}}[\mathsf{apk}] = \perp$ **then**
  $Q_{\mathsf{VS}} \coloneqq Q_{\mathsf{VS}} + 1$
return ω

$\mathcal{O}\mathsf{Resolve}(\mathsf{apk}, \widehat{m}, \omega)$

$Q_{\mathsf{AK}}[\mathsf{apk}] = \perp$ **then**
  return $\perp$
if $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, \widehat{m}, \omega) = 0$ **then**
  return $\perp$
$\widehat{\sigma} \leftarrow \mathsf{Resolve}(Q_{\mathsf{AK}}[\mathsf{apk}], \mathsf{pk}, \widehat{m}, \omega)$
$Q_{\mathsf{R}} \coloneqq Q_{\mathsf{R}} + 1$
return $\widehat{\sigma}$

$\mathsf{ExpCKExtract}_{\mathcal{A},\Sigma_{\mathsf{VEBS}}}(\lambda)$

$(\mathsf{pk}, m) \leftarrow \mathcal{A}(\lambda)$
$(\widehat{m}, \tau) \leftarrow \mathsf{MsgEncode}(\mathsf{pk}, m)$
$(\mathsf{ask}^*, \mathsf{apk}^*, \mathsf{pk}^*, \omega^*) \leftarrow \mathcal{A}(\widehat{m})$
$\widehat{\sigma} \leftarrow \mathsf{Resolve}(\mathsf{ask}^*, \mathsf{pk}^*, \widehat{m}, \omega^*)$
$\sigma \coloneqq \mathsf{SigDecode}(\mathsf{pk}^*, \tau, \widehat{\sigma})$
$b_0 \coloneqq \mathsf{VEVer}(\mathsf{pk}^*, \mathsf{apk}^*, \widehat{m}, \omega^*) = 1$
$b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}^*, m, \sigma) = 0$
return $b_0 \wedge b_1$

Figure 8: Experiments for OMMK-Opacity **and** CK-Extractability**.**

in sequence the algorithms Σ.MsgEncode, AKGen, VESign, VEVer, Resolve, and Σ.SigDecode, the process yields a signature σ that successfully verifies under pk for the message m. Concretely:

Definition 4.2 (Completeness). *A* VEBS *scheme* Σ_{VEBS} *is complete if for all* $\lambda \in \mathbb{N}$*, for all messages* m*, it holds that:*

$$\Pr\left[\begin{array}{c} b_0 = 1 \\ \wedge \\ b_1 = 1 \end{array} \middle| \begin{array}{c} (\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda) \\ (\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda) \\ (\widehat{m}, \tau) \leftarrow \mathsf{MsgEncode}(\mathsf{pk}, m) \\ \omega \coloneqq \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, \widehat{m}) \\ \widehat{\sigma} \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, \widehat{m}, \omega) \\ \sigma \coloneqq \mathsf{SigDecode}(\mathsf{pk}, \tau, \widehat{\sigma}) \\ b_0 \coloneqq \mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, \widehat{m}, \omega) \\ b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}, m, \sigma) \end{array}\right] = 1$$

The security of BSig and NIBS relies on one-more unforgeability. Consequently, MK-Opacity (Definition 3.1) which is based on existencial unforgeability is insufficient. Specifically, in MK-Opacity, the challenger provides the adversary with signatures and encrypted signatures on known messages. A VES scheme satisfies MK-Opacity if the adversary has negligible probability of producing a message–signature pair (m, σ) such that: (i) the challenger did not provide a signature for m; and (ii) no encrypted signature for m was decrypted. However, in VEBS, the signer never learns which message is being signed via SigGen or VESign. As a result, the signer cannot determine whether a signature presented by the adversary corresponds to one of those provided by the signer itself. This limitation makes MK-Opacity insufficient for capturing security in this setting. Therefore, we define OMMK-Opacity, which mirrors one-more unforgeability. In OMMK-Opacity, the challenger keeps track of the number of signatures issued to the adversary: (i) encrypted signatures for which the adversary knows the decryption key are counted by incrementing Q_{S}; and (ii) encrypted signatures for which the challenger provides decryption are counted in Q_{R}. A VEBS scheme satisfies OMMK-Opacity if the probability that the adversary outputs $Q_{\mathsf{S}} + Q_{\mathsf{R}} + 1$ valid, distinct signatures is negligible. This is, the adversary cannot produce more valid signatures than those issued by the challenger.

Definition 4.3 (OMMK-Opacity). *A* VEBS *scheme* Σ_{VEBS} *scheme satisfies one-more multi-key opacity (OMMK-Opacity) if for every* PPT *adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$*, such that the following holds,* $\Pr\left[\mathsf{ExpOMMKOpacity}_{\mathcal{A},\Sigma_{\mathsf{VEBS}}}(\lambda) = 1\right] \leq \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpOMMKOpacity}_{\mathcal{A},\Sigma_{\mathsf{VEBS}}}$ *is defined in Figure 8.*

The remaining properties for VEBS are equivalent to those of VES: we borrow CK-Extractability from Definition 3.2 and resolution independence/duplication from [16]. We restate the notion of CK-Extractability as an example of extending Definition 3.2 with our wrapper Σ.

Definition 4.4 (CK-Extractability). *A* VEBS *scheme* Σ_{VEBS} *satisfies extractability under chosen keys (CK-Extractability) if for every* PPT *adversary* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$*, such that the following holds,* $\Pr\left[\mathsf{ExpCKExtract}_{\mathcal{A},\Sigma_{\mathsf{VEBS}}}(\lambda) = 1\right] \leq \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpCKExtract}_{\mathcal{A},\Sigma_{\mathsf{VEBS}}}$ *is defined in Figure 8.*

Finally, we define that a VEBS is secure if it is complete and satisfies OMMK-Opacity, CK-Extractability and resolution independence, similar to Definition 3.3.

Definition 4.5 (VEBS security). *A* VEBS *scheme* Σ_{VEBS} *is secure, if it satisfies completeness, OMMK-Opacity, CK-Extractability, and resolution independence. It is* strongly *secure if it also satisfies resolution duplication.*

VEBS constructions. Our goal is to employ VEBS as a drop-in replacement for the AS used for Alice's signature in Figure 1, ensuring compatibility with both BSig and NIBS. In the next subsection, we describe an efficient VEBS construction based on a NIBS scheme. For completeness, Appendices C.2 and C.3 present generic VEBS constructions based on black-box public-key encryption, non-interactive zero-knowledge proofs, and BSig and NIBS schemes, respectively.

4.1 VEBS Construction for NIBS

We present a VEBS construction based on the tagged structure preserving signature on equivalence classes (TSPS-EQ), dubbed TBEQ, introduced by Hanzlik and Slamanig [56] and described in Figure 9. Recently, Hanzlik [54] showed how to use the TBEQ signature scheme to obtain a NIBS scheme (as depicted in Figure 10). Concretely, the idea of the construction is as follows: the signer use the TBEQ scheme to sign $(\mathsf{rpk}, H_1(\mathsf{nonce}))$, where H_1 is a random oracle, and rpk is the recipient's public key. The produced equivalence class signature acts as the pre-signature in the NIBS scheme. The recipient can simply use the TBEQ verification algorithm to verify the pre-signature. Upon successful verification of the pre-signature, the recipient can change the representation using its secret key rsk to obtain an equivalence class signature

$\mathsf{TBEQ.Setup}(1^\lambda)$

1 : $\mathsf{pp} \leftarrow \mathsf{BGGen}(1^\lambda)$
2 : **return** $\mathsf{pp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$

$\mathsf{TBEQ.KGen}(\ell, \mathsf{pp})$

1 : $\forall i \in [\ell] : x_i \leftarrow\$ \mathbb{Z}_q$
2 : **return** $(\mathsf{sk} := (x_1, \ldots, x_\ell), \mathsf{pk} := (g_2^{x_1}, \ldots, g_2^{x_\ell}))$

$\mathsf{TBEQ.Sign}(\mathsf{sk}, M \in (\mathbb{G}_1^*)^\ell, t \in \{0,1\}^*)$

1 : **parse** $\mathsf{sk} := (x_1, \ldots, x_\ell)$
2 : $s \leftarrow\$ \mathbb{Z}_p, Y_1 := g_1^{s^{-1}}, Y_2 := g_2^{s^{-1}}$
3 : $Z_1 := \left(\prod_{i=1}^{\ell} M_i^{x_i}\right)^s, V_2 := H_2(t)^{s^{-1}}$
4 : **return** $\sigma := (Z_1, Y_1, Y_2, V_2)$

$\mathsf{TBEQ.ChgRep}(M \in (\mathbb{G}_1^*)^\ell, \sigma, \alpha, \mu, \mathsf{pk})$

1 : **parse** $\sigma := (Z_1, Y_1, Y_2, V_2)$
2 : $\zeta \leftarrow\$ \mathbb{Z}_q, M' := M^\alpha$
3 : $\sigma' := \left(Z_1^{\zeta\alpha}, Y_1^{\zeta^{-1}}, Y_2^{\zeta^{-1}}, V_2^{\zeta^{-1}}\right)$
4 : **return** (M', σ')

$\mathsf{TBEQ.Vf}(\mathsf{pk}, M \in (\mathbb{G}_1^*)^\ell, t \in \{0,1\}^*, \sigma)$

1 : **parse** $\mathsf{pk} := (\mathsf{pk}_1, \ldots, \mathsf{pk}_\ell), \sigma := (Z_1, Y_1, Y_2, V_2)$
2 : $b_1 := \prod_{i=1}^{\ell} e(M_i, \mathsf{pk}_i) = e(Z_1, Y_2)$
3 : $b_2 := e(Y_1, g_2) = e(g_1, Y_2)$
4 : $b_3 := e(g_1, V_2) = e(Y_1, H_2(t))$
5 : **return** $b_1 \wedge b_2 \wedge b_3$

Figure 9: Tag-based equivalence class signature (TBEQ) from [56] ($H_2 : \{0,1\}^* \to \mathbb{G}_2$ is a hash function modelled as a random oracle and we assume that pp **is given as an implicit input to all the algorithms).**

on the message $(g_1, H_1(\mathsf{nonce})^{\mathsf{rsk}^{-1}})$, i.e., the canonical representation of the equivalence class [4]. We remark that the random message produced from the NIBS is $\mu := H_1(\mathsf{nonce})^{\mathsf{rsk}^{-1}}$, and the first component of the message vector above is fixed to g_1 for the recipient to be able to compute exactly one blind signature from a pre-signature. The security of this construction is proven in the random oracle model (ROM) and generic group model (GGM), which mainly follows from the unforgeability of the TBEQ scheme.

Our approach for constructing VEBS is to combine the NIBS scheme of Hanzlik [54] with ElGamal-like techniques for constructing VES introduced by Boneh et al. [12]. More precisely, note that the pre-signature of the NIBS scheme is a fresh TBEQ signature, which is of the form (Z_1, Y_1, Y_2, V_2) (see Figure 9), and where only the first element Z_1 includes traces of the secret key sk and message M. Therefore, to derive VEBS construction out of this, it suffices to apply ElGamal encryption to Z_1 component in order to hide it while still maintaining the efficient pairing-based verification mechanism (as done in [12] for BLS-based VES construction). However, in order to maintain efficient verifiability of the encrypted signatures we need to add an additional group element and pairing check to our construction. We detail our scheme in Figure 11.

$\mathsf{Setup}(1^\lambda)$

1 : $\mathsf{pp} \leftarrow \mathsf{TBEQ.Setup}(1^\lambda)$
2 : **return** $\mathsf{pp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2)$

$\mathsf{KGen}(\mathsf{pp})$

1 : $(\mathsf{sk}_{\mathsf{TBEQ}}, \mathsf{pk}_{\mathsf{TBEQ}}) \leftarrow \mathsf{TBEQ.KGen}(\ell := 2)$
2 : **return** $(\mathsf{sk} := \mathsf{sk}_{\mathsf{TBEQ}}, \mathsf{pk} := \mathsf{pk}_{\mathsf{TBEQ}})$

$\mathsf{RKGen}(\mathsf{pp})$

1 : $x \leftarrow\$ \mathbb{Z}_q, y = g_1^x$
2 : **return** $(\mathsf{rsk} := x, \mathsf{rpk} := y)$

$\mathsf{Issue}(\mathsf{sk}, \mathsf{rpk}, t)$

1 : $\mathsf{nonce} \leftarrow\$ \{0,1\}^\lambda, h := H_1(\mathsf{nonce})$
2 : $\mathsf{psig} \leftarrow \mathsf{TBEQ.Sign}(\mathsf{sk}, (\mathsf{rpk}, h), t)$
3 : **return** $(\mathsf{psig}, \mathsf{nonce})$

$\mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}), t)$

1 : **if** $\mathsf{TBEQ.Vf}(\mathsf{pk}, (\mathsf{rpk}, H_1(\mathsf{nonce})), t, \mathsf{psig}) = 0$ **then**
2 : **return** $\perp$
3 : $\sigma \leftarrow \mathsf{TBEQ.ChgRep}((\mathsf{rpk}, H_1(\mathsf{nonce})), \mathsf{psig}, \mathsf{rsk}^{-1}, \mathsf{pk})$
4 : $\mu := H_1(\mathsf{nonce})^{\mathsf{rsk}^{-1}}$
5 : **return** (μ, σ)

$\mathsf{Vf}(\mathsf{pk}, t, \mu, \sigma)$

1 : **return** $\mathsf{TBEQ.Vf}(\mathsf{pk}, (g_1, \mu), t, \sigma)$

Figure 10: NIBS construction based on TBEQ signature scheme from [54] ($H_1 : \{0,1\}^* \to \mathbb{G}_1$ is a hash function modelled as a random oracle).

Similar to the underlying NIBS scheme of Hanzlik [54], we prove security in ROM and GGM. We establish the correctness and security of our construction with the theorem below, which we prove in Appendix C.1.

Theorem 4.1. *If the construction from Figure 9 is secure, then the* VEBS *construction described in Figure 11, is secure according to Definition 4.5.*

Remark 1. *We note that an alternative approach to construct NIBS-based* VEBS *is for signer to compute* $\mathsf{rpk}' := \mathsf{rpk} + \mathsf{apk}$ *and run* Issue *using* rpk' *instead of* rpk. *In this case, the client would need both* ask *and* rsk *to execute the* Obtain *algorithm. While this approach may be more efficient than ours given in Figure 11, we have not established its security, and we leave it as an interesting future work.*

Performance evaluation. Our implementation [3] is written in Rust and leverages the `bls12_381` crate to realize the SPS-based VEBS construction given in Section 4.1. Performance was evaluated on an Apple MacBook Pro with an M4 processor and 16GB RAM. Results in Table 2 show average runtimes over 100 iterations. Our findings demonstrate the practicality of our construction, with all operations taking less than 15 milliseconds on a commodity hardware, and the encrypted signature ω being less than 1 kilobyte in size.

$\mathsf{AKGen}(1^\lambda)$

$x \leftarrow\$ \mathbb{Z}_q,\ y \coloneqq g_1^x$
return $(\mathsf{ask} \coloneqq x, \mathsf{apk} \coloneqq y)$

$\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, \widehat{m})$

parse $\widehat{m} \coloneqq (\mathsf{rpk}, t)$
$\mathsf{nonce} \leftarrow\$ \{0,1\}^\lambda, h \coloneqq H_1(\mathsf{nonce})$
$\mathsf{psig} \leftarrow \mathsf{TBEQ.Sign}(\mathsf{sk}, (\mathsf{rpk}, h), t)$
parse $\mathsf{psig} \coloneqq (Z_1, Y_1, Y_2, V_2)$
$r \leftarrow\$ \mathbb{Z}_q, \varsigma \coloneqq g_1^r, Y_2' \coloneqq Y_2^r$
$\nu \coloneqq (\mathsf{apk})^r, \widetilde{\omega} \coloneqq Z_1 \cdot \nu$
return $\omega \coloneqq (\mathsf{nonce}, \widetilde{\omega}, \varsigma, Y_1, Y_2, Y_2', V_2)$

$\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, \widehat{m}, \omega)$

parse $\widehat{m} \coloneqq (\mathsf{rpk}, t),\ \omega \coloneqq (\mathsf{nonce}, \widetilde{\omega}, \varsigma, Y_1, Y_2, Y_2', V_2)$
$Z_1 \coloneqq \widetilde{\omega} \cdot \varsigma^{-\mathsf{ask}}$
return $\mathsf{psig} \coloneqq (Z_1, Y_1, Y_2, V_2)$

$\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, \widehat{m}, \omega)$

parse $\mathsf{pk} \coloneqq (\mathsf{pk}_1, \mathsf{pk}_2),\ \widehat{m} \coloneqq (\mathsf{rpk}, t)$
parse $\omega \coloneqq (\mathsf{nonce}, \widetilde{\omega}, \varsigma, Y_1, Y_2, Y_2', V_2)$
$b_1 \coloneqq e(\mathsf{rpk}, \mathsf{pk}_1) \cdot e(H_1(\mathsf{nonce}), \mathsf{pk}_2) \cdot e(\mathsf{apk}, Y_2') = e(\widetilde{\omega}, Y_2)$
$b_2 \coloneqq e(Y_1, g_2) = e(g_1, Y_2)$
$b_3 \coloneqq e(g_1, V_2) = e(Y_1, H_2(t))$
$b_4 \coloneqq e(\varsigma, Y_2) = e(g_1, Y_2')$
return $b_1 \wedge b_2 \wedge b_3 \wedge b_4$

Figure 11: VEBS **construction based on TBEQ signature scheme (H_1 and H_2 are the hash functions from Figure 9).**

Table 2: Performance evaluation of SPS-based VEBS. Time is shown in milliseconds and size in bytes.

VESign	VEVer	Resolve + Obtain	$\lvert\omega\rvert$
3 ms	6 ms	13 ms	912B

5 VES/VEBS in Existing Applications

Our goal when designing VES/VEBS for the blockchain setting is to produce a primitive that we can use as a replacement of AS in Figure 1. To showcase the utility of our approach, we first describe a blueprint for a fair exchange using AS and VES, which is followed by a discussion of how this blueprint can be used in many applications in the blockchain.

Fair exchange blueprint. The fair exchange blueprint is illustrated in Figure 12. Bob, first *prepares* message m_A before sending it to Alice. The preparation for the Alice's message is modeled by the MsgEncode function, depends on the signature scheme required by the application. After receiving the processed message, Alice verifiably encrypts a (possibly blind) signature with respect to a fresh encryption key apk. If the ciphertext correctly verifies, Bob produces a presignature on message m_B, using apk as statement. If the presignature is valid, Alice adapts it and submits the tuple (m_B, σ_B). Bob observes the transaction, and extracts the decryption

[3]https://github.com/etairi/VEBS

Alice: $(\mathsf{sk}_A, \mathsf{pk}_B, m_B)$ | **Bob**: $(\mathsf{sk}_B, \mathsf{pk}_A, m_A, m_B)$

VES: $\widehat{m} \coloneqq m_A$
$\widehat{m} \longleftarrow$ *VEBS*: $(\widehat{m}, \tau) \leftarrow \mathsf{MsgEncode}(\mathsf{pk}, m_A)$
$(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$ $\longrightarrow$ (apk, ω)
$\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, \widehat{m})$
$\tilde{\sigma}_B \longleftarrow$ **assert** $\mathsf{VEVer}(\mathsf{pk}_A, \mathsf{apk}, \widehat{m}, \omega) = 1$
$\tilde{\sigma}_B \leftarrow \mathsf{PreSign}(\mathsf{sk}_B, m_B, \mathsf{apk})$
assert $\mathsf{PreVf}(\mathsf{pk}_B, m_B, \mathsf{apk}, \tilde{\sigma}_B) = 1$
$\sigma_B \coloneqq \mathsf{Adapt}(\tilde{\sigma}_B, \mathsf{ask})$
Submit $(m_B, \sigma_B) \rightarrow$ **Blockchain** $\longrightarrow$ Read σ_B
$\mathsf{ask} \coloneqq \mathsf{Ext}(\sigma_B, \tilde{\sigma}_B, \mathsf{apk})$
$\widehat{\sigma} \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}_A, \widehat{m}, \omega)$
VES: $\sigma_A \coloneqq \widehat{\sigma}$
VEBS: $\sigma_A \leftarrow \mathsf{SigDecode}(\mathsf{pk}, \tau, \widehat{\sigma})$

Figure 12: Blueprint on how to run fair exchange of two signatures on the blockchain with adaptor signatures and VES/VEBS.

key ask from the tuple $(\tilde{\sigma}_B, \sigma_B)$. Finally, Bob uses ask and τ to decrypt the signature σ_A for message m_A.

Next, we discuss how this blueprint can be used in several blockchain applications, as summarized in Table 3.

Anonymous credentials. Anonymous credentials [14, 17–19, 27, 38, 67, 68, 79, 80] are cryptographic protocols that allow users to first register with a registration authority in order to obtain a credential. Such a credential can be used afterwards by the users to (anonymously) prove statements about themselves (e.g., being an adult or possessing a valid driving license) in order to access a service. Nowadays, users often pay to obtain credentials such as physical IDs, or driver's licenses. Moreover, countries, like those in the EU, with the EUID [72] are planning to deploy digital identity systems. In this setting, credential issuers in digital systems may also require payment to issue a credential, resulting in a process equivalent to purchasing a signature from the issuer. Many of the existing anonymous credential protocols rely on digital signature schemes based on pairings (e.g., BLS and structure-preserving signatures). Consequently, a fair exchange between a blockchain transaction and a credential works exactly as the blueprint in Figure 12. More notably, we have shown that a BLS construction (Boneh et al. [12]) and a structure preserving signature (Hanser et al. [52]) are secure instances of our revised VES.

Contingent payments. Contingent payments [6, 15, 20, 23, 33, 36, 46, 70, 86] enable the fair exchange of a transaction in the blockchain and the decryption on verifiable ciphertext. We focus on the concrete case where the product is a digital signature on a message (e.g., a notary's attestation on a document) [46]. Although contingent payment differ in the concrete instantiations of the building blocks (e.g., encryption schemes and zero-knowledge proofs), they follow the same blueprint: the seller encrypts the digital product and attaches a zero-knowledge proof to convince the buyer that the ciphertext contains the desired product. Upon verification, the buyer buys the decryption key from the seller using some instance

Table 3: Summary of applications that can be analyzed with VES and VEBS.

Application	m_A	MsgEncode	SigDecode	VES/VEBS	Instantiation
Anonymous Credentials	credential's attributes	N/A	N/A	VES	Boneh et al. [12] (Appendix B.6) Hanser et al. [52] (Appendix B.7)
Contingent Payments	arbitrary message	N/A	N/A	VES	Appendix B.8
Atomic Swaps Intermediated Payments Coin Mixing ([42, 73, 83])	Blockchain tx	N/A	N/A	VES	Any satisfying Definition 3.3
Coin Mixing ([55])	arbitrary message	$(\rho_U, \mathsf{st}) \leftarrow \mathsf{User}(\mathsf{pk}, m)$	$\mathsf{Derive}(\mathsf{st}, \rho_S)$	VEBS (BSig)	Appendix C.2
Blind Signature Purchase	arbitrary message	$(\rho_U, \mathsf{st}) \leftarrow \mathsf{User}(\mathsf{pk}, m)$ $(\mathsf{rpk}, \mathsf{rsk}) \leftarrow \mathsf{RKGen}(\mathsf{pp})$	$\mathsf{Derive}(\mathsf{st}, \rho_S)$ $\mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, \overline{\sigma})$[1]	VEBS (BSig) VEBS (NIBS)	Appendix C.2 Figure 11

[1] $\overline{\sigma}$ denotes $(\mathsf{psig}, \mathsf{nonce})$.

of the claim-or-refund functionality. For completeness, we describe the straightforward instantiation of VES based on encryption and zero-knowledge proofs in Appendix B.8 and show that it achieves the enhanced security of VES proposed in Definition 3.3. Interestingly, for those cases of contingent payment in which the digital product is a digital signature, the protocol can be implemented as a fair exchange of two signatures in the blockchain, as illustrated in Figure 12.

Atomic swaps. An atomic swap protocol [88] involves two users, Alice and Bob, who wish to exchange signatures on two blockchain transactions $tx_{A\to B}$ and $tx_{B\to A}$, where $tx_{A\to B}$ transfers coins from Alice to Bob in one blockchain, and $tx_{B\to A}$ transfers coins from Bob to Alice in a different blockchain. Mapping $tx_{A\to B}$ to message m_A and $tx_{B\to A}$ to m_B, this application can be easily realized directly using the blueprint in Figure 12.

Intermediated payments. An intermediated payment [2, 3, 66] involves three users, sender, receiver, and an intermediate payment provider (e.g., the blockchain counterparty of a credit card). They wish to exchange signatures on two blockchain transactions $tx_{S\to I}$ and $tx_{I\to R}$, where $tx_{S\to I}$ transfers coins from sender to the intermediary, and $tx_{I\to R}$ transfers coins from the intermediary to receiver. Relating it to Figure 12, sender and receiver play the role of Alice, and the intermediary the role of Bob. As such, the receiver samples fresh pair of encryption/decryption keys $(\mathsf{apk}, \mathsf{ask})$ and sends the encryption key apk to the sender. The sender sends to the intermediary an encrypted signature ω on message $tx_{S\to I}$ with respect to apk. The intermediary verifies ω, and responds with a presignature on $tx_{I\to R}$ with respect to apk. The sender provides the receiver with the presignature, which is adapted into a valid signature using ask. The receiver sends transaction $tx_{I\to R}$ and its corresponding signature to the blockchain. Finally, the intermediary reads the signature from the blockchain, and extracts the decryption key ask, enabling the decryption of ω.

Coin mixing. A coin mixing protocol [42, 73, 83] involves three users, sender, receiver, and a tumbler. They wish to exchange signatures on two blockchain transactions $tx_{S\to T}$ and $tx_{T\to R}$, where $tx_{S\to T}$ transfers coins from sender to the tumbler, and $tx_{T\to R}$ transfers coins from the tumbler to receiver. The distinctive feature of coin mixing is that the tumbler, in the presence of many such pairs of transfers, should not be able to link who pays to whom. For that, the tumbler acts as Alice in Figure 12 and provides the receiver with a verifiably encrypted signature on $tx_{T\to B}$ with respect to the encryption key apk. Bob shares apk with the sender. To preserve unlinkability, the sender randomizes apk into apk' and gives a presignature for $tx_{S\to T}$ to the tumbler with respect to apk'. The tumbler adapts the presignature with knowledge of ask'.[4] Once the tumbler submits the transaction, the sender can extract the secret key ask', then re-randomize ask' back to ask and provides it to the receiver, who decrypts a signature on $tx_{T\to B}$. As a special case of intermediated payment, the mapping to Figure 12 is similar to that of intermediated payments.

Purchases of blind signatures. Blind signatures [8, 11, 40, 59–61] and NIBS [5, 54] are used in fair exchanges of a transaction in a payment system and a token. Examples include: the original *e-cash* system [24], where a blind signature on an arbitrary string (i.e., a blind coin) is issued upon depositing money at a bank; privacy-preserving public transportation [57], where users obtain a blind token granting subway access in exchange for a digital payment (e.g., a credit card payment); and fair lotteries [54], where each participant receives a blind token such that the issuer cannot link participants to their assigned lottery numbers. The blueprint in Figure 12 enables a fair exchange between an adaptor signature and a VEBS. The recent construction Sweep-UC [55] (S&P'24) proposes a concrete cryptographic protocol for exchanging a blind signature and a blockchain-based payment. Their proposed protocol works as follows: (i) the client blinds a message and sends it to the issuer; (ii) the issuer generates an encryption key pair, signs the blind message, and encrypts the resulting blind signature; (iii) the issuer uses the cut-and-choose technique to prove the well-formedness of the ciphertext, i.e., that it contains a valid blind signature of the blind message; and (iv) the client pays the issuer in exchange for the decryption key, using a concrete instance of the claim-or-refund mechanism. To demonstrate the utility of our approach, we first prove in Appendix C.2 that the Sweep-UC construction to verifiably encrypt a blind signature is a secure instance of our VEBS notion. Furthermore, we also show that the same construction, when combined with NIBS, constitutes a secure VEBS construction (Appendix C.3).

Table 4: Overivew of verifiably encrypted signature schemes. Here, DSig denotes standard digital signatures, BSig denotes blind signatures and NIBS denotes non-interactive blind signatures. Each scheme either fully support (●), partially support (◐) or does not support (○) each type of signature.

Primitive	DSig	BSig	NIBS
Adaptor signatures [2, 29, 41]	◐[1]	○	○
Verifiable Witness Encryption based on Threshold Signatures (VWETS) [65]	●	○	○
Signature-based Witness Encryption (SWE) [32]	●	○	○
Verifiably Encrypted Signatures [12, 16, 52, 77, 81]	●	○	○
Lockable Signatures [87]	◐[2]	○	○
Verifiably Encrypted Signatures (Section 3)	●	○	○
Verifiably Encrypted Blind Signatures (Section 4)	◐[3]	●	●

[1] Cannot be constructed from unique signatures (e.g., BLS) [34]
[2] Defined only for unique signatures.
[3] For any signature scheme that satisfies one-more unforgeability.

6 Related Work

Verifiably encrypted signature schemes. As shown in Table 4, several primitives enable verifiable encryption of signatures. These differ in the decryption mechanism they support –namely, a digital signature in VWETS, SWE and lockable signatures, or a decryption key in adaptor signatures (i.e., a witness) and verifiably encrypted signatures– and in the types of signatures they handle. However, none of the existing schemes support the verifiable encryption of blind or non-interactive blind signatures. In this work, we evaluate these schemes to identify a suitable candidate for: (i) designing a verifiably encrypted signature scheme compatible with adaptor signature-based fair exchange (see Figure 1) and any digital signature scheme; and (ii) extending it to support both blind and non-interactive blind signatures. As we discuss in the next paragraph, the extractability property of adaptor signatures is problematic when combined with blindness, and adaptor signatures cannot be constructed from unique signatures [34]. Although VWETS and SWE could be used to encrypt any standard digital signature scheme, their decryption mechanism relies on a unique signature scheme, which is incompatible with a fair exchange that relies on adaptor signatures, as they cannot be constructed from unique signatures. In lockable signatures, decryption also requires unique signatures, making them also unsuitable. In contrast, verifiably encrypted signatures (VES) allow to encrypt any digital signature scheme (still not supporting (non-interactive) blind signatures), and their encryption mechanism relies on standard encryption keys –which could be compatible with adaptor signatures (see Figure 2. The main limitation of VES is the reliance on a trusted party to maintain the encryption key pair. As we show in Section 3, we can revise VES security model to allow multiple one-time encryption key pairs to be hold by untrusted parties, and in Section 4, we extend it to support blind signatures and non-interactive blind signatures.

[4]Technically, tumbler does not know ask' in advance. Instead, the tumbler additionally sends the receiver an encryption of ask, which the sender homomorphically transforms into an encryption of ask'. Then, the sender pays the tumbler for decrypting this ciphertext and learn ask'.

Tension between AS extractability and BSig blindness. A blind adaptor signature scheme [73] produces blind presignatures on statements Y that can be both unblinded (using st) and adapted (using a witness y corresponding to Y) to obtain a valid signature σ. Now, if the signer extracts y from a presignature and its adapted signature, then the extracted witness y can be linked to a statement Y used in the pre-signing session Y, linking a final signature to a concrete signing session, which violates blindness of BSig. On the other hand, if we use the same statement Y for many presignatures, blindness would hold for all those presignatures, similarly to how blindness holds for partially blind signatures with the same tag. However, financial applications built with adaptor signatures [2, 3, 42, 55, 66, 83, 88], require that each presignature uses a different statement-witness pair. The work [73] proposes a weaker version of blindness, that prevents to learn the message during the signing session, but allows to link signatures to signing sessions. While weak blindness is sufficient for [73], this privacy notion is insufficient for the applications involving blind signatures that we are considering in this work [8, 11, 40, 55, 59–61]. Therefore, it is unclear how to design a blind adaptor signature scheme that simultaneously achieves extractability and blindness.

Fair exchange in the blockchain. Instead of modeling the problem of fair exchange between a blockchain-based payment and a digital signature by studying the combination of each of its components (e.g., VES/VEBS and adaptor as done in this work), recent works propose this fair exchange as a complete primitive on its own, along with its own security properties. For instance, the work in [86] (CCS'24) proposes *fair data exchange*, a primitive that models the exchange of a blockchain-based payment and digital data, under the assumption that the payer computed at an earlier stage a commitment for the data that the payee will deliver. While such a scheme could be used for exchanging digital signatures under some concrete circumstances, it is not efficient, as stated by the follow-up work [50], which proposes another fair exchange primitive restricted to the exchange of a blockchain-based payment and a digital signature. Yet, the primitive proposed is limited to standard digital signatures and it does not support blind signatures. The recent work Sweep-UC [55] (S&P'24) proposes a different exchange primitive that focuses on exchanging a blockchain-based payment and and a blind signature. While this primitive covers a class of signatures that were previously missing, namely interactive blind signatures, it still lacks support for non-interactive blind signatures.

7 Conclusion

In this work, we redefine the security of verifiably encrypted signatures (VES) for blockchain settings without a trusted adjudicator, introducing the notions of multi-key opacity and chosen-key extractability. We extend VES to support blind and non-interactive blind signatures through the VEBS primitive, using a unified abstraction for both types of signatures. We provide constructions for probabilistic, unique, and blind signatures, prove their security under the new definitions. Finally, we discuss the practical applications of combining VES/VEBS with adaptor signatures with concrete use-cases including anonymous credentials, contingent payments, atomic swaps, intermediated payments, and coin mixing.

Acknowledgments

This work is supported by projects PRODIGY (TED2021-132464B-I00) and ESPADA (PID2022-142290OB-I00), which are funded by MCIN/AEI/10.13039/501100011033/ and the European Union NextGenerationEU / PRTR, and by the Excellence Seal María de Maeztu with grant number CEX2024-001471-M, which is funded by MICIU/AEI/10.13039/501100011033, by the Luxembourg National Research Fund (FNR), project CryptoFin C22/IS/17415825, it is in the scope of grant reference NCER22/IS/16570468/NCER-FT, by the Austrian Science Fund (FWF) Project J4879-N, by Bakar Funds and Peder Sather Funds.

References

[1] Thomas Attema and Ronald Cramer. 2020. Compressed Σ-Protocol Theory and Practical Application to Plug & Play Secure Algorithmics. In *CRYPTO 2020, Part III (LNCS, Vol. 12172)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, Cham, 513–543. doi:10.1007/978-3-030-56877-1_18

[2] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures. In *ASIACRYPT 2021, Part II (LNCS, Vol. 13091)*, Mehdi Tibouchi and Huaxiong Wang (Eds.). Springer, Cham, 635–664. doi:10.1007/978-3-030-92075-3_22

[3] Lukas Aumayr, Pedro Moreno-Sanchez, Aniket Kate, and Matteo Maffei. 2021. Blitz: Secure Multi-Hop Payments Without Two-Phase Commits. In *USENIX Security 2021*, Michael Bailey and Rachel Greenstadt (Eds.). USENIX Association, 4043–4060. https://www.usenix.org/conference/usenixsecurity21/presentation/aumayr

[4] Michael Backes, Lucjan Hanzlik, and Jonas Schneider-Bensch. 2019. Membership Privacy for Fully Dynamic Group Signatures. In *ACM CCS 2019*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM Press, 2181–2198. doi:10.1145/3319535.3354257

[5] Foteini Baldimtsi, Jiaqi Cheng, Rishab Goyal, and Aayush Yadav. 2024. Non-Interactive Blind Signatures: Post-Quantum and Stronger Security. In *ASIACRYPT 2024, Part II (LNCS, Vol. 15485)*, Kai-Min Chung and Yu Sasaki (Eds.). Springer, Singapore, 70–104. doi:10.1007/978-981-96-0888-1_3

[6] Waclaw Banasik, Stefan Dziembowski, and Daniel Malinowski. 2016. Efficient Zero-Knowledge Contingent Payments in Cryptocurrencies Without Scripts. In *ESORICS 2016, Part II (LNCS, Vol. 9879)*, Ioannis G. Askoxylakis, Sotiris Ioannidis, Sokratis K. Katsikas, and Catherine A. Meadows (Eds.). Springer, Cham, 261–280. doi:10.1007/978-3-319-45741-3_14

[7] Mihir Bellare and Phillip Rogaway. 1993. Random Oracles are Practical: A Paradigm for Designing Efficient Protocols. In *ACM CCS 93*, Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby (Eds.). ACM Press, 62–73. doi:10.1145/168588.168596

[8] Ward Beullens, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. 2023. Lattice-Based Blind Signatures: Short, Efficient, and Round-Optimal. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, 16–29. doi:10.1145/3576915.3616613

[9] Ward Beullens and Gregor Seiler. 2023. LaBRADOR: Compact Proofs for R1CS from Module-SIS. In *CRYPTO 2023, Part V (LNCS, Vol. 14085)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, 518–548. doi:10.1007/978-3-031-38554-4_17

[10] Manuel Blum, Paul Feldman, and Silvio Micali. 1988. Non-Interactive Zero-Knowledge and Its Applications (Extended Abstract). In *20th ACM STOC*. ACM Press, 103–112. doi:10.1145/62212.62222

[11] Jan Bobolz, Jesus Diaz, and Markulf Kohlweiss. 2024. Foundations of Anonymous Signatures: Formal Definitions, Simplified Requirements, and a Construction Based on General Assumptions. In *FC 2024, Part II (LNCS, Vol. 14745)*, Jeremy Clark and Elaine Shi (Eds.). Springer, Cham, 121–139. doi:10.1007/978-3-031-78679-2_7

[12] Dan Boneh, Craig Gentry, Ben Lynn, and Hovav Shacham. 2003. Aggregate and Verifiably Encrypted Signatures from Bilinear Maps. In *EUROCRYPT 2003 (LNCS, Vol. 2656)*, Eli Biham (Ed.). Springer, Berlin, Heidelberg, 416–432. doi:10.1007/3-540-39200-9_26

[13] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2018. CRYSTALS - Kyber: A CCA-Secure Module-Lattice-Based KEM. In *2018 IEEE European Symposium on Security and Privacy*. IEEE Computer Society Press, 353–367. doi:10.1109/EuroSP.2018.00032

[14] Daniel Bosk, Davide Frey, Mathieu Gestin, and Guillaume Piolle. 2022. Hidden Issuer Anonymous Credential. *PoPETs* 2022, 4 (Oct. 2022), 571–607. doi:10.56553/popets-2022-0123

[15] Sergiu Bursuc and Sjouke Mauw. 2022. Contingent payments from two-party signing and verification for abelian groups. In *CSF 2022 Computer Security Foundations Symposium*. IEEE Computer Society Press, 195–210. doi:10.1109/CSF54842.2022.9919674

[16] Theresa Calderon, Sarah Meiklejohn, Hovav Shacham, and Brent Waters. 2014. Rethinking Verifiably Encrypted Signatures: A Gap in Functionality and Potential Solutions. In *CT-RSA 2014 (LNCS, Vol. 8366)*, Josh Benaloh (Ed.). Springer, Cham, 349–366. doi:10.1007/978-3-319-04852-9_18

[17] Jan Camenisch, Maria Dubovitskaya, Kristiyan Haralambiev, and Markulf Kohlweiss. 2015. Composable and Modular Anonymous Credentials: Definitions and Practical Constructions. In *ASIACRYPT 2015, Part II (LNCS, Vol. 9453)*, Tetsu Iwata and Jung Hee Cheon (Eds.). Springer, Berlin, Heidelberg, 262–288. doi:10.1007/978-3-662-48800-3_11

[18] Jan Camenisch and Anna Lysyanskaya. 2001. An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation. In *EUROCRYPT 2001 (LNCS, Vol. 2045)*, Birgit Pfitzmann (Ed.). Springer, Berlin, Heidelberg, 93–118. doi:10.1007/3-540-44987-6_7

[19] Jan Camenisch and Anna Lysyanskaya. 2004. Signature Schemes and Anonymous Credentials from Bilinear Maps. In *CRYPTO 2004 (LNCS, Vol. 3152)*, Matthew Franklin (Ed.). Springer, Berlin, Heidelberg, 56–72. doi:10.1007/978-3-540-28628-8_4

[20] Matteo Campanelli, Rosario Gennaro, Steven Goldfeder, and Luca Nizzardo. 2017. Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services. In *ACM CCS 2017*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM Press, 229–243. doi:10.1145/3133956.3134060

[21] Ryann Cartor[1], Nathan Daly, Giulia Gaggero, Jason T LeGrow, Andrea Sanguineti, and Silvia Sconza. 2026. Post-quantum Adaptor Signatures with Strong Security from Cryptographic Group Actions. In *Post-Quantum Cryptography: 17th International Workshop, PQCrypto 2026, Saint-Malo, France, April 14–16, 2026, Proceedings, Part I*. Springer Nature, 370.

[22] Diego Castejon-Molina, Dimitrios Vasilopoulos, and Pedro Moreno-Sanchez. 2025. MixBuy: Contingent Payment in the Presence of Coin Mixers. *PoPETs* 2025, 1 (Jan. 2025), 671–706. doi:10.56553/popets-2025-0036

[23] Diego Castejon-Molina, Dimitrios Vasilopoulos, and Pedro Moreno-Sanchez. 2025. MixBuy: Contingent Payment in the Presence of Coin Mixers. In *Proceedings on Privacy Enhancing Technologies (PoPETs)*, Vol. 2025. 671–706. doi:10.56553/popets-2025-0036

[24] David Chaum. 1983. Blind Signature System. In *CRYPTO'83*, David Chaum (Ed.). Plenum Press, New York, USA, 153. doi:10.1007/978-1-4684-4730-9_14

[25] Rong Cheng, Bo Zhang, and Fangguo Zhang. 2011. Secure Obfuscation of Encrypted Verifiable Encrypted Signatures. In *ProvSec 2011 (LNCS, Vol. 6980)*, Xavier Boyen and Xiaofeng Chen (Eds.). Springer, Berlin, Heidelberg, 188–203. doi:10.1007/978-3-642-24316-5_14

[26] Chia Network Inc. 2022. Chia Business Whitepaper Version 2.0. https://www.chia.net/wp-content/uploads/2022/07/Chia-Business-Whitepaper-2022-02-02-v2.0.pdf Accessed: 2026-03-31.

[27] Aisling Connolly, Pascal Lafourcade, and Octavio Perez-Kempner. 2022. Improved Constructions of Anonymous Credentials from Structure-Preserving Signatures on Equivalence Classes. In *PKC 2022, Part I (LNCS, Vol. 13177)*, Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe (Eds.). Springer, Cham, 409–438. doi:10.1007/978-3-030-97121-2_15

[28] Jean-Sébastien Coron and David Naccache. 2003. Boneh et al.'s k-Element Aggregate Extraction Assumption Is Equivalent to the Diffie-Hellman Assumption. In *ASIACRYPT 2003 (LNCS, Vol. 2894)*, Chi-Sung Laih (Ed.). Springer, Berlin, Heidelberg, 392–397. doi:10.1007/978-3-540-40061-5_25

[29] Wei Dai, Tatsuaki Okamoto, and Go Yamamoto. 2022. Stronger Security and Generic Constructions for Adaptor Signatures. In *INDOCRYPT 2022 (LNCS, Vol. 13774)*, Takanori Isobe and Santanu Sarkar (Eds.). Springer, Cham, 52–77. doi:10.1007/978-3-031-22912-1_3

[30] Wei Dai, Tatsuaki Okamoto, and Go Yamamoto. 2022. Stronger Security and Generic Constructions for Adaptor Signatures. Cryptology ePrint Archive, Report 2022/1687. https://eprint.iacr.org/2022/1687

[31] Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. 2018. Privacy Pass: Bypassing Internet Challenges Anonymously. *PoPETs* 2018, 3 (July 2018), 164–180. doi:10.1515/popets-2018-0026

[32] Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. 2023. McFly: Verifiable Encryption to the Future Made Practical. In *FC 2023, Part I (LNCS, Vol. 13950)*, Foteini Baldimtsi and Christian Cachin (Eds.). Springer, Cham, 252–269. doi:10.1007/978-3-031-47754-6_15

[33] Stefan Dziembowski, Lisa Eckey, and Sebastian Faust. 2018. FairSwap: How To Fairly Exchange Digital Goods. In *ACM CCS 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM Press, 967–984. doi:10.1145/3243734.3243857

[34] Andreas Erwig, Sebastian Faust, Kristina Hostáková, Monosij Maitra, and Siavash Riahi. 2021. Two-Party Adaptor Signatures from Identification Schemes. In *PKC 2021, Part I (LNCS, Vol. 12710)*, Juan Garay (Ed.). Springer, Cham, 451–480. doi:10.1007/978-3-030-75245-3_17

[35] Muhammed F. Esgin, Oguzhan Ersoy, and Zekeriya Erkin. 2020. Post-Quantum Adaptor Signatures and Payment Channel Networks. In *ESORICS 2020, Part II (LNCS, Vol. 12309)*, Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider (Eds.). Springer, Cham, 378–397. doi:10.1007/978-3-030-59013-0_19

[36] Georg Fuchsbauer. 2019. WI Is Not Enough: Zero-Knowledge Contingent (Service) Payments Revisited. In *ACM CCS 2019*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM Press, 49–62. doi:10.1145/3319535.3354234

[37] Georg Fuchsbauer, Christian Hanser, Chethan Kamath, and Daniel Slamanig. 2016. Practical Round-Optimal Blind Signatures in the Standard Model from Weaker Assumptions. In *SCN 16 (LNCS, Vol. 9841)*, Vassilis Zikas and Roberto De Prisco (Eds.). Springer, Cham, 391–408. doi:10.1007/978-3-319-44618-9_21

[38] Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. 2019. Structure-Preserving Signatures on Equivalence Classes and Constant-Size Anonymous Credentials. *Journal of Cryptology* 32, 2 (April 2019), 498–546. doi:10.1007/s00145-018-9281-4

[39] Georg Fuchsbauer, Christian H. Hanser, and Daniel Slamanig. 2014. EUF-CMA-Secure Structure-Preserving Signatures on Equivalence Classes. *IACR Cryptol. ePrint Arch.* 2014 (2014), 944. https://api.semanticscholar.org/CorpusID:17538503

[40] Georg Fuchsbauer and Mathias Wolf. 2024. Concurrently Secure Blind Schnorr Signatures. In *EUROCRYPT 2024, Part II (LNCS, Vol. 14652)*, Marc Joye and Gregor Leander (Eds.). Springer, Cham, 124–160. doi:10.1007/978-3-031-58723-8_5

[41] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri Aravinda Krishnan Thyagarajan. 2024. Foundations of Adaptor Signatures. In *EUROCRYPT 2024, Part II (LNCS, Vol. 14652)*, Marc Joye and Gregor Leander (Eds.). Springer, Cham, 161–189. doi:10.1007/978-3-031-58723-8_6

[42] Noemi Glaeser, Matteo Maffei, Giulio Malavolta, Pedro Moreno-Sanchez, Erkan Tairi, and Sri Aravinda Krishnan Thyagarajan. 2022. Foundations of Coin Mixing Services. In *ACM CCS 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM Press, 1259–1273. doi:10.1145/3548606.3560637

[43] Huseyin Gokay, Foteini Baldimtsi, and Giuseppe Ateniese. 2024. Atomic Swaps for Boneh-Lynn-Shacham (BLS) Based Blockchains. In *ESORICS 2024, Part III (LNCS, Vol. 14984)*, Joaquin Garcia-Alfaro, Rafał Kozik, Michał Choraś, and Sokratis Katsikas (Eds.). Springer, Cham, 341–361. doi:10.1007/978-3-031-70896-1_17

[44] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. 1986. How to Construct Random Functions. *Journal of the ACM* 33, 4 (Oct. 1986), 792–807. doi:10.1145/6490.6503

[45] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. 1988. A Digital Signature Scheme Secure Against Adaptive Chosen-message Attacks. *SIAM J. Comput.* 17, 2 (April 1988), 281–308.

[46] Javier Gomez-Martinez, Dimitrios Vasilopoulos, Pedro Moreno-Sanchez, and Dario Fiore. 2025. Algebraic Zero Knowledge Contingent Payment. In *ACNS 2025, Part II (LNCS, Vol. 15826)*, Marc Fischlin and Veelasha Moonsamy (Eds.). Springer, Cham, 224–254. doi:10.1007/978-3-031-95764-2_10

[47] M. Choudary Gorantla and Ashutosh Saxena. 2006. Verifiably Encrypted Signature Scheme with Threshold Adjudication. Cryptology ePrint Archive, Report 2006/343. https://eprint.iacr.org/2006/343

[48] Vipul Goyal, Abhiram Kothapalli, Elisaweta Masserova, Bryan Parno, and Yifan Song. 2022. Storing and Retrieving Secrets on a Blockchain. In *PKC 2022, Part I (LNCS, Vol. 13177)*, Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe (Eds.). Springer, Cham, 252–282. doi:10.1007/978-3-030-97121-2_10

[49] Satoshi Hada. 2010. Secure Obfuscation for Encrypted Signatures. In *EUROCRYPT 2010 (LNCS, Vol. 6110)*, Henri Gilbert (Ed.). Springer, Berlin, Heidelberg, 92–112. doi:10.1007/978-3-642-13190-5_5

[50] Hossein Hafezi, Aditi Partap, Sourav Das, and Joseph Bonneau. 2025. Fair Signature Exchange. Cryptology ePrint Archive, Report 2025/059. https://eprint.iacr.org/2025/059

[51] Panpan Han, Zheng Yan, Laurence T. Yang, and Elisa Bertino. 2025. P2C2T: Preserving the Privacy of Cross-Chain Transfer. In *2025 IEEE Symposium on Security and Privacy*, Marina Blanton, William Enck, and Cristina Nita-Rotaru (Eds.). IEEE Computer Society Press, 1474–1492. doi:10.1109/SP61157.2025.00051

[52] Christian Hanser, Max Rabkin, and Dominique Schröder. 2015. Verifiably Encrypted Signatures: Security Revisited and a New Construction. In *ESORICS 2015, Part I (LNCS, Vol. 9326)*, Günther Pernul, Peter Y. A. Ryan, and Edgar R. Weippl (Eds.). Springer, Cham, 146–164. doi:10.1007/978-3-319-24174-6_8

[53] Christian Hanser and Daniel Slamanig. 2014. Structure-Preserving Signatures on Equivalence Classes and Their Application to Anonymous Credentials. In *ASIACRYPT 2014, Part I (LNCS, Vol. 8873)*, Palash Sarkar and Tetsu Iwata (Eds.). Springer, Berlin, Heidelberg, 491–511. doi:10.1007/978-3-662-45611-8_26

[54] Lucjan Hanzlik. 2023. Non-interactive Blind Signatures for Random Messages. In *EUROCRYPT 2023, Part V (LNCS, Vol. 14008)*, Carmit Hazay and Martijn Stam (Eds.). Springer, Cham, 722–752. doi:10.1007/978-3-031-30589-4_25

[55] Lucjan Hanzlik, Julian Loss, Sri Aravinda Krishnan Thyagarajan, and Benedikt Wagner. 2024. Sweep-UC: Swapping Coins Privately. In *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 3822–3839. doi:10.1109/SP54263.2024.00081

[56] Lucjan Hanzlik and Daniel Slamanig. 2021. With a Little Help from My Friends: Constructing Practical Anonymous Credentials. In *ACM CCS 2021*, Giovanni Vigna and Elaine Shi (Eds.). ACM Press, 2004–2023. doi:10.1145/3460120.3484582

[57] Thomas S. Heydt-Benjamin, Hee-Jin Chae, Benessa Defend, and Kevin Fu. 2006. Privacy for Public Transportation. In *PET 2006 (LNCS, Vol. 4258)*, George Danezis and Philippe Golle (Eds.). Springer, Berlin, Heidelberg, 1–19. doi:10.1007/11957454_1

[58] Joseph Jaeger and Akshaya Kumar. 2022. Memory-Tight Multi-challenge Security of Public-Key Encryption. In *ASIACRYPT 2022, Part III (LNCS, Vol. 13793)*, Shweta Agrawal and Dongdai Lin (Eds.). Springer, Cham, 454–484. doi:10.1007/978-3-031-22969-5_16

[59] Julia Kastner, Julian Loss, and Jiayu Xu. 2022. The Abe-Okamoto Partially Blind Signature Scheme Revisited. In *ASIACRYPT 2022, Part IV (LNCS, Vol. 13794)*, Shweta Agrawal and Dongdai Lin (Eds.). Springer, Cham, 279–309. doi:10.1007/978-3-031-22972-5_10

[60] Julia Kastner, Julian Loss, and Jiayu Xu. 2022. On Pairing-Free Blind Signature Schemes in the Algebraic Group Model. In *PKC 2022, Part II (LNCS, Vol. 13178)*, Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe (Eds.). Springer, Cham, 468–497. doi:10.1007/978-3-030-97131-1_16

[61] Shuichi Katsumata, Michael Reichle, and Yusuke Sakai. 2023. Practical Round-Optimal Blind Signatures in the ROM from Standard Assumptions. In *ASIACRYPT 2023, Part II (LNCS, Vol. 14439)*, Jian Guo and Ron Steinfeld (Eds.). Springer, Singapore, 383–417. doi:10.1007/978-981-99-8724-5_12

[62] Michael Klooß, Michael Reichle, and Benedikt Wagner. 2024. Practical Blind Signatures in Pairing-Free Groups. Cryptology ePrint Archive, Report 2024/1378. https://eprint.iacr.org/2024/1378

[63] Xiangyu Liu, Ioannis Tzannetos, and Vassilis Zikas. 2024. Adaptor Signatures: New Security Definition and a Generic Construction for NP Relations. In *ASIACRYPT 2024, Part II (LNCS, Vol. 15485)*, Kai-Min Chung and Yu Sasaki (Eds.). Springer, Singapore, 168–193. doi:10.1007/978-981-96-0888-1_6

[64] Steve Lu, Rafail Ostrovsky, Amit Sahai, Hovav Shacham, and Brent Waters. 2013. Sequential Aggregate Signatures, Multisignatures, and Verifiably Encrypted Signatures Without Random Oracles. *Journal of Cryptology* 26, 2 (April 2013), 340–373. doi:10.1007/s00145-012-9126-5

[65] Varun Madathil, Sri Aravinda Krishnan Thyagarajan, Dimitrios Vasilopoulos, Lloyd Fournier, Giulio Malavolta, and Pedro Moreno-Sanchez. 2023. Cryptographic Oracle-based Conditional Payments. In *NDSS 2023*. The Internet Society.

[66] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. 2019. Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability. In *NDSS 2019*. The Internet Society. doi:10.14722/ndss.2019.23330

[67] Rebekah Mercer, Kaoutar El Khiyaoui, Angelo De Caro, and Elli Androulaki. 2025. Efficient Aggregate Anonymous Credentials for Decentralized Identity. Cryptology ePrint Archive, Paper 2025/1724. https://eprint.iacr.org/2025/1724

[68] Omid Mir, Balthazar Bauer, Scott Griffy, Anna Lysyanskaya, and Daniel Slamanig. 2023. Aggregate Signatures with Versatile Randomization and Issuer-Hiding Multi-Authority Anonymous Credentials. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, 30–44. doi:10.1145/3576915.3623203

[69] Pedro Moreno-Sanchez, Arthur Blue, Duc Viet Le, Sarang Noether, Brandon Goodell, and Aniket Kate. 2020. DLSAG: Non-interactive Refund Transactions for Interoperable Payment Channels in Monero. In *FC 2020 (LNCS, Vol. 12059)*, Joseph Bonneau and Nadia Heninger (Eds.). Springer, Cham, 325–345. doi:10.1007/978-3-030-51280-4_18

[70] Ky Nguyen, Miguel Ambrona, and Masayuki Abe. 2020. WI is Almost Enough: Contingent Payment All Over Again. In *ACM CCS 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM Press, 641–656. doi:10.1145/3372297.3417888

[71] Ryo Nishimaki and Keita Xagawa. 2013. Verifiably Encrypted Signatures with Short Keys Based on the Decisional Linear Problem and Obfuscation for Encrypted VES. In *PKC 2013 (LNCS, Vol. 7778)*, Kaoru Kurosawa and Goichiro Hanaoka (Eds.). Springer, Berlin, Heidelberg, 405–422. doi:10.1007/978-3-642-36362-7_25

[72] European Parliament and Council of the European Union. 2024. Regulation (EU) 2024/1183 amending Regulation (EU) 910/2014 as regards establishing the European Digital Identity Framework. https://eur-lex.europa.eu/eli/reg/2024/1183/oj

[73] Xianrui Qin, Shimin Pan, Arash Mirzaei, Zhimei Sui, Oguzhan Ersoy, Amin Sakzad, Muhammed F. Esgin, Joseph K. Liu, Jiangshan Yu, and Tsz Hon Yuen. 2023. BlindHub: Bitcoin-Compatible Privacy-Preserving Payment Channel Hubs Supporting Variable Amounts. In *2023 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 2462–2480. doi:10.1109/SP46215.2023.10179427

[74] Siavash Riahi and Orfeas Stefanos Thyfronitis Litos. 2024. Bitcoin Clique: Channel-Free Off-Chain Payments Using Two-Shot Adaptor Signatures. In *ACNS 2024, Part III (LNCS, Vol. 14585)*, Christina Pöpper and Lejla Batina (Eds.). Springer, Cham, 28–50. doi:10.1007/978-3-031-54776-8_2

[75] Markus Rückert. 2009. Verifiably Encrypted Signatures from RSA without NIZKs. In *INDOCRYPT 2009 (LNCS, Vol. 5922)*, Bimal K. Roy and Nicolas Sendrier (Eds.). Springer, Berlin, Heidelberg, 363–377. doi:10.1007/978-3-642-10628-6_24

[76] Markus Rückert, Michael Schneider, and Dominique Schröder. 2010. Generic Constructions for Verifiably Encrypted Signatures without Random Oracles or NIZKs. In *ACNS 2010 (LNCS, Vol. 6123)*, Jianying Zhou and Moti Yung (Eds.). Springer, Berlin, Heidelberg, 69–86. doi:10.1007/978-3-642-13708-2_5

[77] Markus Rückert and Dominique Schröder. 2009. Security of Verifiably Encrypted Signatures and a Construction without Random Oracles. In *PAIRING 2009 (LNCS, Vol. 5671)*, Hovav Shacham and Brent Waters (Eds.). Springer, Berlin, Heidelberg, 17–34. doi:10.1007/978-3-642-03298-1_2

[78] Markus Rückert and Dominique Schroeder. 2013. Aggregate and Verifiably Encrypted Signatures from Multilinear Maps Without Random Oracles. Cryptology ePrint Archive, Report 2013/020. https://eprint.iacr.org/2013/020

[79] Olivier Sanders. 2020. Efficient Redactable Signature and Application to Anonymous Credentials. In *PKC 2020, Part II (LNCS, Vol. 12111)*, Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas (Eds.). Springer, Cham, 628–656. doi:10.1007/978-3-030-45388-6_22

[80] Olivier Sanders and Jacques Traoré. 2024. Compact Issuer-Hiding Authentication, Application to Anonymous Credential. *PoPETs* 2024, 3 (July 2024), 645–658. doi:10.56553/popets-2024-0097

[81] Jae Hong Seo, Keita Emura, Keita Xagawa, and Kazuki Yoneyama. 2015. Accumulable Optimistic Fair Exchange from Verifiably Encrypted Homomorphic Signatures. In *ACNS 2015 (LNCS, Vol. 9092)*, Tal Malkin, Vladimir Kolesnikov, Allison Bishop Lewko, and Michalis Polychronakis (Eds.). Springer, Cham, 192–214. doi:10.1007/978-3-319-28166-7_10

[82] Victor Shoup. 1997. Lower Bounds for Discrete Logarithms and Related Problems. In *EUROCRYPT'97 (LNCS, Vol. 1233)*, Walter Fumy (Ed.). Springer, Berlin, Heidelberg, 256–266. doi:10.1007/3-540-69053-0_18

[83] Erkan Tairi, Pedro Moreno-Sanchez, and Matteo Maffei. 2021. A^{2}L: Anonymous Atomic Locks for Scalability in Payment Channel Hubs. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 1834–1851. doi:10.1109/SP40001.2021.00111

[84] Erkan Tairi, Pedro Moreno-Sanchez, and Matteo Maffei. 2021. Post-Quantum Adaptor Signature for Privacy-Preserving Off-Chain Payments. In *FC 2021, Part II (LNCS, Vol. 12675)*, Nikita Borisov and Claudia Díaz (Eds.). Springer, Berlin, Heidelberg, 131–150. doi:10.1007/978-3-662-64331-0_7

[85] Erkan Tairi, Pedro Moreno-Sanchez, and Clara Schneidewind. 2023. LedgerLocks: A Security Framework for Blockchain Protocols Based on Adaptor Signatures. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, 859–873. doi:10.1145/3576915.3623149

[86] Ertem Nusret Tas, István András Seres, Yinuo Zhang, Márk Melczer, Mahimna Kelkar, Joseph Bonneau, and Valeria Nikolaenko. 2024. Atomic and Fair Data Exchange via Blockchain. In *ACM CCS 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM Press, 3227–3241. doi:10.1145/3658644.3690248

[87] Sri Aravinda Krishnan Thyagarajan and Giulio Malavolta. 2021. Lockable Signatures for Blockchains: Scriptless Scripts for All Signatures. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 937–954. doi:10.1109/SP40001.2021.00065

[88] Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, and Pedro Moreno-Sanchez. 2022. Universal Atomic Swaps: Secure Exchange of Coins Across All Blockchains. In *2022 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 1299–1316. doi:10.1109/SP46214.2022.9833731

[89] Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, Fritz Schmidt, and Dominique Schröder. 2020. PayMo: Payment Channels For Monero. Cryptology ePrint Archive, Report 2020/1441. https://eprint.iacr.org/2020/1441

[90] Nikhil Vanjani, Pratik Soni, and Sri Aravinda Krishnan Thyagarajan. 2024. Functional Adaptor Signatures: Beyond All-or-Nothing Blockchain-based Payments. In *ACM CCS 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM Press, 1493–1507. doi:10.1145/3658644.3690240

[91] Fangguo Zhang, Reihaneh Safavi-Naini, and Willy Susilo. 2003. Efficient Verifiably Encrypted Signature and Partially Blind Signature from Bilinear Pairings. In *INDOCRYPT 2003 (LNCS, Vol. 2904)*, Thomas Johansson and Subhamoy Maitra (Eds.). Springer, Berlin, Heidelberg, 191–204. doi:10.1007/978-3-540-24582-7_14

[92] Lei Zhang, Qianhong Wu, and Bo Qin. 2009. Identity-Based Verifiably Encrypted Signatures without Random Oracles. In *ProvSec 2009 (LNCS, Vol. 5848)*, Josef Pieprzyk and Fangguo Zhang (Eds.). Springer, Berlin, Heidelberg, 76–89. doi:10.1007/978-3-642-04642-1_8

A Additional Preliminaries

A.1 Hard Relation

We first recall the definition of a hard relation.

Definition A.1 (Hard Relation). *Let R be a relation with statement/witness pairs (Y, y). Let us denote L the associated language defined as $L := \{Y \mid \exists y \text{ s.t. } (Y, y) \in R\}$. We say that R is a hard relation if the following holds:*

- *There exists a* PPT *sampling algorithm* $\mathsf{GenR}(1^\lambda)$ *that on input the security parameter* 1^λ *outputs a statement/witness pair* $(Y, y) \in R$.
- *The relation is poly-time decidable.*
- *For all* PPT *adversaries* $\mathcal{A}$ *there exists a negligible function* negl, *such that:*

$$\Pr\left[(Y, y^*) \in R \,\middle|\, \begin{array}{l}(Y, y) \leftarrow \mathsf{GenR}(1^\lambda)\\ y^* \leftarrow \mathcal{A}(Y)\end{array}\right] \leq \mathsf{negl}(\lambda),$$

A.2 Hardness Assumptions

Definition A.2 (co-CDH). *We say that the co-computational Diffie-Hellman (co-CDH) assumption holds relative to the bilinear group generator* BGGen, *if for all* PPT *adversaries* $\mathcal{A}$, *there exists a negligible function* $\mathsf{negl}(\cdot)$, *such that*

$$\Pr\left[\ \mathcal{A}(\mathsf{pp}, g_2^a, h) = h^a \ \middle|\ \begin{array}{l}\mathsf{pp} \leftarrow \mathsf{BGGen}(1^\lambda)\\ h \leftarrow\$ \mathbb{G}_1, a \leftarrow\$ \mathbb{Z}_q\end{array}\right] \leq \mathsf{negl}(\lambda),$$

where $\mathsf{pp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, \psi)$.

Definition A.3 (Chosen target co-CDH). *We say that the chosen-target co-computational Diffie-Hellman (CT-co-CDH) assumption holds relative to the bilinear group generator* BGGen, *if for all* PPT *adversaries* $\mathcal{A}$, *there exists a negligible function* $\mathsf{negl}(\cdot)$, *such that*

$$\Pr[\mathsf{CT\text{-}co\text{-}CDH}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{CT\text{-}co\text{-}CDH}_{\mathcal{A}}$ *is defined as follows:*

$\mathsf{CT\text{-}co\text{-}CDH}_{\mathcal{A}}(\lambda)$

1: $\mathsf{pp} \leftarrow \mathsf{BGGen}(1^\lambda)$
2: **parse** $\mathsf{pp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, \psi)$
3: $x \leftarrow\$ \mathbb{Z}_q, \mathsf{cid} := 0, \ell := 0$
4: $(j_i, Z_i)_{i \in [\ell+1]} \leftarrow \mathcal{A}^{\mathcal{O}_{\mathrm{Chal}}, \mathcal{O}_{\mathrm{DH}}(\cdot)}(\mathsf{pp}, g_2^x)$
5: $b_1 = \forall i \neq j \in [\ell+1], Z_i \neq Z_j$
6: $b_2 = \forall i \in [\ell+1], Z_i = Y_{j_i}^x$
7: **return** $(b_1 \wedge b_2)$

$\mathcal{O}_{\mathrm{Chal}}()$

1: $\mathsf{cid} := \mathsf{cid} + 1$
2: $Y_{\mathsf{cid}} \leftarrow\$ \mathbb{G}_1$
3: **return** Y_{cid}

$\mathcal{O}_{\mathrm{DH}}(Y)$

1: $\ell := \ell + 1$
2: **return** Y^x

The bilinear aggregate extraction assumption was first introduced by Boneh et al. [12], and subsequently proven by Coron and Naccache [28] that it is equivalent to co-CDH assumption. It is formally defined as follows.

Definition A.4 (BLS aggregate extraction). *Let* $\mathsf{pp} := (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, \psi)$ *be bilinear group parameters. Consider k-user aggregation and let $I := \{1, ..., k\}$. Each user has a private key $x_i \in \mathbb{Z}_q$, a public key $v_i := g_2^{x_i}$, and selects a distinct message $m_i \in \{0,1\}^*$, whose hash is $h_i := g_1^{y_i} \in \mathbb{G}_1$. Each user creates a BLS signature $\sigma_i := h_i^{x_i} \in \mathbb{G}_1$. Note that each user signature can be expressed as $\sigma_i := g_1^z := g_1^{x_i y_i}$. The signatures for all users are aggregated, which yields $\sigma := \prod_{i \in I} \sigma_i := g_1^{\sum_{i \in I} x_i y_i} \in \mathbb{G}_1$. Then, for any* PPT *extractor algorithm $\mathcal{E}$, there exists a negligible function* $\mathsf{negl}(\cdot)$,

such that.

$$\Pr\left[\begin{array}{c|r} & x_1,\ldots,x_k,\, y_1,\ldots,y_k \leftarrow\$ \mathbb{Z}_q \\ & \sigma \coloneqq g_1^{\sum_{i\in I} x_i y_i} \\ b_0 = 1 & \\ \wedge & (\sigma', I') \leftarrow \mathcal{E}(g_2^{x_1},\ldots,g_2^{x_k}, g_1^{y_1},\ldots,g_1^{y_k}, \sigma) \\ b_1 = 1 & b_0 \coloneqq \emptyset \neq I' \subsetneq I \\ & b_1 \coloneqq \sigma' \stackrel{?}{=} g_1^{\sum_{i\in I'} x_i y_i} \end{array}\right] \leq \mathsf{negl}(\lambda).$$

A.3 Pseudorandom Function

A pseudorandom function (PRF) [44] F is a deterministic algorithm that has two inputs: a key k from the key space $\mathcal{K}$ and an input x from the input space $\mathcal{X}$. It outputs $y \coloneqq \mathsf{F}(k, x)$ from the output space $\mathcal{Y}$.

Definition A.5 (Secure PRF). *A PRF is secure if for all* PPT *adversaries $\mathcal{A}$, a random function* f *from $\mathcal{X}$ to $\mathcal{Y}$, and a negligible function* negl$(\cdot)$*, it holds* $\Pr[\mathsf{ExpPRF}(\lambda)] \leq 1/2 + \mathsf{negl}(\lambda)$*, where* ExpPRF *is defined as follows:*

$\mathsf{ExpPRF}(\lambda)$	$\mathcal{O}_b(x)$
$k \leftarrow\$ \mathcal{K}$	**if** $b = 0$ **then**
$b \leftarrow\$ \{0,1\}$	**return** $\mathsf{F}(k, x)$
$b' \leftarrow \mathcal{A}^{\mathcal{O}_b}(\lambda)$	**else**
return $b = b'$	**return** $\mathsf{f}(x)$

A.4 Digital Signature Security

Definition A.6 (EUF-CMA Security). *Σ is* EUF-CMA *secure if for every* PPT *adversary $\mathcal{A}$ there exists a negligible function* negl *such that* $\Pr[\mathsf{ExpForge}_{\mathcal{A},\Sigma}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$ *where* $\mathsf{ExpForge}_{\mathcal{A},\Sigma}$ *is defined as follows:*

$\mathsf{ExpForge}_{\mathcal{A},\Sigma}(\lambda)$	$\mathcal{O}\mathsf{Sign}(m)$
$Q \coloneqq \emptyset$	$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$	$Q \coloneqq Q \cup \{m\}$
$(m, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{Sign}}(\mathsf{pk})$	**return** σ
return $(m \notin Q \wedge \mathsf{Vf}(\mathsf{pk}, m, \sigma))$	

A.5 Verifiably Encrypted Signature Security

Definition A.7 (Completeness [12]). *A VES scheme Σ_{VES} is complete if for all $\lambda \in \mathbb{N}$, for all* $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$*, for all* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$*, for all messages m, and for all $\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$, it holds that*

$$\Pr[\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega) = 1] = 1 \quad \textit{and}$$
$$\Pr[\mathsf{Vf}(\mathsf{pk}, m, \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)) = 1] = 1.$$

Definition A.8 (Unforgeability [12]). *A VES scheme Σ_{VES} is unforgeable if for all* PPT *adversaries $\mathcal{A}$ there is a negligible function* negl$(\cdot)$*, such that* $\Pr[\mathsf{ExpForge}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda)] \leq \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpForge}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *is defined in Figure 13.*

Definition A.9 (Opacity [12]). *A VES scheme Σ_{VES} is opaque if for all* PPT *adversaries $\mathcal{A}$ there is a negligible function* negl$(\cdot)$*, such that* $\Pr[\mathsf{ExpOpacity}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda)] \leq \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpOpacity}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *is defined as* $\mathsf{ExpMKOpacity}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *in Figure 3, without the gray lines.*

Definition A.10 (Extractability [77]). *A VES scheme Σ_{VES} is extractable if for all* PPT *adversaries $\mathcal{A}$ there is a negligible function* negl$(\cdot)$*, such that* $\Pr[\mathsf{ExpExtract}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda)] \leq \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpExtract}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *is defined as* $\mathsf{ExpCKExtract}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *in Figure 3 without the gray lines.*

Definition A.11 (Abuse freeness [77]). *A VES scheme Σ_{VES} is abuse free if for all* PPT *adversaries $\mathcal{A}$ there is a negligible function* negl$(\cdot)$*, such that* $\Pr[\mathsf{ExpAbuseFree}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda)] \leq \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpAbuseFree}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ *is defined in Figure 13.*

Definition A.12 (Resolution independence [16]). *A VES scheme Σ_{VES} is resolution independent if for all $\lambda \in \mathbb{N}$, for all* $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$*, for all* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$*, for all messages m, it holds that*

$$[\mathsf{Sign}(\mathsf{sk}, m)] \ \textit{and}\ [\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m))],$$

are identically distributed.

Definition A.13 (Resolution duplication [16]). *A VES scheme Σ_{VES} is resolution duplicate if (1) it is resolution independent, (2)* Resolve *is a* DPT *algorithm, and (3) there exists a* DPT *algorithm* Extract*, such that for all* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$*, for all* $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$*, for all m, and for all $r \in \{0,1\}^*$, it holds that*

$$\mathsf{Extract}(m, \mathsf{sk}, r) = \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m; r)).$$

Definition A.14 (Secure verifiably encrypted signature). *A VES scheme Σ_{VES} is secure, if it is complete, unforgeable, opaque, extractable, abuse free and resolution independent. It is strongly secure if it also satisfies resolution duplication.*

A.6 Non-Interactive Zero-Knowledge Argument

Let R be a binary relation as defined above, with L the language consisting of statements in R. A non-interactive zero-knowledge (NIZK) argument system [10] for a language L allows to prove in a non-interactive manner that some statements are in L without leaking information about the corresponding witnesses. We formally define it as follows.

Definition A.15 (Non-Interactive Zero-Knowledge Argument System). *A non-interactive zero-knowledge (NIZK) argument system* NIZK *for a language $L \in$* NP *(with witness relation R) is a tuple of* PPT *algorithms* NIZK = (PGen, P, V)*, such that:*

- $\underline{\mathsf{PGen}(1^\lambda)}$*: on input a security parameter 1^λ, outputs a common reference string* crs*.*
- $\underline{\mathsf{P}(\mathsf{crs}, x, w)}$*: on input a common reference string* crs*, a statement x and a witness w, outputs a proof π.*
- $\underline{\mathsf{V}(\mathsf{crs}, x, \pi)}$*: on input a common reference string* crs*, a statement x and a proof π, outputs a bit b.*

We require NIZK *to meet the following properties:*

Completeness. For every $(x, w) \in R$ we have that

$$\Pr\Big[\mathsf{V}(\mathsf{crs}, x, \pi) = 1 \mid \mathsf{crs} \leftarrow \mathsf{PGen}(1^\lambda), \pi \leftarrow \mathsf{P}(\mathsf{crs}, x, w)\Big] = 1.$$

Knowledge soundness. There exists a PPT *extractor $\mathcal{E} = (\mathcal{E}_1, \mathcal{E}_2)$ such that for every* PPT *distinguisher $\mathcal{D}$ it holds*

$$\Big|\Pr\Big[\mathcal{D}(\mathsf{crs}) = 1 \mid \mathsf{crs} \leftarrow \mathsf{PGen}(1^\lambda)\Big] - \Pr\Big[\mathcal{D}(\mathsf{crs}) = 1 \mid (\mathsf{crs}, \tau_s) \leftarrow \mathcal{E}_1(1^\lambda)\Big]\Big| \leq \mathsf{negl}(\lambda),$$

$\mathsf{ExpForge}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda)$

1 : $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
3 : $Q_{\mathsf{S}} \coloneqq Q_{\mathsf{VS}} \coloneqq Q_{\mathsf{R}} \coloneqq \emptyset$
4 : $\mathcal{O} \coloneqq \{\mathsf{VESign}, \mathsf{Resolve}, \mathsf{Sign}\}$
5 : $(m^*, \omega^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk}, \mathsf{apk})$
6 : $b_0 \coloneqq m^* \notin \{Q_{\mathsf{S}}, Q_{\mathsf{VS}}, Q_{\mathsf{R}}\}$
7 : $b_1 \coloneqq \mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m^*, \omega^*) = 1$
8 : **return** $b_0 \wedge b_1$

$\mathcal{O}\mathsf{Sign}(m)$

1 : $\sigma \coloneqq \mathsf{Sign}(\mathsf{sk}, m)$
2 : $Q_{\mathsf{S}} \coloneqq Q_{\mathsf{S}} \cup \{m\}$
3 : **return** σ

$\mathcal{O}\mathsf{VESign}(m)$

1 : $\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$
2 : $Q_{\mathsf{VS}} \coloneqq Q_{\mathsf{VS}} \cup \{m\}$
3 : **return** ω

$\mathcal{O}\mathsf{Resolve}(m, \omega)$

1 : **if** $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega) = 0$
2 : **return** $\perp$
3 : $\sigma \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)$
4 : $Q_{\mathsf{R}} \coloneqq Q_{\mathsf{R}} \cup \{m\}$
5 : **return** σ

$\mathsf{ExpAbuseFree}_{\mathcal{A},\Sigma_{\mathsf{VES}}}(\lambda))$

1 : $Q_{\mathsf{VS}} \coloneqq \emptyset$
2 : $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$
3 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
4 : $(m^*, \omega^*) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{VESign}}(\mathsf{pk}, \mathsf{ask}, \mathsf{apk})$
5 : $b_0 \coloneqq m^* \notin Q_{\mathsf{VS}}$
6 : $b_1 \coloneqq \mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m^*, \omega^*) = 1$
7 : **return** $b_0 \wedge b_1$

Figure 13: Experiments $\mathsf{ExpForge}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ and $\mathsf{ExpAbuseFree}_{\mathcal{A},\Sigma_{\mathsf{VES}}}$ for VES security.

and for every PPT *adversary* $\mathcal{A}$ *it holds*

$$\Pr\left[\begin{array}{c} \mathsf{V}(\mathsf{crs}, x, \pi) = 1 \\ \wedge \\ (x, w) \notin R \end{array} \middle| \begin{array}{r} (\mathsf{crs}, \tau_s) \leftarrow \mathcal{E}_1(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\mathsf{crs}) \\ w \leftarrow \mathcal{E}_2(\mathsf{crs}, \tau_s, x, \pi) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

Computational zero-knowledge. There exists a PPT *algorithm* $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ *such that for every* PPT *adversary* $\mathcal{A}$,

$$\left|\Pr\left[\mathcal{A}^{\mathsf{P}(\mathsf{crs},\cdot,\cdot)}(\mathsf{crs}) = 1 \mid \mathsf{crs} \leftarrow \mathsf{PGen}(1^\lambda)\right]\right.$$
$$\left.-\Pr\left[\mathcal{A}^{\mathcal{O}(\mathsf{crs},\tau,\cdot,\cdot)}(\mathsf{crs}) = 1 \mid (\mathsf{crs}, \tau) \leftarrow \mathcal{S}_1(1^\lambda)\right]\right| \leq \mathsf{negl}(\lambda),$$

where $\mathcal{O}(\mathsf{crs}, \tau, \cdot, \cdot)$ *is an oracle that outputs* $\perp$ *on input* (x, w) *when* $(x, w) \notin R$ *and outputs* $\pi \leftarrow \mathcal{S}_2(\mathsf{crs}, \tau, x)$ *when* $(x, w) \in R$.

A.7 Public Key Encryption

Next, we recall the definition of a public key encryption (PKE) scheme.

Definition A.16 (Public Key Encryption). *A public key encryption (PKE) scheme* PKE *consists of three algorithms* PKE = (KGen, Enc, Dec) *defined as:*

- $\underline{\mathsf{KGen}(1^\lambda)}$*: is a* PPT *algorithm that on input a (unary encoded) security parameter* 1^λ*, outputs a secret/public key pair* (sk, pk).
- $\underline{\mathsf{Enc}(\mathsf{pk}, m)}$*: is a* PPT *algorithm that on input a public key* pk *and a message* m*, outputs a ciphertext* c.
- $\underline{\mathsf{Dec}(\mathsf{sk}, c)}$*: is a* DPT *algorithm that on input a secret key* sk *and a ciphertext* c*, outputs a message* m *or* $\perp$*, where* $\perp$ *denotes the error symbol.*

Correctness. For correctness we require that for every $\lambda \in \mathbb{N}$, for every m and every $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$, we have that

$$\Pr\left[\mathsf{Dec}(\mathsf{sk}, \mathsf{Enc}(\mathsf{pk}, m)) = m\right] = 1.$$

Security. We consider multi-user and multi-challenge indistinguishability against chosen ciphertext attacks (IND-CCA) security for the PKE scheme as defined in [58].

Definition A.17 (Multi-user and multi-challenge IND-CCA Security [58]). *A public key encryption scheme* PKE *is multi-user and multi-challenge IND-CCA secure, for some* $n = \mathsf{poly}(\lambda)$*, if for every* PPT *adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that the following holds:*

$$\Pr\left[\mathsf{IND\text{-}CCA}^n_{\mathcal{A},\mathsf{PKE}}(\lambda) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{IND\text{-}CCA}^n_{\mathcal{A},\mathsf{PKE}}$ *is defined as follows*

$\mathsf{IND\text{-}CCA}^n_{\mathcal{A},\mathsf{PKE}}(\lambda)$

1 : $b \leftarrow\$ \{0, 1\}$, $Q_{\mathsf{C}} \coloneqq [\cdot]$, $Q_{\mathsf{M}} \coloneqq [\cdot]$
2 : $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{KGen}(1^\lambda)$ *for* $i \in [n]$
3 : $b' \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{NewPK},\mathcal{O}\mathsf{Enc},\mathcal{O}\mathsf{Dec}}(1^\lambda)$
4 : **return** $b = b'$

$\mathcal{O}\mathsf{Dec}(k, c)$

1 : **if** $Q_{\mathsf{M}}[k, c] \neq \perp$ **then**
2 : **return** $Q_{\mathsf{M}}[k, c]$
3 : $m \coloneqq \mathsf{Dec}(sk_k, c)$
4 : **return** m

$\mathcal{O}\mathsf{NewPK}(k)$

1 : **if** $k > n$ **then**
2 : **return** $\perp$
3 : **return** pk_k

$\mathcal{O}\mathsf{Enc}(k, i, m)$

1 : **if** $Q_{\mathsf{C}}[k, i, m] \neq \perp$ **then**
2 : **return** $Q_{\mathsf{C}}[k, i, m]$
3 : $c_0 \leftarrow \mathsf{Enc}(\mathsf{pk}_k, 0^{|m|})$
4 : $c_1 \leftarrow \mathsf{Enc}(\mathsf{pk}_k, m)$
5 : $Q_{\mathsf{M}}[k, c_b] \coloneqq m$
6 : $Q_{\mathsf{C}}[k, i, m] \coloneqq c_b$
7 : **return** c_b

A.8 (Tagged) Structure-Preserving Signature on Equivalence Classes

We restate the syntax of (tagged) structure-preserving signatures on equivalence classes from [37, 39, 53].

Definition A.18 (Tagged structure-preserving signature on equivalence classes (TSPS-EQ)). *A TSPS-EQ scheme* TSPSEQ *with a tag space* $\mathcal{T}$ *consists on the following polynomial-time algorithms:*

- $\underline{\mathsf{Setup}_{\mathsf{R}}(1^\lambda)}$*:* PPT *algorithm that on input a security parameter* λ*, outputs public parameters* pp*. (We assume* pp *is an implicit input to all the other algorithms.)*
- $\underline{\mathsf{KeyGen}_{\mathsf{R}}(\ell)}$*: is a* PPT *algorithm that on input a vector length* $\ell \in \mathbb{N}^+$*, it outputs a key pair* (sk, pk).
- $\underline{\mathsf{Sign}_{\mathsf{R}}(\mathsf{sk}, M, t)}$*: is a* PPT *algorithm that on input a secret key* sk*, a representative* $M \in (\mathbb{G}^*)^\ell$ *of an equivalence class* $[M]$ *and a tag* $t \in \mathcal{T}$*, it outputs a signature* σ *for the representative* M *of the equivalence class.*

- $\underline{\mathsf{ChgRep}_{\mathsf{R}}(M, \sigma, \alpha, \mathsf{pk})}$*: is a* PPT *algorithm which on input a representative* $M \in (\mathbb{G}^*)^\ell$ *of an equivalence class* $[M]$*, a signature* σ *for* M*, a scalar value* α *and a public key* pk*, returns an updated message-signature pair* (M', σ')*, where* $M' = M^\alpha$ *is the new representative and* σ' *its updated signature.*
- $\underline{\mathsf{Verify}_{\mathsf{R}}(\mathsf{pk}, M, t, \sigma)}$*: is a* DPT *algorithm that on input a public key* pk*, a representative* $M \in (\mathbb{G}^*)^\ell$ *of an equivalence class* $[M]$*, a tag* $t \in \mathcal{T}$ *and a signature* σ*, it outputs* 1 *if the signature is valid or* 0 *otherwise.*
- $\underline{\mathsf{VKey}_{\mathsf{R}}(\mathsf{sk}, \mathsf{pk})}$*: is a* DPT *algorithm which given a secret key* sk *and a public key* pk *checks both keys for consistency and returns* 1 *on success and* 0 *otherwise.*

We obtain *structure-preserving signatures on equivalence classes (SPS-EQ)* by ignoring the tag $t \in \mathcal{T}$ or setting it to a predefined value, e.g., $t = \perp$. In the rest of the definitions we consider the more generic tagged version.

Definition A.19 (Perfect correctness). *A TSPS-EQ scheme* TSPSEQ *over* $\mathbb{G}$ *is perfectly correct if for all* $\lambda \in \mathbb{N}$*, for all* $\ell > 1$*, for all public parameters* $\mathsf{pp} \leftarrow \mathsf{Setup}_R(1^\lambda)$*, for all key pairs* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}_{\mathsf{R}}(\ell)$*, for all tag/message pairs* $(t, M) \in (\mathcal{T} \times (\mathbb{G}^*)^\ell)$ *and for all scalars* $\alpha \in \mathbb{Z}_q^*$*, we have*

$$\mathsf{VKey}_{\mathsf{R}}(\mathsf{sk}, \mathsf{pk}) = 1 \quad \textit{and}$$
$$\Pr\left[\mathsf{Verify}_{\mathsf{R}}(\mathsf{pk}, M, t, \mathsf{Sign}_{\mathsf{R}}(\mathsf{sk}, M, t)) = 1\right] = 1 \quad \textit{and}$$
$$\Pr\left[\mathsf{Verify}_{\mathsf{R}}(\mathsf{pk}, M^\alpha, t, \mathsf{ChgRep}_{\mathsf{R}}(M, \mathsf{Sign}_{\mathsf{R}}(\mathsf{sk}, M, t), \alpha, \mathsf{pk})) = 1\right] = 1.$$

Definition A.20 (EUF-CMA security). *A TSPS-EQ scheme* TSPSEQ *is EUF-CMA secure if for any* $\ell > 1$*, for all* PPT *algorithms* $\mathcal{A}$ *having access to the a signing oracle* $\mathcal{O}(\mathsf{sk}, M, t)$ *there is a negligible function* $\mathsf{negl}(\lambda)$ *such that:*

$$\Pr\left[\begin{array}{c} b_0 = 1 \\ \wedge \\ b_1 = 1 \end{array} \middle| \begin{array}{r} \mathsf{pp} \leftarrow \mathsf{Setup}_{\mathsf{R}}(1^\lambda) \\ (\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}_{\mathsf{R}}(\ell) \\ (M^*, \sigma^*, t^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk}) \\ b_0 \coloneqq \forall (M, t) \in Q_{\mathsf{M}}: \\ ([M^*], t^*) \neq ([M], t) \\ b_1 \coloneqq \mathsf{Verify}_{\mathsf{R}}(\mathsf{pk}, M^*, t^*, \sigma^*) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

We refer the reader to [38, 52] for the description of the remaining security properties.

A.9 Generic Group Model

We consider the *generic group model* (GGM) [82]. Loosely speaking, the generic group model is a model to study the runtime of generic algorithms in cyclic groups. Such algorithms do not exploit any special structure of the representation of the group elements. Instead, they are only allowed to perform abstract group operations and test whether two group elements are equal; they thus work for any group. This is modelled by providing group operations to an algorithm solely via oracles. In particular, we consider oracles for obtaining encodings of group elements, performing group operations as well as equality checks on encodings of group elements. In the bilinear group setting, we consider all three groups $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ as generic and algorithms have additionally access to a pairing oracle.

A.10 (Partially) Blind Signature

We restate partially blind signatures following the definition given in [61].

Definition A.21 (Partially blind signature scheme). *A partially blind signature scheme with a tag space* $\mathcal{T}$ *and a message space* $\mathcal{M}$ *is a tuple of algorithms* $\Sigma_B = (\mathsf{KGen}, \mathsf{User}, \mathsf{Signer}, \mathsf{Derive})$ *such that:*

- $\underline{\mathsf{KGen}(1^\lambda)}$*: is a* PPT *algorithm that on input a (unary encoded) security parameter* λ*, it outputs a secret and public keys* $(\mathsf{sk}, \mathsf{pk})$*.*
- $\underline{\mathsf{User}(\mathsf{pk}, t, m)}$*: is a* PPT *algorithm that on input a public key* pk*, a tag* $t \in \mathcal{T}$ *and a message* $m \in \mathcal{M}$*, it outputs a user message* ρ_U *and a state* st*.*
- $\underline{\mathsf{Signer}(\mathsf{sk}, t, \rho_U)}$*: is a* PPT *algorithm that on input a secret key* sk*, a tag* $t \in \mathcal{T}$ *and a user message* ρ_U*, it outputs a signer message* ρ_S*.*
- $\underline{\mathsf{Derive}(\mathsf{st}, t, \rho_S)}$*: is a* PPT *(or potentially* DPT*) algorithm that on input a state* st*, a tag* $t \in \mathcal{T}$ *and a signer message* ρ_S*, it outputs a signature* σ*.*
- $\underline{\mathsf{Vf}(\mathsf{pk}, t, m, \sigma)}$*: is a* DPT *algorithm that on input a public key* pk*, a tag* $t \in \mathcal{T}$*, a message* $m \in \mathcal{M}$ *and a signature* σ*, it outputs a bit* $b \in \{0, 1\}$*.*

We retrieve the standard definition of *blind signature scheme* by ignoring the tag $t \in \mathcal{T}$ or alternatively setting it to a predefined value (e.g., $\perp$). In the rest of the definitions, we keep it generic and include the tag.

Definition A.22 (Perfect correctness). *A partially blind signature scheme* Σ_B *is perfectly correct if for every* $\lambda \in \mathbb{N}$*, all secret/public key pairs* $(\mathsf{sk}, \mathsf{pk}) \in \mathsf{KGen}(1^\lambda)$ *and every tag/message pair* $(t, m) \in \mathcal{T} \times \mathcal{M}$*, we have that*

$$\Pr\left[\mathsf{Vf}(\mathsf{pk}, t, m, \sigma) = 1 \middle| \begin{array}{r} (\mathsf{st}, \rho_U) \leftarrow \mathsf{User}(\mathsf{pk}, t, m) \\ \rho_S \leftarrow \mathsf{Signer}(\mathsf{sk}, t, \rho_U) \\ \sigma \leftarrow \mathsf{Derive}(\mathsf{st}, t, \rho_S) \end{array}\right] = 1.$$

Partial blindness roughly requires the transcript to be independent of the signature even if the signer chooses the keys maliciously.

Definition A.23 (Partial blindness under chosen keys). *A partially blind signature scheme* Σ_B *is partially blind under chosen keys if for every* PPT *adversary* $\mathcal{A}$ *there is a negligible function* $\mathsf{negl}(\cdot)$*, such that* $\Pr[\mathsf{ExpBlind}_{\mathcal{A},\Sigma_B}(\lambda)] \leq 1/2 + \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpBlind}_{\mathcal{A},\Sigma_B}$ *is defined in Figure 14.*

One-more unforgeability ensures that at most one valid signature is generated after each execution of signer algorithm.

Definition A.24 (One-more unforgeability). *A partially blind signature scheme* Σ_B *is one-more unforgeable, if for any* $q = \mathsf{poly}(\lambda)$ *and every* PPT *adversary* $\mathcal{A}$ *there is a negligible function* $\mathsf{negl}(\cdot)$*, such that* $\Pr[\mathsf{ExpOMForge}_{\mathcal{A},\Sigma_B}(\lambda)] \leq \mathsf{negl}(\lambda)$*, where* $\mathsf{ExpOMForge}_{\mathcal{A},\Sigma_B}$ *is defined in Figure 14.*

Definition A.25 (Secure PBS). *A PBS scheme* Σ_B *is secure, if it is (perfectly) correct, partially blind under chosen keys and one-more unforgeable.*

$\mathsf{ExpOMForge}_{\mathcal{A},\Sigma_B}(\lambda)$	$\mathsf{Signer}(t, \rho_U)$	$\mathsf{ExpBlind}_{\mathcal{A},\Sigma_B}(\lambda)$
$Q_{\mathsf{M}} := [\cdot]$	$\rho_S \leftarrow \mathsf{Signer}(\mathsf{sk}, t, \rho_U)$	$(\mathsf{pk}, t, m_0, m_1) \leftarrow \mathcal{A}(1^\lambda), c \leftarrow\$ \{0,1\}$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$	**if** $Q_{\mathsf{M}}[t] = \bot$	$\forall b \in \{0,1\}$: $(\mathsf{st}_b, \rho_U^b) \leftarrow \mathsf{User}(\mathsf{pk}, t, m_b)$
$\{t, (m_i, \sigma_i)\}_{i \in [q]} \leftarrow \mathcal{A}^{\mathsf{Signer}}(\mathsf{pk})$	$\quad Q_{\mathsf{M}}[t] = 1$	$(\rho_S^c, \rho_S^{1-c}) \leftarrow \mathcal{A}(\rho_U^c, \rho_U^{1-c})$
$b_0 := \forall i \neq j \in [q]: m_i \neq m_j$	**else**	$\forall b \in \{0,1\}$: $\sigma_b \leftarrow \mathsf{Derive}(\mathsf{st}_b, t, \rho_S^b)$
$b_1 := \forall i \in [q]: \mathsf{Vf}(\mathsf{pk}, t, m_i, \sigma_i) = 1$	$\quad Q_{\mathsf{M}}[t] = Q_{\mathsf{M}}[t] + 1$	**if** $\exists b$ s.t. $\mathsf{Vf}(\mathsf{pk}, t, m_b, \sigma_b) = 0$ **then**
$b_2 := Q_{\mathsf{M}}[t] \leq q - 1$	**return** ρ_S	$\quad \sigma_0 = \sigma_1 = \bot$
return $b_0 \wedge b_1 \wedge b_2$		$c^* \leftarrow \mathcal{A}(\sigma_0, \sigma_1)$
		return $c = c^*$

Figure 14: One-More Unforgeability and Partial Blindness Under Chosen Keys.

A.11 (Tagged) Non-interactive Blind Signature

In a (tagged) non-interactive blind signature ((T)NIBS) scheme, a signer issues a random *pre-signature* for a receiver, who can use its secret key to extract a blind signature for a random message. We define tagged non-interactive blind signatures following the definitions given in [5, 54].

Definition A.26 (Tagged non-interactive blind signature scheme). *A tagged non-interactive blind signature (TNIBS) scheme with a tag space $\mathcal{T}$ is a tuple of algorithms $\Sigma_{NIBS} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{RKGen}, \mathsf{Issue}, \mathsf{Obtain}, \mathsf{Vf})$ such that:*

- $\mathsf{Setup}(1^\lambda)$*: is a* PPT *algorithm that on input a (unary encoded) security parameter λ, it outputs public parameters* pp*. We implicitly assume that the rest of the algorithms take* pp *as input.*
- $\mathsf{KGen}(\mathsf{pp})$*: is a* PPT *algorithm that on input parameter parameters* pp*, it outputs signer's secret and public keys* $(\mathsf{sk}, \mathsf{pk})$*.*
- $\mathsf{RKGen}(\mathsf{pp})$*: is a* PPT *algorithm that on input public parameters* pp*, it outputs receiver's secret and public keys* $(\mathsf{rsk}, \mathsf{rpk})$*.*
- $\mathsf{Issue}(\mathsf{sk}, \mathsf{rpk}, t)$*: is a* PPT *algorithm that on input a signer's secret key* sk*, receiver's public key* rpk *and a tag $t \in \mathcal{T}$, outputs a pre-signature* psig *and a nonce* nonce*, which represents (a portion of) the signer's random coins.*
- $\mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}), t)$*: is a* DPT *algorithm that on input a receiver's secret key* rsk*, signer's public key* pk*, a pre-signature/nonce pair* $(\mathsf{psig}, \mathsf{nonce})$ *and a tag $t \in \mathcal{T}$, it outputs a message/signature pair (μ, σ) or $\bot$ (which denotes the failure symbol).*
- $\mathsf{Vf}(\mathsf{pk}, t, \mu, \sigma)$*: is a* DPT *algorithm that on input a signer's public key* pk*, a tag $t \in \mathcal{T}$, a message μ and a signature σ, it outputs a bit $b \in \{0,1\}$.*

We retrieve the notion of *non-interactive blind signature (NIBS)*, by ignoring the tag $t \in \mathcal{T}$ or alternatively setting it to a predefined value (e.g., $\bot$). In the rest of the definitions, we keep it generic and include the tag.

A TNIBS scheme needs to satisfy correctness, reusability, one-more unforgeability, (strong) receiver blindness and (strong) nonce blindness.

Definition A.27 (Correctness). *A TNIBS scheme Σ_{NIBS} is correct, if for every $\lambda \in \mathbb{N}$, $t \in \mathcal{T}$, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(\mathsf{pp})$, and $(\mathsf{rsk}, \mathsf{rpk}) \leftarrow \mathsf{RKGen}(\mathsf{pp})$, it holds that*

$$\Pr\left[\mathsf{Vf}(\mathsf{pk}, t, \mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, \mathsf{Issue}(\mathsf{sk}, \mathsf{rpk}, t), t)) = 1\right] = 1,$$

where the probability is taken over the random coins of Issue *and* Obtain*.*

Definition A.28 (Reusability). *A NIBS scheme Σ_{NIBS} satisfies the reusability property, if there exists a negligible function $\mathsf{negl}(\cdot)$, such that for every $\lambda \in \mathbb{N}$ and $t \in \mathcal{T}$, it holds that $\Pr[\mathsf{ExpReuse}_{\Sigma_{NIBS}}(\lambda)] \leq \mathsf{negl}(\lambda)$, where $\mathsf{ExpReuse}_{\Sigma_{NIBS}}$ is defined in Figure 15.*

Definition A.29 (One-more unforgeability). *A TNIBS scheme Σ_{NIBS} is one-more unforgeable, if for every stateful* PPT *adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$, such that $\Pr[\mathsf{ExpOM}_{\mathcal{A},\Sigma_{NIBS}}(\lambda)] \leq \mathsf{negl}(\lambda)$, where $\mathsf{ExpOM}_{\mathcal{A},\Sigma_{NIBS}}$ is defined in Figure 15.*

Definition A.30 ((Strong) receiver blindness). *A TNIBS scheme Σ_{NIBS} satisfies strong receiver blindness, if for every stateful* PPT *adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$, such that $\Pr[\mathsf{ExpBlindRec}_{\mathcal{A},\Sigma_{NIBS}}(\lambda)] \leq 1/2 + \mathsf{negl}(\lambda)$, where the experiment $\mathsf{ExpBlindRec}_{\mathcal{A},\Sigma_{NIBS}}$ is defined in Figure 15.*

Definition A.31 ((Strong) nonce blindness). *A TNIBS scheme Σ_{NIBS} satisfies strong nonce blindness, if for every stateful admissible* PPT *adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$, such that $\Pr[\mathsf{ExpBlindNonce}_{\mathcal{A},\Sigma_{NIBS}}(\lambda)] \leq 1/2 + \mathsf{negl}(\lambda)$, where the experiment $\mathsf{ExpBlindNonce}_{\mathcal{A},\Sigma_{NIBS}}$ is defined in Figure 15.*

Remark 2. *In Definitions A.30 and A.31, receiver blindness and nonce blindness are said to be* strong *if the adversary has access to the oracles in their respective games.*

Combining all the above properties we recover the definition of a secure TNIBS.

Definition A.32 (Secure TNIBS). *A TNIBS scheme Σ_{NIBS} is secure, if it is complete, reusable, one-more unforgeable, (strongly) receiver blind and (strongly) nonce blind.*

A.12 Adaptor Signature

Definition A.33 (Adaptor signature scheme). *An adaptor signature scheme w.r.t. a hard relation R and a signature scheme $\Sigma = (\mathsf{KGen}, \mathsf{Sig}, \mathsf{Vf})$ consists of four algorithms $\Sigma_{AS} = (\mathsf{PreSign}, \mathsf{Adapt}, \mathsf{PreVf}, \mathsf{Ext})$ defined as:*

$\mathsf{ExpReuse}_{\Sigma_{NIBS}}(\lambda)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(\mathsf{pp}), (\mathsf{rsk}, \mathsf{rpk}) \leftarrow \mathsf{RKGen}(\mathsf{pp})$
$\forall b \in \{0,1\}$:
  $(\mathsf{psig}_b, \mathsf{nonce}_b) \leftarrow \mathsf{Issue}(\mathsf{sk}, \mathsf{rpk}, t)$
  $(\mu_b, \sigma_b) \coloneqq \mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}_b, \mathsf{nonce}_b), t)$
$b_0 \coloneqq \mathsf{nonce}_0 = \mathsf{nonce}_1$
$b_1 \coloneqq \mu_0 = \mu_1$
return $b_0 \vee b_1$

$\mathsf{ExpOM}_{\mathcal{A},\Sigma_{NIBS}}(\lambda)$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\ \ell \coloneqq 0$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(\mathsf{pp})$
$\{t, (\mu_i, \sigma_i)\}_{i \in [\ell+1]} \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{Issue}}(\mathsf{pp}, \mathsf{pk})$
$b_0 \coloneqq \bigwedge_{i \in [\ell+1]} \mathsf{Vf}(\mathsf{pk}, t, \mu_i, \sigma_i) = 1$
$b_1 \coloneqq \bigwedge_{i \neq j \in [\ell+1]} \mu_i \neq \mu_j$
return $b_0 \wedge b_1$

$\mathcal{O}\mathsf{Issue}(\mathsf{rpk}, t)$

$(\mathsf{psig}, \mathsf{nonce}) \leftarrow \mathsf{Issue}(\mathsf{sk}, \mathsf{rpk})$
$\ell \coloneqq \ell + 1$
return $(\mathsf{psig}, \mathsf{nonce})$

$\mathsf{ExpBlindRec}_{\mathcal{A},\Sigma_{NIBS}}(\lambda))$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\ c \leftarrow\$ \{0,1\},\ Q \coloneqq \emptyset$
$\forall b \in \{0,1\}$: $(\mathsf{rsk}_b, \mathsf{rpk}_b) \leftarrow \mathsf{RKGen}(\mathsf{pp})$
$(\mathsf{pk}, (\mathsf{psig}_0, \mathsf{nonce}_0), (\mathsf{psig}_1, \mathsf{nonce}_1), t) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{rsk}_b}(\mathsf{pp}, \mathsf{rpk}_0, \mathsf{rpk}_1)$
$\forall b \in \{0,1\}$: $(\mu_b, \sigma_b) \coloneqq \mathsf{Obtain}(\mathsf{rsk}_b, \mathsf{pk}, (\mathsf{psig}_b, \mathsf{nonce}_b), t)$
$c' \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{rsk}_b}(\mu_c, \sigma_c, \mu_{1-c}, \sigma_{1-c})$
return $c = c' \wedge \mathsf{nonce}_b \notin Q$

$\mathsf{ExpBlindNonce}_{\mathcal{A},\Sigma_{NIBS}}(\lambda))$

$\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda), c \leftarrow\$ \{0,1\},\ Q \coloneqq \emptyset$
$(\mathsf{rsk}, \mathsf{rpk}) \leftarrow \mathsf{RKGen}(\mathsf{pp})$
$(\mathsf{pk}, (\mathsf{psig}_0, \mathsf{nonce}_0), (\mathsf{psig}_1, \mathsf{nonce}_1), t) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{rsk}}(\mathsf{pp}, \mathsf{rpk})$
$\forall b \in \{0,1\}$: $(\mu_b, \sigma_b) \coloneqq \mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}_b, \mathsf{nonce}_b), t)$
if $\exists b \in \{0,1\}$ s.t.$\mathsf{Vf}(\mathsf{pk}, t, \mu_b, \sigma_b) = 0$
  $\mu_0 \coloneqq \mu_1 \coloneqq \sigma_0 \coloneqq \sigma_1 \coloneqq \perp$
$c' \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{rsk}}(\mu_c, \sigma_c, \mu_{1-c}, \sigma_{1-c})$
return $c = c' \wedge \mathsf{nonce}_b \notin Q$

$\mathcal{O}\mathsf{rsk}_b(b, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}), t)$

$Q \coloneqq Q \cup \{\mathsf{nonce}\}$
return $\mathsf{Obtain}(\mathsf{rsk}_b, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}), t)$

$\mathcal{O}\mathsf{rsk}(\mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}), t)$

$Q \coloneqq Q \cup \{\mathsf{nonce}\}$
return $\mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}), t)$

Figure 15: Experiments for the security notions of NIBS.

- $\mathsf{PreSign}(\mathsf{sk}, m, Y)$*: is a* PPT *algorithm that on input a secret key* sk*, message* $m \in \{0,1\}^*$ *and statement* $Y \in L$*, outputs a pre-signature* $\tilde{\sigma}$*.*
- $\mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma})$*: is a* DPT *algorithm that on input a public key* pk*, message* $m \in \{0,1\}^*$*, statement* $Y \in L$ *and pre-signature* $\tilde{\sigma}$*, outputs a bit* b*.*
- $\mathsf{Adapt}(\tilde{\sigma}, y)$*: is a* DPT *algorithm that on input a pre-signature* $\tilde{\sigma}$ *and witness* y*, outputs a signature* σ*.*
- $\mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$*: is a* DPT *algorithm that on input a signature* σ*, pre-signature* $\tilde{\sigma}$ *and statement* $Y \in L$*, outputs a witness* y *such that* $(Y, y) \in R$*, or* $\perp$*.*

We note that an adaptor signature scheme Σ_{AS} also inherits the KGen and Vf algorithms from the underlying signature scheme Σ. In addition to the standard signature correctness, an adaptor signature scheme has to satisfy *pre-signature correctness.* Informally, an honestly generated pre-signature w.r.t. a statement $Y \in L$ is a valid pre-signature and can be adapted into a valid signature from which a witness for Y can be extracted.

Definition A.34 (Pre-signature correctness). *An adaptor signature scheme* Σ_{AS} *satisfies pre-signature correctness if for every* $\lambda \in \mathbb{N}$*, every message* $m \in \{0,1\}^*$ *and every statement/witness pair* $(Y, y) \in R$*, the following holds:*

$$\Pr\left[\begin{array}{c} \mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma}) = 1 \\ \wedge \\ \mathsf{Vf}(\mathsf{pk}, m, \sigma) = 1 \\ \wedge \\ (Y, y') \in R \end{array} \middle| \begin{array}{r} (\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda) \\ \tilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, Y) \\ \sigma \coloneqq \mathsf{Adapt}(\tilde{\sigma}, y) \\ y' \coloneqq \mathsf{Ext}(\sigma, \tilde{\sigma}, Y) \end{array}\right] = 1.$$

Definition A.35 (Pre-signature adaptability). *An adaptor signature scheme* Σ_{AS} *satisfies pre-signature adaptability if for any* $\lambda \in \mathbb{N}$*, any message* $m \in \{0,1\}^*$*, any statement/witness pair* $(Y, y) \in R$*, any key pair* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$ *and any pre-signature* $\tilde{\sigma} \leftarrow \{0,1\}^*$ *with* $\mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma}) = 1$*, we have*

$$\Pr[\mathsf{Vf}(\mathsf{pk}, m, \mathsf{Adapt}(\tilde{\sigma}, y)) = 1] = 1.$$

For security we consider the *full extractability* definition given by Dai et al. [29].

Definition A.36 (Full extractability). *An adaptor signature scheme* Σ_{AS} *is full extractable if for every* PPT *adversary* $\mathcal{A}$*, there exists a negligible function* negl *such that the following holds,*

$$\Pr[\mathsf{aFExt}_{\mathcal{A},\Sigma_{AS}}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{aFExt}_{\mathcal{A},\Sigma_{AS}}$ *is defined as follows*

$\mathsf{aFExt}_{\mathcal{A},\Sigma_{AS}}(\lambda)$

1: $Q_Q \coloneqq \emptyset, Q_P \coloneqq \emptyset, Q_C \coloneqq \emptyset$
2: $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
3: $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{Sig}, \mathcal{O}\mathsf{PreSign}, \mathsf{NewY}}(\mathsf{pk})$
4: $b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}, m^*, \sigma^*) = 1$
5: $b_2 \coloneqq (m^* \notin Q_Q)$
6: $b_3 \coloneqq \forall (Y, \tilde{\sigma}) \in Q_P[m^*]$ *s.t.* $Y \notin Q_C$, $(Y, \mathsf{Ext}(Y, \tilde{\sigma}, \sigma^*)) \notin R$
7: **return** $(b_1 \wedge b_2 \wedge b_3)$

$\mathcal{O}\mathsf{Sig}(m)$

1: $\sigma \leftarrow \mathsf{Sig}(\mathsf{sk}, m)$
2: $Q_M \coloneqq Q_M \cup \{m\}$
3: **return** σ

$\mathsf{NewY}()$

1: $(Y, y) \leftarrow \mathsf{GenR}(1^\lambda)$
2: $Q_C \coloneqq Q_C \cup \{Y\}$
3: **return** Y

$\mathcal{O}\mathsf{PreSign}(m, Y)$

1: $\tilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, Y)$
2: $Q_P[m] \coloneqq Q_P[m] \cup \{(Y, \tilde{\sigma})\}$
3: **return** $\tilde{\sigma}$

We note that Dai et al. [29] also defined the notion of *strong full extractability*, which is a stronger notion than full extractability with the difference between them being akin to the difference between SUF-CMA and EUF-CMA security notions for plain signatures. We note that we do not consider strong full extractability here, as regular full extractability is sufficient for the constructions and applications given in this work.

Recently, Gerhart et al. [41] introduced an additional security notion for adaptor signatures, called *pre-verify soundness*. Intuitively, it ensures that the pre-verification algorithm satisfies soundness w.r.t. the relation R.

Definition A.37 (Pre-verify soundness). *An adaptor signature scheme Σ_{AS} satisfies computational pre-verify soundness if for every* PPT *adversary $\mathcal{A}$ there exists a negligible function* negl *such that for every $\lambda \in \mathbb{N}$ and polynomially bounded $Y \notin L$,*

$$\Pr\left[\ \mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma}) = 1 \ \middle|\ \begin{matrix}(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda) \\ (m, \tilde{\sigma}) \leftarrow \mathcal{A}(\mathsf{sk})\end{matrix}\right] \leq \mathsf{negl}(\lambda).$$

We say that Σ_{AS} satisfies statistical pre-verify soundness if additionally for every key pair $(\mathsf{sk}, \mathsf{pk})$ *in the support of* $\mathsf{KGen}(1^\lambda)$,

$$\Pr[\exists (m, \tilde{\sigma}) \text{ s.t. } \mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma}) = 1] = 0.$$

As was pointed out in [41], like non-interactive plain model proofs, the statistical notion of pre-verify soundness is unachievable for non-trivial NP relations. However, for NP relations where checking membership can be done efficiently, such as the discrete logarithm language over efficiently recognizable groups, statistical pre-verify soundness can be achieved simply by letting the pre-verification algorithm decide membership of the given statement.

Lastly, we consider the privacy property called *unlinkability* that was described in [30, 85]. Intuitively, it ensures that the adapted signatures are indistinguishable from freshly computed signatures.

Definition A.38 (Unlinkability). *An adaptor signature scheme Σ_{AS} is unlinkable, if for every* PPT *adversary $\mathcal{A}$, there exists a negligible function* negl *such that the following holds,*

$$Pr[\mathsf{aUnlink}_{\mathcal{A},\Sigma_{AS}}(\lambda) = 1] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{aUnlink}_{\mathcal{A},\Sigma_{AS}}$ *is defined as follows:*

$\mathsf{aUnlink}_{\mathcal{A},\Sigma_{AS}}(\lambda)$

1: $b \leftarrow\$ \{0, 1\}$
2: $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
3: $(m, (Y, y)) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{Sig}, \mathcal{O}\mathsf{PreSign}}(\mathsf{pk})$
4: $\tilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, Y)$
5: $\sigma_0 \coloneqq \mathsf{Adapt}(\tilde{\sigma}, y)$
6: $\sigma_1 \leftarrow \mathsf{Sig}(\mathsf{sk}, m)$
7: $b' \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{Sig}, \mathcal{O}\mathsf{PreSign}}(\sigma_b)$
8: **return** $((Y, y) \in R \wedge b = b')$

$\mathcal{O}\mathsf{Sig}(m)$

1: $\sigma \leftarrow \mathsf{Sig}(\mathsf{sk}, m)$
2: **return** σ

$\mathcal{O}\mathsf{PreSign}(m, Y)$

1: $\tilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, Y)$
2: **return** $\tilde{\sigma}$

We say that Σ_{AS} is statistically *unlinkable if for all adversaries $\mathcal{A}$ that run in time at most 2^λ, we have that*

$$\Pr[\mathsf{aUnlink}_{\mathcal{A},\Sigma_{AS}}(\lambda) = 1] \leq 2^{-\lambda}.$$

We say that Σ_{AS} is perfectly *unlinkable if for all (even unbounded) adversaries $\mathcal{A}$, we have that*

$$\Pr[\mathsf{aUnlink}_{\mathcal{A},\Sigma_{AS}}(\lambda) = 1] = 0.$$

Combining the afore-described properties, we can define the notion of secure adaptor signature scheme as follows.

Definition A.39 (Secure adaptor signature). *An adaptor signature scheme Σ_{AS} is secure, if it is pre-verify sound, full extractable and unlinkable.*

B Proofs regarding VES

B.1 Security of VES'

Theorem 3.1. VES′ *does not achieve MK-Opacity.*

Proof. Adversary $\mathcal{A}$ wins MK-Opacity with the following steps:

- $\mathcal{A}$ initializes the challenger and receives pk.
- $\mathcal{A}$ invokes oracle AKGen until it obtains two encryption keys such that $\mathsf{LSB}(\mathsf{apk}_a) = 0$ and $\mathsf{LSB}(\mathsf{apk}_b) = 1$. This event occurs with non-negligible probability, since apk is sampled at random from a uniform distribution.
- $\mathcal{A}$ samples message m^* and queries oracle $\mathcal{O}$VESign twice for m^*. First, with apk_a and then with apk_b. $\mathcal{A}$ receives ω_a and ω_b.
- $\mathcal{A}$ parses $(\omega'_a, C_a) \coloneqq \omega_a$ and $(\omega'_b, C_b) \coloneqq \omega_b$.
- $\mathcal{A}$ computes $\sigma^* \coloneqq C_a \oplus C_b$.
- $\mathcal{A}$ presents the tuple (m^*, σ^*) as a forgery.

Since $\mathcal{A}$ only computes PPT algorithms, it is also PPT. For both queries to oracle $\mathcal{O}$VESign, $\mathcal{A}$ uses the same message m^*. Therefore, the values for C_0, σ^* and C_1 are the same, because $(C_0, r) \leftarrow \mathcal{H}(\mathsf{sk}, m)$, $\sigma \leftarrow \mathsf{Sign}'(\mathsf{sk}, m; r)$ and $C_1 \coloneqq C_0 \oplus \sigma$. The only difference between the two queries is the least significant bit of apk. This means that $\omega_a \coloneqq (\omega'_a, C_0)$ and $\omega_b \coloneqq (\omega'_b, C_1)$. Hence, $\sigma^* \coloneqq C_0 \oplus C_1$. Finally, both apk_a and apk_b were obtained from queries to oracle AKGen, so neither of the queries to $\mathcal{O}$VESign appended m^* to the list Q_S. We conclude that this is an efficient attack on MK-Opacity. □

Theorem 3.2. VES′ *does not achieve CK-Extractability.*

Proof. Adversary $\mathcal{A}$ wins CK-Extractability with the following steps:

- $\mathcal{A}$ initializes the challenger of CK-Extractability.
- $\mathcal{A}$ samples a key pair $(\mathsf{sk}^*, \mathsf{pk}^*) \leftarrow \mathsf{KGen}(1^\lambda)$.
- $\mathcal{A}$ sets $\mathsf{apk}^* = \widehat{\mathsf{apk}}$ and $\mathsf{ask}^* = \widehat{\mathsf{ask}}$.
- $\mathcal{A}$ samples ω^* and m^* at random from a uniform distribution.
- $\mathcal{A}$ presents $(\mathsf{ask}^*, \mathsf{apk}^*, \mathsf{pk}^*, m^*, \omega^*)$ to the challenger.

$\mathcal{A}$ is PPT, since it only computes PPT algorithms. Note that $\mathsf{apk}^* = \widehat{\mathsf{apk}}$, hence algorithm VEVer returns 1. Since, m^* and ω^* were sampled at random, Resolve returns a valid signature with negligible probability. We conclude that this is an efficient attack on CK-Extractability. □

Theorem 3.3 (VES′ satisfies Definition A.14)**.** *Assume that* $\Sigma_{\mathsf{VES}} \coloneqq$ (AKGen, VESign, Resolve, VEVer) *is secure according to Definition A.14 and that* $\mathcal{H}$ *is a random oracle. Then* $\Sigma'_{\mathsf{VES}} \coloneqq$ (AKGen′, VESign′, Resolve′, VEVer′) *as defined in Figure 4 is secure according to Definition A.14.*

Proof.

Lemma B.1. *Assume that* VES *satisfies completeness. Then* VES′ *as in Figure 4 satisfies completeness.*

Proof. Let us fix an arbitrary message m and let

$$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$$
$$(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$$
$$\omega \leftarrow \mathsf{VESign}'(\mathsf{sk}, \mathsf{apk}, m)$$
$$\sigma \coloneqq \mathsf{Resolve}'(\mathsf{ask}, \mathsf{pk}, m, \omega)$$

We require two prove two conditions: (i) $\mathsf{VEVer}'(\mathsf{pk}, \mathsf{apk}, m, \omega) = 1$, and (ii) $\mathsf{Vf}(\mathsf{pk}, m, \sigma) = 1$.

Condition $\mathsf{VEVer}'(\mathsf{pk}, \mathsf{apk}, m, \omega) = 1$ As described in **??**, we have that $(\omega', C_b) \coloneqq \omega$, such that ω is the output of $\omega' \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$, while C_b is either a one-time pad or a random string. Computing $\mathsf{VEVer}'(\mathsf{pk}, \mathsf{apk}, m, \omega)$ only requires to compute $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega')$. Hence, since Σ_{VES} satisfies completeness, it holds that $\mathsf{VEVer}'(\mathsf{pk}, \mathsf{apk}, m, \omega) = 1$.

Condition $\mathsf{Vf}(\mathsf{pk}, m, \sigma) = 1$ Note that σ is computed by invoking algorithm $\mathsf{Resolve}'(\mathsf{ask}, \mathsf{pk}, m, \omega)$, which requires to compute $\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega')$, in which $(\omega', C_b) \coloneqq \omega$. As discussed before, $\omega' \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$. Hence, since Σ_{VES} satisfies completeness, it holds that $\mathsf{Vf}(\mathsf{pk}, m, \sigma) = 1$. □

Lemma B.2. *Assume that* Σ_{VES} *satisfies abuse freeness and that* $\mathcal{H}$ *is a random oracle. Then* Σ'_{VES} *as defined in Figure 4 satisfies abuse freeness.*

Proof. The proof follows in a series of hybrids, where we gradually change our construction given in **??**. We consider the following hybrids, which are detailed in Figure 16.

$\mathcal{H}_0$: The original game for abuse freeness (c.f **??**), expanded with the details of our construction.

$\mathcal{H}_1$: Instead of using the same key in the signature and the random oracle, we run the random oracle on input freshly sampled key, sk', and the message m.

$\mathcal{H}_0(\lambda), \mathcal{H}_1(\lambda), \mathcal{H}_2(\lambda))$	$\mathcal{O}\mathsf{VESign}(m)$
$Q_{\mathsf{VS}} \coloneqq \emptyset$	$b \coloneqq \mathsf{LSB}(\mathsf{apk})$
$(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$	$(C_0, r) \leftarrow \mathcal{H}(\mathsf{sk}, m)$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$	$\mathcal{H}_1 :\ (C_0, r) \leftarrow \mathcal{H}(\mathsf{sk}', m)$
$\mathcal{H}_1 :\ (\mathsf{sk}', \mathsf{pk}') \leftarrow \mathsf{KGen}(1^\lambda)$	$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m; r)$
$(m^*, \omega^*) \leftarrow \mathcal{A}^{\mathcal{O}\mathsf{VESign}}(\mathsf{pk}, \mathsf{ask}, \mathsf{apk})$	$C_1 \coloneqq C_0 \oplus \sigma$
$b_0 \coloneqq m^* \notin Q_{\mathsf{VS}}$	$\omega' \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$
$b_1 \coloneqq \mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m^*, \omega^*) = 1$	$\omega \coloneqq (\omega', C_b)$
return $b_0 \wedge b_1$	$\mathcal{H}_2 :\ \omega \coloneqq (\omega', C_0)$
	$Q_{\mathsf{VS}} \coloneqq Q_{\mathsf{VS}} \cup \{m\}$
	return ω

Figure 16: Hybrids to prove that Σ'_{VES} satisfies abuse freeness. $\mathcal{H}_0$ corresponds to the original abuse freeness game, and $\mathcal{H}_1$ and $\mathcal{H}_2$ are highlighted in gray.

Claim B.1. *Let* Bad_1 *be the event that:*

$$\left| \begin{array}{l} \Pr[\mathcal{H}_0(\lambda) = 1] \\ - \Pr[\mathcal{H}_1(\lambda) = 1] \end{array} \right| > \mathsf{negl}(\lambda)$$

Assume that $\mathcal{H}$ *is a random oracle. Then* $\Pr[\mathsf{Bad}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. The only change between the two games if the key used in the random oracle is sk or sk′. Both keys are sampled using the KGen algorithm of the digital signature scheme, which means that they are sampled from an space that is at least λ bits. Since are in the random oracle assumption, to distinguish both executions, the adversary should be able to guess correctly sk'. The probability of this event is negligible, therefore, $\Pr[\mathsf{Bad}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$. □

$\mathcal{H}_2$: The challenger always returns C_0 instead of C_b in $\mathcal{O}\mathsf{VESign}$.

Claim B.2. *Let* Bad_2 *be the event that:*

$$\left| \begin{array}{l} \Pr[\mathcal{H}_1(\lambda) = 1] \\ - \Pr[\mathcal{H}_2(\lambda) = 1] \end{array} \right| > \mathsf{negl}(\lambda)$$

We claim that $\Pr[\mathsf{Bad}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. We show that $\Pr[\mathsf{Bad}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$. We distinguish two cases, depending on the least significant bit of apk:

$\Pr[\mathsf{Bad}_2(\lambda) = 1] \leq \Pr[\mathsf{LSB}_0(\lambda) = 1] + \Pr[\mathsf{LSB}_1(\lambda) = 1]$.

Consider the case in which the challenger selects apk such that $\mathsf{LSB}(\mathsf{apk}) = 0$. In this scenario, $\mathcal{H}_1$ and $\mathcal{H}_2$ return the tuple (ω', C_0). Therefore, both games return exactly the same output. Therefore, the adversary cannot distinguish between $\mathcal{H}_1$ and $\mathcal{H}_2$. We conclude that $\Pr[\mathsf{LSB}_0(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Consider the case in which the challenger selects apk such that $\mathsf{LSB}(\mathsf{apk}) = 1$. In this scenario, $\mathcal{H}_1$ returns (ω', C_1) and $\mathcal{H}_2$ returns the tuple (ω', C_0).

Consider that $\mathcal{A}$ queries VESign twice for the same message, m. In $\mathcal{H}_1$, VESign returns the same value for C_1 in both queries. This occurs because $C_1 \coloneqq C_0 \oplus \sigma$, in which C_0 and r are the output of querying the random oracle on input sk' m and σ is $\mathsf{Sign}(\mathsf{sk}, m; r)$. In $\mathcal{H}_2$, VESign returns the same value for C_0 in both queries, since C_0 and r are the output of inputting sk' and m in the random oracle. Both values appear equally random to $\mathcal{A}$, since C_0 is the output

of a random oracle and C_1 is also random as it is the output of the xor of C_0 and some other string σ. Therefore, $\mathcal{A}$ does not obtain information by querying VESign with the same message multiple times.

Consider that $\mathcal{A}$ queries VESign for two different messages, m_1 and m_2. In $\mathcal{H}_1$, VESign returns a different value for C_1 in each query. Consider that $C_{1,1} \coloneqq C_{0,1} \oplus \sigma_1$ is the output of query with message m_1 and $C_{1,2} \coloneqq C_{0,2} \oplus \sigma_2$ is the output for query with message m_2. Note that $C_{0,1}$, $C_{0,2}$ and r_1, r_2 are the output of inputting sk', and either m_1 or m_2, respectively, to the random oracle and σ_1, σ_2 are the signatures using r_1 and r_2 for messages m_1 and m_2.

Therefore, $C_{0,1}$, $C_{0,2}$ are different, σ_1 and σ_2 are also different and $C_{1,1}$, $C_{1,2}$ are also different. Note that the one-time pads have different elements, hence $C_{1,1}$, $C_{1,2}$ reveal no information about σ_1 and σ_2. Hence, the view of $\mathcal{A}$ in $\mathcal{H}_1$ is of two random values.

In $\mathcal{H}_2$, VESign returns a different value for C_0 in each query, which are the same $C_{0,1}$ and $C_{0,2}$ as in $\mathcal{H}_1$. Therefore, the view of $\mathcal{A}$ in $\mathcal{H}_1$ is $C_{1,1}$, $C_{1,2}$, and the view of $\mathcal{A}$ in $\mathcal{H}_2$ is $C_{0,1}$ and $C_{0,2}$.

In order to distinguish between these two hybrids, $\mathcal{A}$ should be able to determine that the values obtained in $\mathcal{H}_2$ are drawn directly from f, while the ones obtained in $\mathcal{H}_1$ are the result of an *XOR*. However, both tuples $C_{1,1}$, $C_{1,2}$ and $C_{0,1}$, $C_{0,2}$ are sampled from the same distribution, f. Since, $\mathcal{A}$ cannot distinguish between the two, they cannot differentiate $\mathcal{H}_2$ and $\mathcal{H}_1$. We conclude that $\Pr[\mathsf{LSB}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Since $\Pr[\mathsf{Bad}_2(\lambda) = 1] \leq \Pr[\mathsf{LSB}_0(\lambda) = 1] + \Pr[\mathsf{LSB}_1(\lambda) = 1]$, and both $\Pr[\mathsf{LSB}_0(\lambda) = 1]$ and $\Pr[\mathsf{LSB}_1(\lambda) = 1]$ occur with negligible probability, $\Pr[\mathsf{Bad}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

□

Claim B.3. *Assume that Σ_{VES} satisfies abuse freeness. Then,* $\Pr[\mathcal{H}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume by contradiction that there is an adversary $\mathcal{A}$ that breaks abuse freeness in Σ'_{VES} with no negligible probability. We can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break abuse freeness of Σ_{VES} with the following steps:

- $\mathcal{B}$ initializes the challenger for VES abuse freeness and obtains the tuple $(\mathsf{pk}, \mathsf{ask}, \mathsf{apk})$.
- $\mathcal{B}$ invokes adversary $\mathcal{A}$ on input $(\mathsf{pk}, \mathsf{ask}, \mathsf{apk})$.
- $\mathcal{A}$ returns the forgery (m^*, ω^*).
- $\mathcal{B}$ parses $(\omega', C) \coloneqq \omega^*$ and forwards (m^*, ω') to the challenger.

$\underline{\mathcal{O}\mathsf{VESign}(m)}$: $\mathcal{B}$ relays the query to its own oracle $\mathcal{O}$VESign and obtains ω'. Regarding C_0, $\mathcal{B}$ simply queries the random oracle on input sk' and m. Note that since $\mathcal{B}$ only relayed the messages, the memory of both oracles contains the same list of messages.

Since apk is selected by the challenger, we must distinguish two scenarios. The first in which $\mathsf{apk} = \widehat{\mathsf{apk}}$ is denoted as S1. The second, in which $\mathsf{apk} \neq \widehat{\mathsf{apk}}$ is denoted as S2.

We discuss first S1. In this scenario, if the challenger samples apk such that $\mathsf{apk} = \widehat{\mathsf{apk}}$, the adversary trivially wins, since VEVer returns always 1 if $\mathsf{apk} = \widehat{\mathsf{apk}}$, regardless of ω'. However, the tuple (m^*, ω') that $\mathcal{B}$ forwards to the challenger does not win the game, because it does not verify for the challenger.

We now discuss S2. In this scenario, the challenger samples apk such that $\mathsf{apk} \neq \widehat{\mathsf{apk}}$. If the adversary wins in this scenario, note that: (i) both oracles VESign are synchronized and contain the same list of messages; and (ii) a tuple (m^*, ω') returns 1 both with $\mathcal{B}$ and the challenger. Therefore, if $\mathcal{A}$ wins in scenario S2, $\mathcal{B}$ also wins abuse freeness game.

Since $\mathcal{B}$ only executes PPT algorithms, it is easy to see that $\mathcal{B}$ is also a PPT algorithm. Furthermore, it is also easy to see that $\mathcal{B}$ perfectly simulates $\mathcal{H}_3$ to $\mathcal{A}$. $\mathcal{B}$ only wins the abuse freeness game with the same probability that $\mathcal{A}$ wins in S2. Therefore, we must determine what is the probability that scenario S2 occurs. In order for S1 to occur, the challenger must select $\mathsf{apk} = \widehat{\mathsf{apk}}$ when selecting uniformly at random from the key space. The probability of this event is negligible. We conclude that, since S1 occurs with negligible probability, $\mathcal{B}$ wins abuse freeness with the same probability what $\mathcal{A}$ wins in S2. However, our assumption was that Σ_{VES} satisfies abuse freeness, so we have reached a contradiction. Therefore, the probability that $\mathcal{A}$ wins in S2 must be negligible. We conclude that $\Pr[\mathcal{H}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$ □

We conclude that Lemma B.2 holds.

□

Lemma B.3. *Assume that Σ_{VES} satisfies opacity and $\mathcal{H}$ is a random oracle. Then Σ'_{VES} as described in Figure 4 satisfies opacity.*

Proof. The proof follows in a series of hybrids, where we gradually change our construction given in **??**. We consider the following hybrids.

$\mathcal{H}_0$: This hybrid corresponds to the original game for opacity (c.f **??**), expanded with the details of our construction.

$\mathcal{H}_1$: In this hybrid, we switch the key used to query the random oracle for a fresh key sk'.

Claim B.4. *Let* Bad_1 *be the event that:*

$$\left| \begin{array}{l} \Pr[\mathcal{H}_0(\lambda) = 1] \\ -\Pr[\mathcal{H}_1(\lambda) = 1] \end{array} \right| > \mathsf{negl}(\lambda)$$

Assume that $\mathcal{H}$ is a random oracle. Then $\Pr[\mathsf{Bad}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. This proof is equivalent to Claim B.1. □

$\mathcal{H}_2$: In this hybrid, the challenger always returns C_0 instead of C_b in VESign.

Claim B.5. *Let* Bad_2 *be the event that:*

$$\left| \begin{array}{l} \Pr[\mathcal{H}_1(\lambda) = 1] \\ -\Pr[\mathcal{H}_2(\lambda) = 1] \end{array} \right| > \mathsf{negl}(\lambda)$$

We claim that $\Pr[\mathsf{Bad}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. This proof is equivalent to Claim B.2. □

Claim B.6. *Assume that Σ_{VES} satisfies opacity. Then,* $\Pr[\mathcal{H}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume by contradiction that there is an adversary $\mathcal{A}$ who wins in $\mathcal{H}_2$ with no negligible probability. We can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break opacity of VES with the following steps:

- $\mathcal{B}$ initializes the challenger for VES opacity and obtains the tuple $(\mathsf{pk}, \mathsf{apk})$.
- $\mathcal{B}$ invokes adversary $\mathcal{A}$ on input $(\mathsf{pk}, \mathsf{apk})$.

$\mathcal{H}_0(\lambda)\mathcal{H}_1(\lambda), \mathcal{H}_2(\lambda)$

$(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
$\mathcal{H}_1 :\ (\mathsf{sk}', \mathsf{pk}') \leftarrow \mathsf{KGen}(1^\lambda)$
$Q_\mathsf{R} \coloneqq \emptyset$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{O\mathsf{VESign}, O\mathsf{Resolve}}(\mathsf{pk}, \mathsf{apk})$
$b_0 \coloneqq m^* \notin Q_\mathsf{R}$
$b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}, m^*, \sigma^*) = 1$
return $b_0 \wedge b_1$

$O\mathsf{VESign}(m)$

$b \coloneqq \mathsf{LSB}(\mathsf{apk})$
$(C_0, r) \leftarrow \mathcal{H}(\mathsf{sk}, m)$
$\mathcal{H}_1 :\ (C_0, r) \leftarrow \mathsf{H}(\mathsf{sk}', m)$
$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m; r)$
$C_1 \coloneqq C_0 \oplus \sigma$
$\omega' \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$
$\omega \coloneqq (\omega', C_b)$
$\mathcal{H}_2 :\ \omega \coloneqq (\omega', C_0)$
return ω

$O\mathsf{Resolve}(m, \omega)$

parse $(\omega', C) \coloneqq \omega$
if $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega') = 0$ **abort**
$\sigma \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega')$
$Q_\mathsf{R} \coloneqq Q_\mathsf{R} \cup \{m\}$
return σ

Figure 17: Hybrids to prove that Σ'_{VES} satisfies opacity. $\mathcal{H}_0$ corresponds to the original opacity game, and $\mathcal{H}_1$ and $\mathcal{H}_2$ are highlighted in gray.

- $\mathcal{A}$ returns the forgery (m^*, σ^*), which $\mathcal{B}$ forwards to the challenger.

$\underline{O\mathsf{VESign}(m}$: $\mathcal{B}$ relays the query to its own oracle $O\mathsf{VESign}$ and obtains ω'. Regarding C_0, $\mathcal{B}$ simply queries the random oracle on input sk' and m. $\mathcal{B}$ defines $\omega \coloneqq (\omega', C_0)$ and returns it to $\mathcal{A}$.
$\underline{O\mathsf{Resolve}(m, \omega)}$: $\mathcal{B}$ parses $(\omega', C) \coloneqq \omega$ and forwards a query for oracle $O\mathsf{Resolve}$ to the challenger on input (m, ω'). The challenger returns σ, which $\mathcal{B}$ returns to $\mathcal{A}$. The query to both oracles is for the same message, so Q_R contains the same set of messages in both games.

We must distinguish two exclusive scenarios for apk. The first in which $\mathsf{apk} = \widehat{\mathsf{apk}}$ is denoted as S1. The second, in which $\mathsf{apk} \neq \widehat{\mathsf{apk}}$ is denoted as S2.

We discuss first S1. In this scenario, if the challenger samples apk such that $\mathsf{apk} = \widehat{\mathsf{apk}}$, the adversary trivially wins. The reason for this is that the corresponding $\widehat{\mathsf{ask}}$ is hardcoded in the construction and known by all parties. Therefore, $\mathcal{A}$ wins with probability of 1, and so does $\mathcal{B}$. However, since the challenger samples apk at random from a uniform distribution, the probability that $\mathsf{apk} = \widehat{\mathsf{apk}}$ is negligible. We conclude that the probability of winning opacity due to S1 is negligible.

We now discuss S2. In this scenario, the challenger samples apk such that $\mathsf{apk} \neq \widehat{\mathsf{apk}}$. If the adversary wins in this scenario: (i) Q_R contains the same messages in both games; and (ii) tuple (m^*, σ^*) verifies. Therefore, if $\mathcal{A}$ wins in scenario S2, $\mathcal{B}$ wins opacity with the same probability. The probability that S2 occurs is 1, since the probability of S1 is negligible and they are mutually exclusive. We conclude that, $\mathcal{B}$ wins opacity with the same probability what $\mathcal{A}$ wins in S2. However, our assumption was that Σ_{VES} satisfies opacity, so we have reached a contradiction. Therefore, the probability that $\mathcal{A}$ wins in S2 must be negligible. We conclude that $\Pr[\mathcal{H}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$

□

We conclude that Lemma B.3 holds. □

Lemma B.4. *If* VES *satisfies extractability, then* VES′ *in Figure 4 satisfies extractability.*

Proof. Assume by contradiction that adversary $\mathcal{A}$ breaks extractability in Σ'_{VES} with non negligible probability. Adversary $\mathcal{B}$ uses $\mathcal{A}$ to break extractability with the same probability, with the following steps:

- $\mathcal{B}$ initializes the challenger for extractability and obtains apk.
- $\mathcal{B}$ invokes adversary $\mathcal{A}$ on input apk.
- $\mathcal{A}$ returns the forgery $(\mathsf{pk}^*, m^*, \omega^*)$.
- $\mathcal{B}$ parses $(\omega', C) \coloneqq \omega^*$ and forwards $(\mathsf{pk}^*, m^*, \omega')$ to the challenger.

Since apk is selected by the challenger, we must distinguish two scenarios. The first in which $\mathsf{apk} = \widehat{\mathsf{apk}}$ is denoted as S1. The second, in which $\mathsf{apk} \neq \widehat{\mathsf{apk}}$ is denoted as S2. Scenario S1 occurs with negligible probability, since the challenger draws apk at random from a uniform distribution. $\mathcal{B}$ wins extractability with the same probability that $\mathcal{A}$ wins in S2. However, our assumption was that VES satisfies extractability, so we have reached a contradiction. Therefore, the probability that $\mathcal{A}$ wins in S2 must be negligible. We conclude that our construction VES′ satisfies extractability □

Lemma B.5. *If* VES *satisfies resolution duplication, then* VES′ *in Figure 4 satisfies resolution duplication.*

Proof. Assume that for Σ_{VES} there exists an algorithm Extract such that.

$$\mathsf{Extract}(m, \mathsf{sk}, r) = \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m; r)).$$

We show that the same algorithm Extract can be used for Σ'_{VES}. Note that $\omega \coloneqq (\omega' C)$ and that ω' was obtained by computing $\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$:

$$\mathsf{Resolve}'(\mathsf{ask}, \mathsf{pk}, m, \omega) = \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega') =$$
$$\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m; r)) = \mathsf{Extract}(m, \mathsf{sk}, r)$$

□

Claim B.7. *If* VES *satisfies abuse freeness and resolution independence, then* VES′ *in Figure 4 satisfies unforgeability.*

Proof. In **??** we have shown that if Σ_{VES} satisfies abuse freeness, so does Σ'_{VES}. Similarly, in **??** we have show that if Σ_{VES} satisfies resolution duplication, so does Σ'_{VES}. In the next section of the thesis, we show with **??** that abuse freeness and resolution independence imply unforgeability. Since Σ'_{VES} satisfies resolution duplication, it also satisfies resolution independence. Therefore, Σ'_{VES} satisfies unforgeability.

□

□

B.2 MK-Opacity implies Opacity

Theorem 3.4 (MK-Opacity $\implies$ Opacity)**.** *Assume that* VES *satisfy MK-Opacity. Then,* VES *satisfy opacity.*

Proof. Assume towards contradiction that PPT adversary $\mathcal{A}$ breaks opacity with non-negligible probability. Adversary $\mathcal{B}$ uses $\mathcal{A}$ to break MK-Opacity as follows:

- $\mathcal{B}$ initiates MK-Opacity challenger and obtains pk.
- $\mathcal{B}$ queries the oracle AKGen to obtain apk.
- $\mathcal{B}$ invokes $\mathcal{A}$ on input (apk, pk) to obtain (m^*, σ^*)
- $\mathcal{B}$ forwards (m^*, σ^*) to the MK-Opacity challenger.

Whenever $\mathcal{B}$ receives a VESign oracle query by $\mathcal{A}$, it forwards it to its own VESign oracle with the apk that $\mathcal{B}$ gave to $\mathcal{A}$. Since apk is the output of AKGen oracle, challenger's Q_{S} is not populated. When $\mathcal{A}$ makes Resolve query, $\mathcal{B}$ forwards it to its own Resolve oracle using apk. The Resolve queries are appended to Q_{R} both for the challenger and $\mathcal{B}$. It is easy to see that $\mathcal{B}$ perfectly simulates opacity to $\mathcal{A}$ and it is efficient. Given these points, if (m^*, σ^*) wins opacity, then it also wins MK-Opacity. Since we assume that MK-Opacity holds, we have reached a contradiction. We conclude that MK-Opacity implies opacity. □

B.3 CK-Extractability implies Extractability

Theorem 3.5 (CK-Extractability $\implies$ Extractability)**.** *Assume that* VES *satisfy CK-Extractability. Then,* VES *satisfies extractability.*

Proof. Assume towards contradiction that PPT adversary $\mathcal{A}$ breaks the extractability with non-negligible probability. Adversary $\mathcal{B}$ uses $\mathcal{A}$ to break CK-Extractability as follows:

- $\mathcal{B}$ initiates CK-Extractability challenger.
- $\mathcal{B}$ computes $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$.
- $\mathcal{B}$ invokes $\mathcal{A}$ on input apk to obtain $(\mathsf{pk}^*, m^*, \omega^*)$.
- $\mathcal{B}$ forwards $(\mathsf{ask}, \mathsf{apk}, \mathsf{pk}^*, m^*, \omega^*)$ to the challenger.

Whenever $\mathcal{B}$ receives a Resolve query, $\mathcal{B}$ uses ask on the queried ω and returns the resulting σ. Since $\mathcal{A}$ wins extractability, the encrypted signature ω^* verifies for message m^*, public key pk^* and apk, and the decrypted signature, σ does not verify. The pair (ask, apk) is a valid output of AKGen. $\mathcal{B}$ perfectly simulates extractability to $\mathcal{A}$ and only runs PPT algorithms. $\mathcal{B}$ wins CK-Extractability with the same probability that $\mathcal{A}$ wins extractability. Since we assume that CK-Extractability holds, we have reached a contradiction. Therefore, CK-Extractability implies extractability. □

$\mathcal{H}_0, \mathcal{H}_1$

1 : $Q_{\mathsf{VS}} \coloneqq \emptyset$
2 : $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$
3 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
4 : $(m^*, \omega^*) \leftarrow \mathcal{A}^{O\mathsf{VESign}}(\mathsf{pk}, \mathsf{ask}, \mathsf{apk})$
5 : $b_0 \coloneqq m^* \notin Q_{\mathsf{VS}}$
6 : $b_1 \coloneqq \mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m^*, \omega^*) = 1$
7 : $\sigma^* \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m^*, \omega^*)$
8 : $a_0 \coloneqq \mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m^*, \omega^*) = 1$
9 : $a_1 \coloneqq \mathsf{Vf}(\mathsf{pk}, m^*, \sigma^*) = 0$
10 : **if** $a_0 \wedge a_1$ **abort**
11 : **return** $b_0 \wedge b_1$

$\mathsf{VESign}(m)$

1 : $\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$
2 : $Q_{\mathsf{VS}} \coloneqq Q_{\mathsf{VS}} \cup \{m\}$
3 : **return** ω

Figure 18: $\mathcal{H}_0$ is the original abuse freeness game. In $\mathcal{H}_1$ abort if the encrypted signature forged by $\mathcal{A}$ cannot be decrypted into a valid signature.

B.4 MK-Opacity and CK-Extractability Imply Abuse Freeness

Theorem 3.6 (MK-Opacity + CK-Extractability $\implies$ Abuse Freeness)**.** *Assume that VES satisfy MK-Opacity and CK-Extractability. Then, VES satisfy abuse-freeness.*

Proof. We construct our proof by a series of hybrids.

Hybrid $\mathcal{H}_0$: The original abuse freeness game.

Hybrid $\mathcal{H}_1$: Differs from $\mathcal{H}_0$ on the lines highlighted in Figure 18. The game aborts if the encrypted signature forged by $\mathcal{A}$ cannot be decrypted into a valid signature.

Let Bad_1 be the event that $\mathcal{H}_1$ aborts on line 10. Therefore, $Pr[\mathcal{H}_0(\lambda) = 1] \leq Pr[\mathcal{H}_1(\lambda) = 1] + \Pr[\mathsf{Bad}_1(\lambda) = 1]$.

Lemma B.6. *Assume CK-Extractability. Then* $\Pr[\mathsf{Bad}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume towards contradiction that PPT adversary $\mathcal{A}$ forces an abort in $\mathcal{H}_1$ with non-negligible probability. Adversary $\mathcal{B}$ uses $\mathcal{A}$ to break CK-Extractability as follows:

- $\mathcal{B}$ initiates CK-Extractability challenger
- $\mathcal{B}$ computes $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$.
- $\mathcal{B}$ computes$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$.
- $\mathcal{B}$ invokes $\mathcal{A}$ on (pk, ask, apk) to obtain (m^*, ω^*).
- $\mathcal{B}$ forwards $(\mathsf{ask}, \mathsf{apk}, \mathsf{pk}, m^*, \omega^*)$ to the challenger.

For oracle VESign, $\mathcal{B}$ simulates honest responses using sk. It is easy to see that $\mathcal{B}$ is a PPT algorithm and that the simulation is faithful. Now, if $\mathcal{A}$ is able to abort on line 10, this means that the encrypted signature ω^* verifies for message m^*, public key pk and key apk. However, the decrypted signature, σ using ask with algorithm Resolve does not verify. Additionally, note that $\mathcal{B}$ honestly

generated the pair (ask, apk). $\mathcal{B}$ wins CK-Extractability with the same probability that $\mathcal{A}$ aborts $\mathcal{H}_1$. Since we assumed negligible probability of winning CK-Extractability, we have reached a contradiction. The probability of the adversary to trigger Bad_1 must be negligible. □

Lemma B.7. *Assume that MK-Opacity holds. Then,* $\Pr[\mathcal{H}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume towards contradiction that PPT adversary $\mathcal{A}$ wins $\mathcal{H}_1$ with no negligible probability. Adversary $\mathcal{B}$ uses $\mathcal{A}$ to break the MK-Opacity as follows:

- $\mathcal{B}$ initiates MK-Opacity challenger and obtains pk.
- $\mathcal{B}$ samples $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$.
- $\mathcal{B}$ invokes $\mathcal{A}$ on (pk, ask, apk) to obtain (m^*, ω^*).
- $\mathcal{B}$ computes $\sigma^* \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m^*, \omega^*)$ and forwards the pair (m^*, σ^*) to the MK-Opacity challenger.

Whenever $\mathcal{A}$ makes VESign oracle query, then $\mathcal{B}$ forwards it to its own VESign oracle and uses apk that it sampled itself for this query. Since this apk was not generated via a call to the AKGen oracle, we have that such VESign queries of $\mathcal{B}$ are counted populating Q_{VS} of the challenger. Hence, the lists Q_{VS} maintained by $\mathcal{B}$ and the MK-Opacity challenger are synchronized.

$\mathcal{B}$ perfectly simulates the game to $\mathcal{A}$ and is PPT. Next, if $\mathcal{A}$ wins $\mathcal{H}_1$, the signature (m^*, σ^*) extracted from (m^*, ω^*) is valid for pk. Also, m^* is not in Q_{VS} for $\mathcal{B}$ nor the challenger. $\mathcal{B}$ wins the MK-Opacity experiment with the same probability that $\mathcal{A}$ wins $\mathcal{H}_1$. Since we assume that MK-Opacity holds, we have reached a contradiction. Therefore, such an adversary $\mathcal{A}$ does not exist, concluding the proof for this lemma. □

We conclude the proof for Theorem 3.6. □

B.5 Abuse Freeness and Resolution Independence Imply Unforgeability

Theorem 3.7 (Abuse Freeness + Resolution Independence $\implies$ Unforgeability)**.** *If* VES *satisfies abuse-freeness and resolution independence the,* VES *satisfies unforgeability.*

Proof. We construct our proof by a series of hybrids.

Hybrid $\mathcal{H}_0$: The original VES unforgeability game.

Hybrid $\mathcal{H}_1$: Differs from $\mathcal{H}_0$ on the gray lines in Figure 19. For queries to oracle Sign, the challenger computes VESign and then Resolve.

Lemma B.8. *Let* Bad_1 *be the event that:*

$$|\Pr[\mathcal{H}_0(\lambda) = 1] - \Pr[\mathcal{H}_1(\lambda) = 1]| > \mathsf{negl}(\lambda)$$

Assume that the VES *is resolution independent. Then* $\Pr[\mathsf{Bad}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. VES is resolution independent if for all $\lambda \in \mathbb{N}$, for all $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$, for all $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$, for all messages m, it holds

$$[\mathsf{Sign}(\mathsf{sk}, m)], [\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m))]$$

are identically distributed. The only difference between $\mathcal{H}_0$ and $\mathcal{H}_1$ is that in the latter we substitute $\mathsf{Sign}(\mathsf{sk}, m)$ with the concatenation of $\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m))$. Since resolution independence holds, both options are identically distributed, and indistinguishable for any observer. This concludes the proof for the lemma. □

```
H_0, H_1
1 : (ask, apk) ← AKGen(1^λ)
2 : (sk, pk) ← KGen(1^λ)
3 : Q_S := Q_VS := Q_R := ∅
4 : O := {VESign, Resolve, Sign}
5 : (m*, ω*) ← A^O(pk, apk)
6 : b_0 := m* ∉ {Q_S, Q_VS, Q_R}
7 : b_1 := VEVer(pk, apk, m*, ω*)
8 : return b_0 ∧ b_1

VESign(m)
1 : ω ← VESign(sk, apk, m)
2 : Q_VS := Q_VS ∪ {m}
3 : return ω

Resolve(m, ω)
1 : if VEVer(pk, apk, m, ω) = 0
2 :     return ⊥
3 : σ ← Resolve(ask, pk, m, ω)
4 : Q_R := Q_R ∪ {m}
5 : return σ

Sign(m)
1 : σ := Sign(sk, m)
2 : ω ← VESign(sk, apk, m)
3 : σ ← Resolve(ask, pk, m, ω)
4 : Q_S := Q_S ∪ {m}
5 : return σ
```

Figure 19: $\mathcal{H}_0$ is the original unforgeability game. In $\mathcal{H}_1$ in the queries to oracle Sign**, the challenger computes** VESign **and then** Resolve**.**

Lemma B.9. *Assume that* VES *is abuse-free. Then* $\Pr[\mathcal{H}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume towards contradiction that PPT adversary $\mathcal{A}$ has non-negligible advantage against $\mathcal{H}_1$. Adversary $\mathcal{B}$ uses $\mathcal{A}$ to break the abuse freeness as follows:

- $\mathcal{B}$ initiates the challenger and obtains (pk, ask, apk).
- $\mathcal{B}$ invokes $\mathcal{A}$ on input (pk, apk) to obtain (m^*, ω^*).
- $\mathcal{B}$ forwards (m^*, ω^*) to the abuse-freeness challenger.

Whenever $\mathcal{B}$ receives a VESign oracle query, it forwards the call to its own VESign oracle provided by the challenger, and then, forwards the output to $\mathcal{A}$. Upon receiving Resolve query from $\mathcal{A}$, $\mathcal{B}$ uses its ask and responds to such queries by honestly running the Resolve algorithm and forwarding the output to $\mathcal{A}$. When $\mathcal{A}$ makes Sign queries, then $\mathcal{B}$ first forwards the query to the its own VESign oracle to obtain some ω, and then sends $\sigma \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)$ to $\mathcal{A}$ (as before $\mathcal{B}$ has ask and can honestly run Resolve).

Finally, we note that the forgery (m^*, ω^*) returned by $\mathcal{A}$ constitutes a valid forgery also in the abuse freeness experiment. More concretely, if (m^*, ω^*) is a valid forgery in $\mathcal{H}_1$, we have that m^* has not been queried to any of the oracles given in the $\mathcal{H}_1$ game, and that $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m^*, \omega^*) = 1$. Since $\mathcal{B}$ forwards the VESign and Sign oracle queries of $\mathcal{A}$ to its own VESign oracle, and uses ask to answer Resolve queries of $\mathcal{A}$, it implies that the pair (m^*, ω^*) constitutes a valid forgery for $\mathcal{B}$ in the abuse freeness experiment. It is easy to see that $\mathcal{B}$ perfectly simulates the game to $\mathcal{A}$ and that it is efficient. $\mathcal{B}$ wins the abuse freeness experiment with the same probability that $\mathcal{A}$ wins the $\mathcal{H}_1$. However, since we assume that $\mathcal{A}$ wins $\mathcal{H}_1$ with no negligible probability, this is in contradiction

$\mathsf{AKGen}(1^\lambda)$

1 : $x \leftarrow\$ \mathbb{Z}_q$, $y \coloneqq g_2^x$
2 : **return** $(\mathsf{ask} \coloneqq x, \mathsf{apk} \coloneqq y)$

$\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$

1 : $\sigma \leftarrow H(m)^{\mathsf{sk}}$
2 : $r \leftarrow\$ \mathbb{Z}_q, \mu \coloneqq \psi(g_2)^r, \nu \coloneqq \psi(\mathsf{apk})^r$
3 : $\omega' \coloneqq \sigma \cdot \nu$
4 : **return** $\omega \coloneqq (\omega', \mu)$

$\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \widetilde{\omega})$

1 : **parse** $\omega \coloneqq (\omega', \mu)$
2 : **return** $e(\omega', g_2) = e(H(m), \mathsf{pk}) \cdot e(\mu, \mathsf{apk})$

$\mathsf{Resolve}(m, \mathsf{ask}, \mathsf{pk}, \widetilde{\omega})$

1 : **parse** $\omega \coloneqq (\omega', \mu)$
2 : **assert** $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \widetilde{\omega})$
3 : $\sigma \coloneqq \omega' \cdot (\mu)^{-\mathsf{ask}}$
4 : **return** σ

Figure 20: Verifiable encrypted signature scheme introduced by Boneh et al. [12].

with our assumption that abuse freeness holds. Therefore $\mathcal{A}$ does not exist and $\Pr[\mathcal{H}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$. □

We conclude the proof for Theorem 3.7. □

B.6 Verifiably Encrypted BLS Signature

For completeness, let us restate in Figure 20 the construction presented in [12]. We present the proofs for MK-Opacity and CK-Extractability, which follow the same arguments as presented by Boneh et al. [12] to prove opacity and extractability, but modified to the multi-key setting.

Theorem B.1. *Assume that the BLS aggregate extraction assumption (Definition A.4) holds. Then, the VES construction in Figure 20 satisfies MK-Opacity.*

Proof. Assume towards contradiction that there exists a PPT adversary $\mathcal{A}$ that wins the MK-Opacity experiment with a non-negligible probability ϵ. Then, we can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break the aggregate extraction property as follows.
Setup: $\mathcal{B}$ initializes the challenger of aggregate extraction with $k = 2$ and receives $(g_2^\alpha, g_2^\beta) \in \mathbb{G}_2$, and $(g_1^\gamma, g_1^\delta, g_1^{\alpha\gamma+\beta\delta}) \in \mathbb{G}_1$. The goal of $\mathcal{B}$ is to recover either $g_1^{\alpha\gamma}$ or $g_1^{\beta\delta}$. We show here a reduction for recovering $g_1^{\alpha\gamma}$ and note that a symmetric argument also applies if we wanted to recover $g_1^{\beta\delta}$.

$\mathcal{B}$ sets $\mathsf{pk} \coloneqq g_2^\alpha$ and invokes the adversary $\mathcal{B}$ on input pk. Now, we have to note the following things:

- $\mathcal{A}$ will make polynomial q_v queries to VESign oracle with a apk not obtained from AKGen oracle.
- $\mathcal{A}$ will make polynomial q_r queries to Resolve oracle. Those queries will be done for apk obtained from AKGen oracle.
- Since the BLS signature scheme is unique (i.e., one signature per message), we assume, without loss of generality, that for each of the above mentioned queries, $\mathcal{A}$ uses different messages (i.e., $\mathcal{A}$ does not repeat a message for any of those queries).

Hash queries: $\mathcal{B}$ keeps Q_{H} containing the tuple (m, h, c, b) and responds the queries by facilitating h. If the queried message m is not already in Q_{H}, $\mathcal{B}$ adds a new tuple to the list with the following steps:

- Samples $b \leftarrow\$ \mathbb{Z}_q$.
- Flips a coin $c \in \{0, 1\}$. Note that $\Pr[c = 0] = 1/(q_v + q_r + 1)$.
- If c=0, then $h \coloneqq g_1^\gamma \cdot g_1^b$.
- If c=1, then $h \coloneqq g_1^b$.

AKGen queries: $\mathcal{B}$ keeps Q_{AK}, containing the tuples (r, apk). Each time $\mathcal{A}$ queries AKGen, $\mathcal{B}$ provides a new apk with the following steps:

- Samples $r \leftarrow\$ \mathbb{Z}_q$.
- Computes $\mathsf{apk} \coloneqq g_2^\beta \cdot g_2^r$.
- add the tuple (r, apk) to Q_{AK} and returns apk.

VESign queries: $\mathcal{A}$ requests an encrypted signature using as input a message m and an adjudicator's key apk. $\mathcal{B}$ replies as follows:

- Consults Q_{H} to obtain the tuple (m, h, c, b). If m is not in Q_{H}, runs a hash query and obtains the tuple.
- Consults the Q_{AK} to check if apk was produced by $\mathcal{B}$ or sampled by $\mathcal{A}$. If $\mathsf{apk} \in Q_{\mathsf{AK}}$, obtains the tuple (r, apk).
- Samples $x \leftarrow\$ \mathbb{Z}_q$.
- If $\mathsf{apk} \notin Q_{\mathsf{AK}}$ and c=0: $\mathcal{B}$ declares failure and halts.
- If $\mathsf{apk} \in Q_{\mathsf{AK}}$ and c=0: $\mathcal{B}$ computes $\omega \coloneqq (\psi(g_2^\alpha)^b \cdot g_1^{\alpha\gamma+\beta\delta} \cdot g_1^{\delta r} \cdot \psi(\mathsf{apk})^x, g_1^\delta \cdot g_1^x)$ and returns ω.
- If c=1: $\mathcal{B}$ computes $\omega \coloneqq (\psi(g_2^\alpha)^b \cdot \psi(\mathsf{apk})^x, g_1^x)$ and returns ω.

Resolve queries: $\mathcal{A}$ requests to decrypt a signature using as input a message m, an encrypted signature ω and an adjudicator key apk.

- As defined by the oracle, if $\mathsf{apk} \notin Q_{\mathsf{AK}}$, then we return $\perp$. Similarly, if ω does not verify, we gain return $\perp$.
- Consults Q_{H} to obtain the tuple (m, h, c, b). If m is not in Q_{H}, runs a hash query and obtains the tuple.
- If c=0, $\mathcal{B}$ declares failure and halts.
- If c=1, $\mathcal{B}$ computes $\sigma \coloneqq \psi(g_2^\alpha)^b$ and returns σ.

Output: $\mathcal{A}$ returns (m^*, σ^*) to $\mathcal{B}$. $\mathcal{B}$ checks in Q_{H} if c=0 or c=1. $\mathcal{B}$ only proceeds if c=0. In this case, $\mathcal{B}$ can compute $g_1^{\alpha\gamma} \coloneqq \sigma^*/\psi(g_2^\alpha)^b$. This result breaks aggregate extraction.

Now, $\mathcal{B}$ needs to satisfy four events to succeed in breaking aggregate extraction:

- $\mathcal{E}_1$: Do not halt in VESign.
- $\mathcal{E}_2$: Do not halt in Resolve.
- $\mathcal{E}_3$: $\mathcal{A}$ generates a valid non trivial (m^*, σ^*).
- $\mathcal{E}_4$: $\mathcal{E}_3$ holds for a m^* such that c =0.

Hence, $\Pr[\mathcal{E}_1 \wedge \mathcal{E}_2 \wedge \mathcal{E}_3 \wedge \mathcal{E}_4] = \Pr[\mathcal{E}_1] \cdot \Pr[\mathcal{E}_2] \cdot \Pr[\mathcal{E}_3 | \mathcal{E}_1 \wedge \mathcal{E}_2] \cdot \Pr[\mathcal{E}_4 | \mathcal{E}_1 \wedge \mathcal{E}_2 \wedge \mathcal{E}_3]$

Lemma B.10. *The probability of $\mathcal{E}_1$ is at least $1/e$, were e is the base of natural logarithms.*

Proof. We use the same reasoning as in Claim 4.6 in [12]. Without loss of generality, assume that the adversary does not ask multiple signatures for the same message. We prove by induction that after the adversary makes ℓ queries, the probability that it does not

abort is at least $(1-1/(q_v+q_r+1))^{\ell}$. Such claim is trivially true for $\ell = 0$. Consider ℓ-th query. Prior to issuing the query number ℓ, note that the bit c is independent of the view of the adversary. The only parts of information that the adversary could have that depend on c are the hash and the encrypted signature for query ℓ. However, the distributions of these values is the same, independent of the value of c. Therefore, the probability that this query aborts is, at most, $1/(q_v+q_r+1)$ (i.e., $\Pr[c=0] = 1/(q_v+q_r+1)$). Using the inductive hypothesis and the independence of c, the probability that $\mathcal{B}$ does not abort after this query is at least $(1-1/(q_v+q_r+1))^{\ell}$, which proves the inductive claim. Now, since the adversary $\mathcal{A}$ makes at most q_v queries to VESign, the probability that $\mathcal{B}$ does not abort is $(1-1/(q_v+q_r+1))^{q_v} \geq (1-1/(q_v+q_r+1))^{q_v+q_r} \geq 1/e$ □

Claim B.8. *The probability of $\mathcal{E}_2$ is at least $1/e$, were e is the base of natural logarithms.*

Proof. Using the same reasoning as in the previous claim, we can conclude the the probability of not aborting is $(1-1/(q_v+q_r+1))^{q_r} \geq (1-1/(q_v+q_r+1))^{q_v+q_r} \geq 1/e$ □

Claim B.9. $\Pr[\mathcal{E}_3|\mathcal{E}_1 \wedge \mathcal{E}_2] = \epsilon \geq \mathsf{negl}(n)$

Proof. This is the assumption we take for the reduction, that there exists an adversary $\mathcal{A}$ that wins MK-Opacity with a probability $\epsilon \geq \mathsf{negl}(n)$. □

Claim B.10. $\Pr[\mathcal{E}_4|\mathcal{E}_1 \wedge \mathcal{E}_2 \wedge \mathcal{E}_3] \geq 1/(q_v+q_r+1)$

Proof. The same reasoning as Claim 4.8 in [12]. Given that events $\mathcal{E}_1$, $\mathcal{E}_2$ and $\mathcal{E}_3$ happened, adversary $\mathcal{B}$ will abort unless $\mathcal{A}$ generates a forgery for a message such that $c = 0$. Since c is independent, the probability of this event is the probability of assigning $c = 0$ of a message in the hash oracle, which is $\Pr[c = 0] = 1/(q_v+q_r+1)$. □

Therefore, $\Pr[\mathcal{E}_1 \wedge \mathcal{E}_2 \wedge \mathcal{E}_3 \wedge \mathcal{E}_4] = \epsilon/(e^2 \cdot (q_v+q_r+1)) > \mathsf{negl}(n)$. Hence, if $\mathcal{A}$ wins MK-Opacity with probability $\epsilon > \mathsf{negl}(n)$, $\mathcal{B}$ wins aggregate extraction with probability $\epsilon/(e^2 \cdot (q_v+q_r+1)) > \mathsf{negl}(n)$. However, this contradicts our assumption that aggregate extraction holds, so $\mathcal{A}$ does not exist and this theorem has been proven. □

Theorem B.2. *The VES construction in Figure 20 satisfies CK-Extractability.*

Proof. The game starts by receiving $(\mathsf{ask}^*, \mathsf{apk}^*, \mathsf{pk}^*, m^*, \omega^*)$ from the adversary, such that the tuple (ask, apk) satisfies $(\mathsf{ask}, \mathsf{apk}) \coloneqq (a, g_2^a)$ and ω^* consists on the tuple (ω', μ). Let's assume that $\mathsf{pk}^* \coloneqq g_2^x$, $\omega' \coloneqq H(m^*)^x \cdot \psi(g_2^a)^r$ and $\mu \coloneqq \psi(g_2)^r$. The data received must satisfy the following two conditions:

$$e(\omega', g_2) = e(H(m^*), \mathsf{pk}^*) \cdot e(\mu, \mathsf{apk}^*) = e(H(m^*), g_2^x) \cdot e(\mu, g_2^a)$$
$$\sigma = \frac{\omega}{\mu^{-a}} = H(m^*)^{x'} \neq H(m^*)^x$$

Consider now that $\omega' \coloneqq H(m^*)^{x'} \cdot \psi(g_2^a)^{r'}$:

$$\begin{aligned} e(\omega', g_2) &= e(H(m^*)^{x'} \cdot \psi(g_2^a)^{r'}, g_2) = \\ &e((H(m^*)^{x'}, g_2) \cdot e(\psi(g_2^a)^{r'}, g_2) = \\ &e((H(m^*), g_2^{x'}) \cdot e(\psi(g_2)^{r'}, g_2^a) = \\ &e(H(m^*), g_2^x) \cdot e(\mu, g_2^a) \end{aligned}$$

AKGen(1^λ)
1 : $BG \leftarrow \mathsf{BGGen_R}(1^\lambda)$
2 : $a \leftarrow\$ \mathbb{Z}_q, A \coloneqq g^a$
3 : **return** $(\mathsf{ask} \coloneqq a, \mathsf{apk} \coloneqq (BG, g^a))$

VESign(sk, apk, m)
1 : **parse** $\mathsf{apk} \coloneqq (BG, A)$
2 : $s \leftarrow\$ \mathbb{Z}_q$
3 : $\omega' \leftarrow \mathsf{Sign_R}((A^{ms}, A^s, A), \mathsf{sk})$
4 : **return** $\omega \coloneqq (\omega', A^s)$

VEVer(pk, apk, m, ω)
1 : **parse** $\mathsf{apk} \coloneqq (BG, A)$
2 : **parse** $\omega \coloneqq (\omega', W)$
3 : $b \coloneqq \mathsf{Verify_R}((W^m, W, A), \omega', \mathsf{pk})$
4 : **return** b

Resolve(m, ask, pk, ω)
1 : **parse** $\omega \coloneqq (\omega', A^s)$
2 : **parse** $\mathsf{ask} \coloneqq a$
3 : **assert** $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega)$
4 : $((g^{ms}, g^s, g), \sigma') \leftarrow \mathsf{ChgRep_R}((A^{ms}, A^s, A), \omega', \frac{1}{a}, \mathsf{pk})$
5 : $\sigma \coloneqq (\sigma', g^s)$
6 : **return** σ

Figure 21: Verifiably encrypted SPS-EQ-R by Hanser et al. [52].

Therefore, we conclude that $x = x'$ and $r = r'$, which means that both conditions cannot simultaneously hold. Either the verifiable encrypted signature ω does not verify and neither does the extracted signature σ, or the encrypted signature ω verifies and so does the extracted signature σ. □

B.7 Verifiably Encrypted SPS-Eq-R

For completeness, let us restate in Figure 21 the construction presented by Hanser. et al. in [52]. We present the proofs for MK-Opacity and CK-Extractability. They follow similar arguments to the proofs by Hanser et al. for opacity and extractability, but modified to the multi-key setting.

Theorem B.3. *Assume that the underlying SPS-EQ-R signature scheme is unforgeable and that the DHI assumption holds in $\mathbb{G}_1$. Then VES from SPS-EQ-R satisfies MK-Opacity.*

Proof. We assume that there is an efficient adversary $\mathcal{A}$ that wins MK-Opacity with non-negligible probability. Then we are able to construct and adversary $\mathcal{B}$ that uses $\mathcal{A}$ either to break with non-negligible probability (i) the EUF-CMA security of the underlying SPS-EQ-R scheme (Type-1 adversary) if $\mathcal{A}$ has neither queried the VESign oracle for an apk obtained from AKGen oracle for m^* nor the Resolve oracle for m^*; OR (ii) the DHI assumption (Type-2 adversary) if $\mathcal{A}$ has queried VESign but not the Resolve oracle for m^*.

In the following, $\mathcal{B}$ guesses $\mathcal{A}$'s strategy, i.e. the type of forgery $\mathcal{A}$ will conduct. We now describe the setup, the initialization of the environment, the reduction and the abort conditions for each type.

Type-1: $\mathcal{B}$ obtains pk_R of the SPS-EQ-R scheme with $l = 3$ (and thereby implicitly the bilinear group BG) from the challenger C of the EUF-CMA security game and sets $\mathsf{pk} := \mathsf{pk}_R$. Next, $\mathcal{B}$ runs $\mathcal{A}$ on input (pk) and answers $\mathcal{A}$'s queries to the Resolve oracle as in the real game and simulates queries to the other oracles as follows:

$\underline{\mathsf{AKGen}}$: $\mathcal{B}$ picks $a \leftarrow\$ \mathbb{Z}_p$, computes $A \leftarrow g^a$ and sets $(\mathsf{ask}, \mathsf{apk}) := (a, (BG, A))$. Then, remembers the tuple $(\mathsf{ask}, \mathsf{apk})$ in Q_{AK} and returns apk to $\mathcal{A}$.

$\underline{\mathsf{VESign}(m, \mathsf{sk}, \mathsf{apk})}$: $\mathcal{B}$ checks in Q_{AK} if apk was queried in AKGen. If this is not the case, $\mathcal{B}$ appends m to Q_{S}. Regardless of the origin of apk, $\mathcal{B}$ proceeds as follows: (i) samples $s \leftarrow\$ \mathbb{Z}_p$; (ii) parses $((BG, A)) := \mathsf{apk}$; (iii) defines $m' := (A^{ms}, A^s, A)$; (iv) queries the signing oracle of C for message m' to obtain signature ω; and (v) returns to $\mathcal{A}$ the tuple (ω, A^s).

If at some point $\mathcal{A}$ outputs a valid message-signature pair (m^*, σ^*) with $\sigma^* := (\sigma'^*, S^*)$, and neither has queried the VESign oracle for an apk obtained from AKGen oracle for m^* nor the Resolve oracle for m^*, then $\mathcal{B}$ will output $((S^{*m^*}, S^*, g), \sigma'^*)$ to C. In case that $\mathcal{A}$ has queried the VESign oracle for m^*, $\mathcal{B}$ will abort.

It is important to note that the distribution of all values returned to $\mathcal{A}$ during the simulation is identical to the distribution of these values during a real game, which makes the simulation perfect.

By construction, $((S^{*m^*}, S^*, g), \sigma'^*)$ constitutes a valid SPS-EQ-R message-signature pair. It remains to show that for $M^* = (S^{*m^*}, S^*, g)$, the class $[M^*]_R$ is different from all classes queried to C, if m^* is different from all the messages queried to the VESign oracle. $\mathsf{Verify}_{\mathsf{R}}$ demands that the third vector component of M^* be g, which uniquely determines the representative for each class and allows for comparison. Now, if there is some vector in which the representation of the message coincides in the second component, then both vectors still differ in the first component as long as $m^* \neq m_i$. Likewise, if they coincide in the first component for $m^* \neq m_i$, then they cannot have equal second components. Hence, $M^* \neq M_i$ for any M_i queried to C.

Type-2: In the following, let p be some fixed probability, which we will set later. $\mathcal{B}$ obtains an instance (g, g^a) of the DHI problem in group $\mathbb{G}_1 \in BG$ (which fixes the bilinear group BG) from the challenger C. $\mathcal{B}$ executes $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}_{\mathsf{R}}(BG)$, an invokes $\mathcal{A}$ on input (pk), sets up a list $L \leftarrow \emptyset$ and simulates queries to the oracles as follows:

$\underline{\mathsf{AKGen}}$: $\mathcal{B}$ picks $r \leftarrow\$ \mathbb{Z}_p$, sets $A := g^{ar}$ and sets $\mathsf{apk} := (BG, A))$. Then, remembers the tuple (r, apk) in list Q_{AK} and returns apk to $\mathcal{A}$.

$\underline{\mathsf{VESign}(m, \mathsf{sk}, \mathsf{apk})}$: $\mathcal{B}$ parses $(BG, A) := \mathsf{apk}$ and samples $s \leftarrow\$ \mathbb{Z}_p$ and random coin r_2. If apk was not queried in AKGen (i.e. it is not in list Q_{AK}), $\mathcal{B}$ appends m to Q_{S} and assigns $W := A^s$. If apk was queried in AKGen, $\mathcal{B}$ assigns $W := A^s$ with probability p or $W := (g \cdot A)^s$ with probability $(1 - p)$. Regardless of the origin of apk, $\mathcal{B}$ proceeds as follows: (i) computes $\omega' \leftarrow \mathsf{Sign}_R((W^m, W, A), \mathsf{sk}; r_2)$; (ii) sets $\omega := (\omega', W)$; (iii) stores $L[m] := (s, r_2, \omega, \mathsf{apk})$; and (iv) returns ω.

$\underline{\mathsf{Resolve}(\omega, m, \mathsf{apk})}$: $\mathcal{B}$ first checks if apk was queried in AKGen to retrieve the tuple (r, apk) and returns $\perp$ if it is not the case. $\mathcal{B}$ now checks if $\mathsf{VEVer}(m, \omega, \mathsf{pk}, \mathsf{apk})$ verifies and returns $\perp$ if it is not the case. If $\mathcal{B}$ has not returned $\perp$, then it retrieves the entry $(s, r_2, \omega := ((\omega', W), \mathsf{apk}) := L[m]$. Now, $\mathcal{B}$ aborts if $W = (g \cdot A)^s$. If $\mathcal{B}$ has not aborted, it computes $\sigma' \leftarrow \mathsf{Sign}_{\mathsf{R}}((g^{mrs}, g^{rs}, g^r), \mathsf{sk}; r_2)$. $\mathcal{B}$ sets $\sigma := (\sigma', g^{rs})$ and outputs σ to $\mathcal{A}$.

If at some point $\mathcal{A}$ outputs a valid message-signature pair $(m^*, \sigma^* := (\sigma'^*, S^*)$, and has queried the VESign oracle for m^*, but not the Resolve oracle, then $\mathcal{B}$ retrieves $(s^*, r_2, \omega^* := ((\omega', W), \mathsf{apk}) := L[m]$. $\mathcal{B}$ uses apk to search in the memory of AKGen and obtain the corresponding tuple (r, apk). If $S^* = g^{s^* r}$, then $\mathcal{B}$ aborts. Otherwise, we have that $S^* = s^*(g^{-ra} \cdot g)$ and $\mathcal{B}$ outputs $g^{-a} = S^{*-rs^*} \cdot g^{-r}$ as a solution to the DHI problem.

Note that the distribution of all values returned to $\mathcal{A}$ during the simulation is identical to the distribution of these values during a real game, which makes the simulation perfect.

Let q_R be the number of resolve queries. Then, with probability p^{q_R}, $\mathcal{B}$ does not abort during the simulation. Given that the simulation works out, $\mathcal{A}$ outputs a "useful" forgery with probability 1-p. In total, $\mathcal{B}$ is able to return a solution to the DHI problem with probability $p^{q_R}(1-p)$. The function $f(p) = p^{q_R}(1-p)$ reaches its maximum for $\frac{q_R}{q_R+1}$ and after a few calculations we obtain $f(p) = O(\frac{1}{q_R})$. Therefore, if $\mathcal{A}$ is able to break the MK-Opacity of the scheme with non negligible probability ϵ, then $\mathcal{B}$ is able to break the DHI assumption with non-negligible probability $O(\frac{\epsilon}{q_R})$. □

Theorem B.4. *Assume that SPS-EQ-R is correct. Then the VES construction in Figure 21 satisfies CK-Extractability.*

Proof. Assume assume $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}_{\mathsf{R}}(BG)$, and $(\mathsf{ask}, \mathsf{apk}) := (a, (BG, A := g^a))$ as described above and $a \leftarrow\$ \mathbb{Z}_q$. Then, a valid verifiable encrypted signature is:

$$\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m) = (\omega \leftarrow \mathsf{Sign}_{\mathsf{R}}((A^{ms}, A^s, A), \mathsf{sk}), g^{a \cdot s})$$

In order to decrypt a valid verifiable encrypted signature, Resolve performs a representation change for the message and computes its corresponding signature in the new representation. Since the verifiable encrypted signature ω was computed for the message (A^{ms}, A^s, A), the representation change obtains the signature for the message (g^{ms}, g^s, g). Since the factor that differentiates both representations is $1/a$ $(A := g^a)$, Resolve computes:

$$((g^{ms}, g^s, g), \sigma)$$
$$\leftarrow \mathsf{ChgRep}_{\mathsf{R}}((A^{ms}, A^s, A), \mathsf{Sign}_{\mathsf{R}}((A^{ms}, A^s, A), \mathsf{sk}), 1/a, \mathsf{pk})$$

Due to correctness of SPS-EQ-R, if the signature ω verifies for message (A^{ms}, A^s, A) and public key pk, then signature σ verifies for message (g^{ms}, g^s, g) and key pk, since they are computed as a representation change by a factor of $1/a$. Hence, the SPS-EQ-R construction for VES satisfies CK-Extractability. □

B.8 Generic VES Construction

We present in the following a generic VES construction combining a digital signature scheme, a public key encryption scheme and a non-interactive zero knowledge proof scheme.

Theorem B.5. *Let Σ be a correct and unforgeable signature scheme,* PKE *be a correct and multi-user and multi-challenge IND-CCA secure*

```
AKGen(1^λ)
  return PKE.KGen(1^λ)
VESign(sk, apk, m)
  σ ← Sign(sk, m)
  c ← PKE.Enc(apk, σ)
  π ← NIZK.P(c, apk, σ, m, sk)
  return (c, π)
Resolve(ask, pk, m, ω)
  parse (c, π) := ω
  return PKE.Dec(ask, c)
VEVer(pk, apk, m, ω)
  parse (c, π) := ω
  return NIZK.V(pk, apk, m, c, π)
```

Figure 22: Generic VES construction.

public key encryption scheme, and NIZK *satisfy zero knowledge and completeness, then the construction described in Figure 22, is a secure VES according to Definition 3.3.*

Proof. The proof follows via a series of lemmas shown below.

Lemma B.11. *Assume that the the signature scheme is correct, the encryption scheme is correct and the NIZK is complete. Then, the construction described in Figure 22 satisfies VES completeness.*

Proof. Let us fix an arbitrary message m and let

$$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$$
$$(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$$
$$\omega \leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m).$$

From our construction we have that $\omega := (c, \pi)$. We have that $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega) := \mathsf{NIZK.V}(\mathsf{pk}, \mathsf{apk}, m, c, \pi)$. Since in $\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$ we compute $\mathsf{NIZK.P}(c, \mathsf{apk}, \sigma, m, \mathsf{sk})$, by the completeness of NIZK we have that $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega) = 1$. Furthermore, since in $\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$ we compute $c \leftarrow \mathsf{PKE.Enc}(\mathsf{apk}, \sigma)$ and in $\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)$ we compute $\mathsf{PKE.Dec}(\mathsf{ask}, c)$ due to the correctness of PKE, we have that $\sigma := \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)$. Finally, since in $\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$ we compute $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$, by the correctness of Σ we have that $\mathsf{Vf}(\mathsf{pk}, m, \sigma) = 1$. □

Lemma B.12. *Assume that the the signature scheme is unforgeable, the encryption scheme is multi-user and multi-challenge IND-CCA secure and the NIZK is zero knowledge. Then, the construction described in Figure 22 satisfies VES MK-Opacity.*

Proof. The proof follows in a series of hybrids, where we gradually change our construction , and finally reduce MK-Opacity to the signature unforgeability. Let $\mathcal{H}_i \approx \mathcal{H}_{i+1}$ denote $|\Pr[\mathcal{H}_i = 1] - \Pr[\mathcal{H}_{i+1} = 1]| \leq \mathsf{negl}(\lambda)$. We consider the following hybrids.

Hybrid $\mathcal{H}_0$: This corresponds to the original OMMK-Opacity.

Hybrid $\mathcal{H}_1$: In this hybrid, we change the way we answer VESign oracle queries. Concretely, we simulate the proofs.

Claim B.11. *If* NIZK *is zero-knowledge, then it holds that $\mathcal{H}_0 \approx \mathcal{H}_1$.*

Proof. In order to show that $\mathcal{H}_0 \approx \mathcal{H}_1$, we construct a reduction $\mathcal{B}_1$ to the computational zero-knowledge property of NIZK. We note that all queries are answered as in $\mathcal{H}_0$ with the exception of VESign queries. The only difference here is that $\mathcal{B}_1$ receives a CRS crs^* from its zero-knowledge challenger, and sets $\mathsf{crs} := \mathsf{crs}^*$, instead of generating crs via the NIZK.PGen algorithm. When $\mathcal{A}$ makes a VESign query (apk, m), $\mathcal{B}_1$ proceeds as in $\mathcal{H}_0$ to compute $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ (note that $\mathcal{B}_1$ generates the key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$ at the beginning of the game) and $c \leftarrow \mathsf{PKE, Enc}(\mathsf{apk}, \sigma)$, for some randomness $r_0, r_1 \in \{0,1\}^*$. Then, $\mathcal{B}_1$ makes an oracle call to its zero-knowledge challenger with the input $((c, \mathsf{pk}, \mathsf{apk}, m), (\sigma, \mathsf{sk}))$ to obtain a proof π^*, and returns the tuple (c, π^*) to $\mathcal{A}$ as an answer to the VESign query.

If the zero-knowledge challenger of NIZK used the honest setup and prover algorithms to generate crs^* and π^*, then we are exactly in $\mathcal{H}_0$, and if it used the simulator, then we are in $\mathcal{H}_1$. Therefore, if $\mathcal{A}$ can distinguish between the two hybrids with non-negligible advantage, then $\mathcal{B}_1$ can break the zero-knowledge property of NIZK. Hence, the claim holds. □

```
ExpMKOpacity_{A,Σ_VES}
  Q_S := Q_R := ∅, Q_AK := [·]
  (sk, pk) ← KGen(1^λ)
  O := {OAKGen, OVESign, OResolve}
  (m*, σ*) ← A^O(pk)
  b_0 := m* ∉ {Q_S, Q_R}
  b_1 := Vf(pk, m*, σ*)
  return b_0 ∧ b_1
OAKGen()
  (ask, apk) ← PKE.KGen(1^λ)
  Q_AK[apk] := ask
  return apk
OVESign(apk, m)
  σ ← Sign(sk, m)
  c ← PKE.Enc(apk, σ)
  H_3 : if Q_AK[apk] ≠ ⊥ then
  H_3 :    c ← PKE.Enc(apk, 0)
  π ← NIZK.P(c, apk, σ, m, sk)
  H_1 : π ← S(c, apk, m)
  ω := (c, π)
  if Q_AK[apk] = ⊥ then
     Q_S := Q_S ∪ {m}
  return ω
OResolve(apk, m, ω)
  if Q_AK[apk] = ⊥ then
     return ⊥
  parse (c, π) := ω
  if NIZK.V(pk, apk, m, c, π) = 0 then
     return ⊥
  σ ← PKE.Dec(ask, c)
  H_2 : σ ← Sign(sk, m)
  Q_R := Q_R ∪ {m}
  return σ
```

Figure 23: Hybrids to prove MK-Opacity**. Changes between games highlighted in gray.**

Hybrid $\mathcal{H}_2$: In this hybrid, we change the way we answer Resolve oracle queries. More precisely, for a query $\mathcal{O}\mathsf{Resolve}(\mathsf{apk}, m, \omega)$

that verifies, $\mathsf{NIZK.V}(\mathsf{pk}, \mathsf{apk}, m, c, \pi) = 1$, instead of decrypting c, we compute a signature $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$. $\mathcal{B}$ keeps track of which signatures they have returned for a tuple $(\omega, m, \mathsf{apk})$. Hence, if the adversary repeats the query, they receive the same signature. Since the encrypted message was also a computed as $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$, both messages, the value returned to adversary in $\mathcal{O}\mathsf{Resolve}$ is equally distributed in $\mathcal{H}_2$ and $\mathcal{H}_1$

Hybrid $\mathcal{H}_3$: In this hybrid, we change the way we answer VESign oracle queries. More precisely, for a query $\mathcal{O}\mathsf{VESign}(\mathsf{apk}, m)$, if apk was obtained via the AKGen oracle, then we replace the ciphertext $c \leftarrow \mathsf{PKE.Enc}(\mathsf{apk}, \sigma)$ with an encryption of $c^* \leftarrow \mathsf{PKE.Enc}(\mathsf{apk}, 0)$.

Claim B.12. *If* PKE *is multi-user and multi-challenge IND-CCA secure, then it holds that $\mathcal{H}_1 \approx \mathcal{H}_2$.*

Proof. In order to show that $\mathcal{H}_1 \approx \mathcal{H}_2$, we construct a reduction $\mathcal{B}_2$ to the multi-user and multi-challenge IND-CCA security of PKE (see Definition A.17, where n in our case corresponds to the number of AKGen queries that $\mathcal{A}$ makes). $\mathcal{B}_2$ proceeds as in the previous hybrid except for the following changes. $\mathcal{B}_2$ maintains an internal indices k and i and a table $\mathcal{P}$, where k gets incremented each time $\mathcal{A}$ queries the AKGen oracle and i gets incremented each time $\mathcal{A}$ calls the VESign oracle.

Upon receiving an AKGen oracle call, $\mathcal{B}_2$ makes a NewPK oracle call to its IND-CCA challenger with input k, which returns a public key pk_k to $\mathcal{B}_2$. Then, $\mathcal{B}_2$ sets $\mathsf{apk} := \mathsf{pk}_k$, $\mathcal{P}[\mathsf{apk}] = k$, increments k and forwards this apk to $\mathcal{A}$. Upon receiving a $\mathcal{O}\mathsf{VESign}$ query (apk, m) from $\mathcal{A}$, $\mathcal{B}_2$ proceeds as in the previous hybrid to compute $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ (note again that $\mathcal{B}_2$ samples the key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$ at the beginning of the game). Next, $\mathcal{B}_2$ checks if there exists $k \in \mathcal{P}[\mathsf{apk}]$, and if this is the case, then it queries the Enc oracle given by its IND-CCA challenger with the input (k, i, σ) to receive a ciphertext c^* and increments i, otherwise, it computes the ciphertext itself as $c^* \leftarrow \mathsf{PKE.Enc}(\mathsf{apk}, \sigma)$. Finally, $\mathcal{B}_2$ simulates the proof π^* using the NIZK simulator and responds to the VESign query with the pair $\omega := (c, \pi)$. Upon receiving a Resolve query $(\mathsf{apk}, \mathsf{pk}, m, \omega)$, $\mathcal{B}_2$ checks if there exists some $k \in \mathcal{P}[\mathsf{apk}]$, and if not it returns $\perp$ to $\mathcal{A}$ (note that this is consistent with the functionality of the Resolve oracle, which returns $\perp$ for queries involving an apk not obtained from the AKGen oracle). Otherwise, $\mathcal{B}_2$ computes $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ and returns σ to $\mathcal{A}$.

We note that $\mathcal{B}_2$ perfectly simulates the $\mathcal{H}_3$ to $\mathcal{A}$ with the help of its own IND-CCA challenger. More precisely, if the multi-user and multi-challenge IND-CCA challenger of PKE chose $b = 0$, then we are exactly in $\mathcal{H}_3$, and if it chose $b = 1$, then we are in $\mathcal{H}_2$. Therefore, if $\mathcal{A}$ can distinguish between the two hybrids with non-negligible advantage, then $\mathcal{B}_2$ can break the multi-user and multi-challenge IND-CCA security. □

All that remains is to show that the adversary $\mathcal{A}$ has negligible advantage in hybrid $\mathcal{H}_3$. We can bound the advantage of $\mathcal{A}$ by constructing a reduction $\mathcal{B}_3$ to the unforgeability of the signature scheme. $\mathcal{B}_3$ proceeds as in hybrid $\mathcal{H}_3$, and for answering AKGen and Resolve queries it honestly generates PKE key pairs, except for answering VESign queries it does not sample a signing key pair, and instead obtains the pk from the unforgeability challenger of the signature scheme. Since $\mathcal{B}_3$ does not have the corresponding sk, whenever $\mathcal{A}$ submits a VESign query (apk, m), if apk was obtained from AKGen, then $\mathcal{B}_3$ proceeds as in hybrid $\mathcal{H}_3$, where it computes $c \leftarrow \mathsf{PKE.Enc}(\mathsf{apk}, 0)$, simulates π, forwards $\widetilde{\omega} := (c, \pi)$ to $\mathcal{A}$. In the other case, i.e., when apk was *not* obtained from AKGen oracle, then $\mathcal{B}_3$ forwards (m) to the Sign oracle that is provided by the unforgeability challenger to obtain σ. Then, $\mathcal{B}_3$ computes $c \leftarrow \mathsf{PKE.Enc}(\mathsf{apk}, \sigma)$, simulates π and outputs $\omega := (c, \pi)$ to $\mathcal{A}$. Upon receiving a Resolve query $(\mathsf{apk}, \mathsf{pk}, m, \omega)$, if apk was not obtained from the AKGen oracle, then $\mathcal{B}_3$ returns $\perp$, otherwise, it calls its Sign oracle with the input m to obtain σ, and returns this σ as output of Resolve query to $\mathcal{A}$ (note that this is consistent since $\mathcal{A}$ can only query Resolve oracle with apk obtained from the AKGen oracle).

We note that $\mathcal{B}_3$ perfectly simulates the $\mathcal{H}_3$ to $\mathcal{A}$ with the help of its unforgeability challenger. Moreover, we have that whenever $\mathcal{A}$ outputs the forgery (m^*, σ^*), where Q_S are the messages queried to VESign with apk *not* obtained from the AKGen oracle, and Q_R is the messages queried to Resolve queries with an apk obtained from the AKGen oracle, then $\mathcal{B}_3$ can forward the same tuple (m^*, σ^*) as its own forgery for the unforgeability game. During the reduction $\mathcal{B}_3$ forwards all queries to the challenger, meaning that the union of the messages in Q_S and Q_R corresponds to the messages tracked by the unforgeability challenger. Hence, this constitutes a valid forgery for the unforgeability game. Since this contradicts our assumption, MK-Opacity holds.

□

Lemma B.13. *If* NIZK *is knowledge sound, then the construction described in Figure 22 satisfies CK-Extractability.*

Proof. We prove this lemma by constructing an adversary $\mathcal{B}$ against the knowledge soundness of NIZK. $\mathcal{B}$ simulates the CK-Extractability game to $\mathcal{A}$, it first receives a crs^* from the knowledge soundness challenger of NIZK and sets $\mathsf{crs} := \mathsf{crs}^*$. Upon $\mathcal{A}$ outputting $(\mathsf{ask}^*, \mathsf{apk}^*, \mathsf{pk}^*, m^*, \omega^* := (c^*, \pi^*))$, $\mathcal{B}$ outputs the pair $(x^* := (c^*, \mathsf{pk}^*, \mathsf{apk}^*), \pi^*)$ to the knowledge soundness challenger of NIZK.

We observe that $\mathcal{B}$ perfectly simulates the extractability game to $\mathcal{A}$, and if $\mathcal{A}$ wins the extractability game, it holds that , $\mathsf{VEVer}(\mathsf{pk}^*, \mathsf{apk}^*, \omega^*) = \mathsf{NIZK.V}(\mathsf{crs}, x^*, \pi^*) = 1$ and $\mathsf{Vf}(\mathsf{pk}^*, m^*, \sigma) = 0$, where $\sigma := \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)$. The last condition implies (by the correctness of PKE and correctness of Σ) that $(x^*, w^*) \notin R$, where the witness $w^* \leftarrow \mathcal{E}_2(\mathsf{crs}, \tau_s, x^*, \pi^*)$, which gives us the winning condition against the knowledge soundness game. Hence, it follows that if $\mathcal{A}$ wins the CK-Extractability game with non-negligible advantage, then $\mathcal{B}$ also wins the knowledge soundness with non-negligible advantage. □

Lemma B.14. *The construction described in Figure 22 satisfies resolution duplication.*

Proof. In order to show that our construction satisfies resolution duplication we need to show that it satisfies resolution independence, that Resolve is a DPT algorithm and the existence of another DPT algorithm Extract that can simulate the Resolve algorithm given the secret key and the randomness of VESign.

For resolution independence observe that our VESign algorithm computes a signature σ and encrypts it under apk, which gets decrypted during the Resolve algorithm. Hence, it trivially follows that the distributions of $[\mathsf{Sign}(\mathsf{sk}, m)]$ and $[\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m))]$ are identical. Moreover, it is clear that our Resolve algorithm is a DPT algorithm, since it just runs the decryption algorithm of the underlying PKE, which is in turn a DPT algorithm. Lastly, we can define the Extract algorithm as follows[5]

$$\mathsf{Extract}(\mathsf{sk}, m; r) := \mathsf{Sign}(\mathsf{sk}, m; r).$$

We note that since the output of Sign gets encrypted inside the VESign algorithm, and later decrypted and output by the Resolve algorithm, it follows that

$$\mathsf{Extract}(\mathsf{sk}, m; r) = \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m; r)).$$

□

This completes the proof of Theorem B.5.

□

C Proofs regarding VEBS

C.1 VEBS for SPS (NIBS) Proofs

Theorem 4.1. *If the construction from Figure 9 is secure, then the* VEBS *construction described in Figure 11, is secure according to Definition 4.5.*

Proof. In order to prove Theorem 4.1, we need to show that our construction satisfies completeness, OMMK-Opacity, CK-Extractability and resolution independence/duplication as defined in Definition 4.5.

Lemma C.1. *If the NIBS construction in Figure 10 is correct, then the construction in Figure 11, is complete.*

Proof. Let us fix an arbitrary tag $t \in \mathcal{T}$ and let

$$\begin{aligned} \mathsf{pp} &\leftarrow \mathsf{Setup}(1^\lambda) \\ (\mathsf{sk}, \mathsf{pk}) &\leftarrow \mathsf{KGen}(\mathsf{pp}) \\ (\mathsf{rsk}, \mathsf{rpk}) &\leftarrow \mathsf{RKGen}(\mathsf{pp}) \\ (\mathsf{ask}, \mathsf{apk}) &\leftarrow \mathsf{AKGen}(1^\lambda) \\ \omega &\leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, \widehat{m}). \end{aligned}$$

From our construction we have that $\omega := (\mathsf{nonce}, \widetilde{\omega}, \varsigma, Y_1, Y_2, Y_2', V_2)$, where $\widetilde{\omega} := Z_1 \cdot \nu$ and $(\mathsf{psig} := (Z_1, Y_1, Y_2, V_2), \mathsf{nonce}) \leftarrow \mathsf{Issue}(\mathsf{sk}, \mathsf{rpk}, t)$. Moreover, we have that $\nu := (\mathsf{apk})^r$, $\varsigma := g_1^r$ and $Y_2' := Y_2^r$, for a random $r \in \mathbb{Z}_q$. We need to verify $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, (\mathsf{rpk}, t), (\mathsf{nonce}, \widetilde{\omega}, \varsigma, Y_1, Y_2, Y_2', V_2)) = 1$. Expanding we need to verify that $b_1 \wedge b_2 \wedge b_3 \wedge b_4$ for

$$\begin{aligned} b_1 &:= e(\mathsf{rpk}, \mathsf{pk}_1) \cdot e(H_1(\mathsf{nonce}), \mathsf{pk}_2) \cdot e(\mathsf{apk}, Y_2') = e(\omega, Y_2) \\ b_2 &:= e(Y_1, g_2) = e(g_1, Y_2) \\ b_3 &:= e(g_1, V_2) = e(Y_1, H_2(t)) \\ b_4 &:= e(\varsigma, Y_2) = e(g_1, Y_2'). \end{aligned}$$

We note that Issue algorithm computes the partial signature $\mathsf{psig} := (Z_1, Y_1, Y_2, V_2) \leftarrow \mathsf{TBEQ.Sign}(\mathsf{sk}, (\mathsf{rpk}, H_1(\mathsf{nonce})), t)$. Moreover, we have that the above conditions b_2 and b_3 are identical to the same conditions from TBEQ.Vf, and hence, by the correctness of the TBEQ scheme they hold. Therefore, it suffices to verify b_1 and b_4. For b_4 it is easy to see that since $Y_2' := Y_2^r$ and $\varsigma := g_1^r$ share the same exponent r, the pairing equality holds. For b_1, expanding the right-hand side we get

$$\begin{aligned} e(\omega, Y_2) &= e(Z_1 \cdot \nu, g_2^{s^{-1}}) \\ &= e(Z_1 \cdot \mathsf{apk}^r, g_2^{s^{-1}}) \\ &= e((\mathsf{rpk}^{\mathsf{sk}_1} \cdot H_1(\mathsf{nonce})^{\mathsf{sk}_2})^s \cdot \mathsf{apk}^r, g_2^{s^{-1}}) \quad \text{//by TBEQ.Sign} \\ &= e(\mathsf{rpk}^{\mathsf{sk}_1 s}, g_2^{s^{-1}}) \cdot e(H_1(\mathsf{nonce})^{\mathsf{sk}_2 s}, g_2^{s^{-1}}) \cdot e(\mathsf{apk}^r, g_2^{s^{-1}}) \\ &= e(\mathsf{rpk}^{\mathsf{sk}_1}, g_2) \cdot e(H_1(\mathsf{nonce})^{\mathsf{sk}_2}, g_2) \cdot e(\mathsf{apk}^r, g_2^{s^{-1}}) \\ &= e(\mathsf{rpk}, g_2^{\mathsf{sk}_1}) \cdot e(H_1(\mathsf{nonce}), g_2^{\mathsf{sk}_1}) \cdot e(\mathsf{apk}^r, g_2^{s^{-1}}) \\ &= e(\mathsf{rpk}, \mathsf{pk}_1) \cdot e(H_1(\mathsf{nonce}), \mathsf{pk}_2) \cdot e(\mathsf{apk}^r, g_2^{s^{-1}}) \\ &= e(\mathsf{rpk}, \mathsf{pk}_1) \cdot e(H_1(\mathsf{nonce}), \mathsf{pk}_2) \cdot e(\mathsf{apk}, Y_2^r) \\ &= e(\mathsf{rpk}, \mathsf{pk}_1) \cdot e(H_1(\mathsf{nonce}), \mathsf{pk}_2) \cdot e(\mathsf{apk}, Y_2'), \end{aligned}$$

which is identical to the left-hand side. Moreover, inside the Resolve algorithm we compute $\widetilde{\omega} \cdot (\varsigma)^{-\mathsf{ask}} := Z_1 \cdot (\mathsf{apk})^r \cdot (g_1)^{-\mathsf{ask} \cdot r} := Z_1$, because $(\mathsf{apk})^r = (g_1^{\mathsf{ask}})^r = (g_1)^{\mathsf{ask} \cdot r}$. Therefore, by the correctness of the NIBS scheme, for $\mathsf{psig} := (Z_1, Y_1, Y_2, V_2)$, it follows that $\mathsf{Vf}(\mathsf{pk}, t, \mathsf{Obtain}(\mathsf{rsk}, \mathsf{pk}, (\mathsf{psig}, \mathsf{nonce}), t)) = 1$. □

Lemma C.2. *If the NIBS construction given in Figure 10 is one-more unforgeable, then the* VEBS *construction given in Figure 11, satisfies OMMK-Opacity in the generic group model for type-III bilinear groups and random oracle model.*

Proof. In order to prove OMMK-Opacity, we need to ensure that the adversary can neither forge a VEBS signature nor it can extract a signature from a verifiably encrypted signature obtained from the VESign oracle using an honestly generated apk. We note that the one-more unforgeability of NIBS was proven in [54] via a reduction to the unforgeability of TBEQ scheme, which in turn was proven to be unforgeable in the generic group model for type-III bilinear groups (Appendix A.9) and random oracle model in [56] (where the unforgeability proof of the TBEQ scheme heavily relies on the scheme and proof from [38]). Since the unforgeability of TBEQ signatures relies on the generic group model along with the random oracle model, we rely on these models for the proof as well. Concretely, in the generic group model extracting Z_1 from $\widetilde{\omega} := Z_1 \cdot \nu$, for $\nu := (\mathsf{apk})^r$, $\mathsf{apk} := g_1^x$ and $r, x \in \mathbb{Z}_q$ boils down to computing discrete logarithm r or x, for which any generic PPT algorithm has negligible advantage as was shown in [82].

Equipped with this fact, the proof for OMMK-Opacity follows along the lines of the unforgeability proof of TBEQ scheme given in [56, Lemma 3.7], for which we only highlight the differences here. The forgery $\{t, (\mu_i, \sigma_i)\}_{i \in [Q_S + Q_R + 1]}$ output by the adversary $\mathcal{A}$ has to involve at least one forgery $(\mu_{i^*}, \sigma_{i^*})$ for $i^* \in [Q_S + Q_R + 1]$, such that it has not been obtained via oracle queries. As in [56, Lemma 3.7], let us denote the elements from $\mathbb{G}_2$ with "hat", e.g., as $\hat{V}$ instead of V_2. Then, we can denote the forgery as $(Z, Y, \hat{Y}, \hat{V})$, and taking the discrete logarithm of all the available group elements in

[5] To be more precise, r is the randomness of the VESign algorithm used to sign.

the forgery we obtain

$$
\begin{aligned}
z^* &= \pi_z + \sum_{i\in[Q_{\mathsf{AK}}]} \alpha_{z,i} a_i + \sum_{i\in[Q_{\mathsf{H}_1}]} \vartheta_{z,i} h_{1,i} + \sum_{j\in[Q_{\mathsf{S}}]} \varrho_{z,j} r_j \\
&\quad + \sum_{j\in[Q_{\mathsf{S}}]} \varphi_{z,j} o_j + \sum_{j\in[Q_{\mathsf{R}}]} \rho_{z,j} z_j + \sum_{j\in[Q_{\mathsf{S}}+Q_{\mathsf{R}}]} \phi_{z,j} \frac{1}{y_j} \\
y^* &= \pi_y + \sum_{i\in[Q_{\mathsf{AK}}]} \alpha_{y,i} a_i + \sum_{i\in[Q_{\mathsf{H}_1}]} \vartheta_{y,i} h_{1,i} + \sum_{j\in[Q_{\mathsf{S}}]} \varrho_{y,j} r_j \\
&\quad + \sum_{j\in[Q_{\mathsf{S}}]} \varphi_{y,j} o_j + \sum_{j\in[Q_{\mathsf{R}}]} \rho_{y,j} z_j + \sum_{j\in[Q_{\mathsf{S}}+Q_{\mathsf{R}}]} \phi_{y,j} \frac{1}{y_j} \\
\hat{y}^* &= \pi_{\hat{y}} + \sum_{i\in[\ell]} \chi_{\hat{y},i} x_i + \sum_{i\in Q_{\mathsf{H}_2}} \theta_{\hat{y},i} h_{2,i} + \sum_{j\in[Q_{\mathsf{S}}+Q_{\mathsf{R}}]} \nu_{\hat{y},j} v_j \\
&\quad + \sum_{j\in[Q_{\mathsf{S}}+Q_{\mathsf{R}}]} \psi_{\hat{y},j} \frac{1}{y_j} + \sum_{j\in[Q_{\mathsf{S}}]} \phi_{\hat{y},j} \frac{r_j}{y_j} \\
\hat{v}^* &= \pi_{\hat{v}} + \sum_{i\in[\ell]} \chi_{\hat{v},i} x_i + \sum_{i\in Q_{\mathsf{H}_2}} \theta_{\hat{v},i} h_{2,i} + \sum_{j\in[Q_{\mathsf{S}}+Q_{\mathsf{R}}]} \nu_{\hat{v},j} v_j \\
&\quad + \sum_{j\in[Q_{\mathsf{S}}+Q_{\mathsf{R}}]} \psi_{\hat{v},j} \frac{1}{y_j} + \sum_{j\in[Q_{\mathsf{S}}]} \phi_{\hat{y},j} \frac{r_j}{y_j},
\end{aligned}
$$

where $\ell = 2$ in our case, and $Q_{\mathsf{H}_1}, Q_{\mathsf{H}_2}$ denote the number of random oracle queries to H_1 and H_2, and $Q_{\mathsf{AK}}, Q_{\mathsf{S}}, Q_{\mathsf{R}}$ denote the number of oracle queries to AKGen, VESign and Resolve, respectively. The difference with respect to [56, Lemma 3.7] is that we have an additional random oracle H_1 (elements of it are denoted with coefficients $h_{1,i}$), and extra group elements such as apk coming from AKGen queries (with coefficients α_i) and $\widetilde{\omega}, \varsigma$ and Y_2' coming from VESign queries (with coefficients φ_j, ϱ_j and ψ_j, respectively). From the forgery we have that

$$
\begin{aligned}
x_1 + m^* x_2 &= z^* y^* \\
y^* &= \hat{y}^* \\
\hat{v}^* &= y^* \hat{h_2}^*,
\end{aligned}
$$

where m^* denotes the forgery message (note that forgery should be for some pair $(g_1, \mu) \in (\mathbb{G}_1)^2$) and (x_1, x_2) denote the secret key sk (since we have $\ell = 2$).

The rest of the proof follows as in [56, Lemma 3.7], which in turn is based on Claim 1 and Corollary 1 from [38]. □

Lemma C.3. *The* VEBS *construction, given in Figure 11, satsifies CK-Extractability.*

Proof. The challenger samples fresh public parameters pp and shares with the adversary $\mathcal{A}$, who responds with (ask, apk, pk, rsk, (rpk, t), ω). Due to the knowledge of secret key (KOSK) model we have that the pairs (ask, apk) and (rsk, rpk) are well-formed. Let (ask, apk) $:= (a, g_1^a)$ and (rsk, rpk) $:= (w, g_1^w)$, and ω output by the adversary consist on the tuple (nonce, $\widetilde{\omega}, \varsigma, Y_1, Y_2, Y_2', V_2$). The challenger computes psig $:= (Z_1, Y_1, Y_2, V_2) \leftarrow$ Resolve(ask, pk, (rpk, t), ω) and $(\mu, \sigma := (Z_1', Y_1', Y_2'', V_2')) :=$ Obtain(rsk, pk, (psig, nonce), t). In order to win the game, the adversary has to satisfy

$$
\begin{aligned}
&\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, (\mathsf{rpk}, t), \omega) = 1 \quad \text{and} \\
&\mathsf{Vf}(\mathsf{pk}, t, \mu, \sigma) = 0.
\end{aligned}
$$

We observe that VEVer and Vf share some common checks, namely $b_2 := e(Y_1, g_2) = e(g_1, Y_2)$ and $b_3 := e(g_1, V_2) = e(Y_1, H_2(t))$ (analogously defined for Y_1', Y_2'', V_2' as well). Given that the adjudicator and receiver keys are well-formed we have that both b_2 and b_3 should hold in both cases (otherwise VEVer will output 0). Moreover, VEVer additionally has a condition $b_4 := e(\varsigma, Y_2) = e(g_1, Y_2')$ that needs to be satisfied (which is absent from Vf). This condition implies that $\varsigma := g_1^r$ and $Y_2' := Y_2^r$ share the same exponent r. Looking at the last condition b_1, we see that

$$
e(\mathsf{rpk}, \mathsf{pk}_1) \cdot e(H_1(\mathsf{nonce}), \mathsf{pk}_2) \cdot e(\mathsf{apk}, Y_2') = e(\widetilde{\omega}, Y_2),
$$

, where $\widetilde{\omega} := Z_1 \cdot \nu$, for some $\nu := (\mathsf{apk})^{r'}$. Since this condition must be satisfied, it implies that $r = r'$, where r is the exponent involved in the check of condition b_4 (as previously described). Moreover, as ς and ν share the same exponent r, and the encryption key pair (ask, apk) are well-formed, we have that running the Resolve algorithm outputs the correct Z_1, and in turn the correct psig $:= (Z_1, Y_1, Y_2, V_2)$. Additionally, inside the Obtain algorithm we compute $\sigma := (Z_1', Y_1', Y_2'', V_2') \leftarrow$ TBEQ.ChgRep((rpk, H_1(nonce)), psig, rsk^{-1}, pk) , which changes the TBEQ signed message from (rpk, H_1(nonce)) to (g_1, μ), where $\mu := H_1(\mathsf{nonce})^{\mathsf{rsk}^{-1}}$ due to the well-formed receiver key pair (rsk, rpk). All these ensure that

$$
e(g_1, \mathsf{pk}_1) \cdot (\mu, \mathsf{pk}_2) = e(Z_1', Y_2'')
$$

also passes, and hence, we have that $\mathsf{Vf}(\mathsf{pk}, t, \mu, \sigma) = 1$ □

Lemma C.4. *The* VEBS *construction given in Figure 11, satisfies resolution duplication.*

Proof. In order to show that our construction satisfies resolution duplication we need to show that it satisfies resolution independence, that Resolve is a DPT algorithm and the existence of another DPT algorithm Extract that can simulate the Resolve algorithm given the secret key and the randomness of VESign.

For resolution independence we observe that the outputs of the Issue and Resolve algorithms are identical. More concretely, we have that

$$
\begin{aligned}
&\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, (\mathsf{rpk}, t)), \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, (\mathsf{rpk}, t)) \\
&= \left(\frac{\widetilde{\omega}}{\varsigma^{\mathsf{ask}}}, Y_1, Y_2, V_2 \right) \\
&= \left(\frac{Z_1 \cdot \nu}{g_1^{\mathsf{ask} \cdot r}}, Y_1, Y_2, V_2 \right) \\
&= \left(\frac{Z_1 \cdot (\mathsf{apk})^r}{g_1^{\mathsf{ask} \cdot r}}, Y_1, Y_2, V_2 \right) \\
&= \left(\frac{Z_1 \cdot g_1^{\mathsf{ask} \cdot r}}{g_1^{\mathsf{ask} \cdot r}}, Y_1, Y_2, V_2 \right) \\
&= (Z_1, Y_1, Y_2, V_2) \\
&= \mathsf{Issue}(\mathsf{sk}, \mathsf{apk}, \mathsf{rpk}, t).
\end{aligned}
$$

Moreover, it is clear that the Resolve algorithm is a DPT algorithm since it just computes $\widetilde{\omega} \cdot \varsigma^{-\mathsf{ask}}$, where $\widetilde{\omega}, \varsigma$ and ask are all part of the input. Lastly, we can define the Extract algorithm as follows

$$
\mathsf{Extract}(\mathsf{pp}, \mathsf{sk}, \mathsf{rpk}, t, r) := \mathsf{Issue}(\mathsf{sk}, \mathsf{rpk}, t; r).
$$

We note that since the output of Issue gets masked inside the VESign algorithm, and later unmasked and output by the Resolve algorithm,

it follows that

$$\mathsf{Extract}(\mathsf{pp}, \mathsf{sk}, \mathsf{rpk}, t, r) = \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, (\mathsf{rpk}, t)), \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, (\mathsf{rpk}, t); r).$$

□

This concludes the proof of Theorem 4.1. □

C.2 Generic VEBS Construction for Blind Signatures

The VEBS for blind signature simply consists of running the blind signing algorithm to obtain the signer message ρ_S, encrypting it using the PKE scheme under the adjudicator's public key and proving its well-formedness using the NIZK argument system. Then, the adjudicator using its secret key can decrypt and output ρ_S as part of the resolution. This generic construction is shown in Figure 24, where we assume that the scheme also has public parameters composed of the common reference string of the NIZK argument system, i.e., $\mathsf{pp} := \mathsf{crs}$, and this is given as implicit input to all algorithms. The NIZK relation that we consider is defined as follows:

$$R_{\mathsf{NIZK}} = \left\{ (\widetilde{x}, \widetilde{w}) : \begin{array}{c} \widetilde{x} := (c, \mathsf{pk}, \mathsf{apk}, (t, \rho_U)), \widetilde{w} := (\rho_S, r_0, r_1, \mathsf{sk}) \\ \rho_S = \Sigma_B.\mathsf{Signer}(\mathsf{sk}, t, \rho_U; r_0) \wedge \\ c = \mathsf{PKE.Enc}(\mathsf{apk}, \rho_S; r_1) \end{array} \right\}.$$

Remark 3. *Rückert et al. [76] provided a generic VES construction without relying on a NIZK argument system. Concretely, their construction is based on an additional trust assumption in the form of a signed Merkle tree that contains ciphertexts and commitments that allow to verify the encrypted signatures and decrypt them. Hence, the creator of the tree must be trusted to generate it honestly. Therefore, their construction includes such a trusted Merkle tree and it is proven secure under a modified version of the initial VES security model given in [12]. One could remove the NIZK in the VEBS setting by following a similar pattern, but at the cost of incorporating an additional trust assumption. However, we opted here for a more straightforward approach to capture the common mechanism used by contingent payment protocols.*

Instantiations. One efficient (but non-post-quantum) instantiation of our construction involves instantiating the PBS signature with the recent pairing-free scheme given in [62], instantiating the PKE scheme with hashed ElGamal [58] and using compressed Bulletproofs [1] for the NIZK instantiation. Alternatively, for a (plausibly) post-quantum secure instantiation, we can use Kyber [13] as the underlying PKE, instantiate the PBS with the recent lattice-based scheme given in [8], and use the LaBRADOR argument system [9] for the NIZK instantiation.

Correctness and security. The correctness and security of our construction is established with the following theorem.

Theorem C.1. *Let the blins signature scheme be a one-more unforgeable,* PKE *be a multi-user and multi-challenge IND-CCA secure, and* NIZK *be a NIZK argument system, then the* VEBS *scheme, described in Figure 24, is secure.*

Proof. In order to prove Theorem C.1, we need to show that our generic construction satisfies completeness, OMMK-Opacity, CK-Extractability and resolution independence/duplication properties described in Definition 4.5.

```
AKGen(1^λ)
1 : (sk_PKE, pk_PKE) ← PKE.KGen(1^λ)
2 : return (ask := sk_PKE, apk := pk_PKE)
Resolve(ask, pk, m̂, ω)
1 : parse (t, ρ_U) := m̂
2 : if VEVer(pk, apk, m̂, ω) = 0 then
3 :     return ⊥
4 : parse ω := (c, π)
5 : ρ_S := PKE.Dec(ask, c)
6 : return ρ_S
VESign(sk, apk, m̂)
1 : parse (t, ρ_U) := m̂
2 : ρ_S ← Σ_B.Signer(sk, t, ρ_U; r_0)
3 : c ← PKE.Enc(apk, ρ_S; r_1)
4 : x̃ := (c, pk, apk, (t, ρ_U))
5 : w̃ := (ρ_S, r_0, r_1, sk)
6 : π ← NIZK.P(crs, x̃, w̃)
7 : return ω := (c, π)
VEVer(pk, apk, m̂, ω)
1 : parse (t, ρ_U) := m̂
2 : parse ω := (c, π)
3 : x̃ := (c, pk, apk, (t, ρ_U))
4 : return NIZK.V(crs, x̃, π)
```

Figure 24: Generic construction for VEBS for blind signatures.

Lemma C.5. *If the blind signature scheme and* PKE *are correct and* NIZK *is complete, then the* VEBS *scheme, described in Figure 24, is complete.*

Proof. Let us fix an arbitrary tag/message pair $(t, m) \in \mathcal{T} \times \mathcal{M}$ and let

$$\begin{aligned} (\mathsf{sk}, \mathsf{pk}) &\leftarrow \mathsf{KGen}(1^\lambda) \\ (\mathsf{ask}, \mathsf{apk}) &\leftarrow \mathsf{AKGen}(1^\lambda) \\ (\mathsf{st}, \rho_U) &\leftarrow \mathsf{User}(\mathsf{pk}, t, m) \\ \widetilde{\omega} &\leftarrow \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, \widehat{m} := (t, \rho_U)). \end{aligned}$$

From our construction we have that $\omega := (c, \pi)$. We have that $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, \widehat{m}, \omega) := \mathsf{NIZK.V}(\mathsf{crs}, (c, \mathsf{pk}, \mathsf{apk}, (t, \rho_U)), \pi)$, and by the completeness of NIZK we have that $\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, (t, \rho_U), \omega) = 1$. Furthermore, we have that $\rho_S := \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, (t, \rho_U), \omega)$ due to the correctness of PKE. Moreover, by the correctness of blind signatures we have that $\mathsf{Vf}(\mathsf{pk}, (t, m), \sigma) = 1$, where $\sigma := \mathsf{Derive}(\mathsf{st}, \rho_S)$. □

Lemma C.6. *If* NIZK *is computationally zero-knowledge,* PKE *is multi-user and multi-challenge IND-CCA secure and the blind signature scheme is one-more unforgeable, then the* VEBS *scheme, described in Figure 24, satisfies OMMK-Opacity.*

Proof. The proof follows in a series of hybrids, where we gradually change our construction given in Figure 24, and finally reduce OMMK-Opacity to the one-more unforgeability of the underlying blind signature scheme. Let $\mathcal{H}_i \approx \mathcal{H}_{i+1}$ denote $|\Pr[\mathcal{H}_i = 1] - \Pr[\mathcal{H}_{i+1} = 1]| \leq$

$\mathsf{negl}(\lambda)$. We consider the following hybrids.

Hybrid $\mathcal{H}_0$: This corresponds to the original OMMK-Opacity game given in Definition 4.3.

Hybrid $\mathcal{H}_1$: In this hybrid, we change the way we answer VESign oracle queries. Concretely, we replace the CRS and proof (crs, π) with a simulated CRS and proof (crs^*, π^*).

Claim C.1. *If* NIZK *is computationally zero-knowledge, then it holds that $\mathcal{H}_0 \approx \mathcal{H}_1$.*

Proof. In order to show that $\mathcal{H}_0 \approx \mathcal{H}_1$, we construct a reduction $\mathcal{B}_1$ to the computational zero-knowledge property of NIZK. We note that all queries are answered as in $\mathcal{H}_0$ with the exception of VESign queries. The only difference here is that $\mathcal{B}_1$ receives a CRS crs^* from its zero-knowledge challenger, and sets $\mathsf{crs} := \mathsf{crs}^*$, instead of generating crs via the NIZK.PGen algorithm. When $\mathcal{A}$ makes a VESign query $(\mathsf{apk}, (t, \rho_U))$, $\mathcal{B}_1$ proceeds as in $\mathcal{H}_0$ to compute $\rho_S \leftarrow \Sigma_B.\mathsf{Signer}(\mathsf{sk}, (t, \rho_U); r_0)$ (note that $\mathcal{B}_1$ generates the key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \Sigma_B.\mathsf{KGen}(1^\lambda)$ at the beginning of the game) and $c \leftarrow \mathsf{Enc}(\mathsf{apk}, \rho_S; r_1)$, for some randomness $r_0, r_1 \in \{0,1\}^*$. Then, $\mathcal{B}_1$ makes an oracle call to its zero-knowledge challenger with the input $((c, \mathsf{pk}, \mathsf{apk}, (t, \rho_U)), (\rho_S, r_0, r_1, \mathsf{sk}))$ to obtain a proof π^*, and returns the tuple (c, π^*) to $\mathcal{A}$ as an answer to the VESign query.

If the zero-knowledge challenger of NIZK used the honest setup and prover algorithms to generate crs^* and π^*, then we are exactly in $\mathcal{H}_0$, and if it used the simulator, then we are in $\mathcal{H}_1$. Therefore, if $\mathcal{A}$ can distinguish between the two hybrids with non-negligible advantage, then $\mathcal{B}_1$ can break the zero-knowledge property of NIZK. Hence, the claim follows. □

Hybrid $\mathcal{H}_2$: In this hybrid, we change the way we answer VESign oracle queries. More precisely, for a query $(\mathsf{apk}, (t, \rho_U))$, if apk was obtained via the AKGen oracle, then we replace the ciphertext $c \leftarrow \mathsf{Enc}(\mathsf{apk}, \rho_S)$ with an encryption of $c^* \leftarrow \mathsf{Enc}(\mathsf{apk}, 0)$.

Claim C.2. *If* PKE *is multi-user and multi-challenge IND-CCA secure, then it holds that $\mathcal{H}_1 \approx \mathcal{H}_2$.*

Proof. In order to show that $\mathcal{H}_1 \approx \mathcal{H}_2$, we construct a reduction $\mathcal{B}_2$ to the multi-user and multi-challenge IND-CCA security of PKE (see Definition A.17, where n in our case corresponds to the number of AKGen queries that $\mathcal{A}$ makes). $\mathcal{B}_2$ proceeds as in the previous hybrid except for the following changes. $\mathcal{B}_2$ maintains an internal indices k and i and a table $\mathcal{P}$, where k gets incremented each time $\mathcal{A}$ queries the AKGen oracle and i gets incremented each time $\mathcal{A}$ calls the VESign oracle.

Upon receiving an AKGen oracle call, $\mathcal{B}_2$ makes a NewPK oracle call to its IND-CCA challenger with input k, which returns a public key pk_k to $\mathcal{B}_2$. Then, $\mathcal{B}_2$ sets $\mathsf{apk} := \mathsf{pk}_k$, $\mathcal{P}[\mathsf{apk}] = k$, increments k and forwards this apk to $\mathcal{A}$. Upon receiving a VESign query $(\mathsf{apk}, (t, \rho_U))$ from $\mathcal{A}$, $\mathcal{B}_2$ proceeds as in the previous hybrid to compute $\rho_S \leftarrow \Sigma_B.\mathsf{Signer}(\mathsf{sk}, (t, \rho_U))$ (note again that $\mathcal{B}_2$ samples the key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \Sigma_B.\mathsf{KGen}(1^\lambda)$ at the beginning of the game). Next, $\mathcal{B}_2$ checks if there exists $k \in \mathcal{P}[\mathsf{apk}]$, and if this is the case, then it queries the Enc oracle given by its IND-CCA challenger with the input (k, i, ρ_S) to receive a ciphertext c^* and increments i, otherwise, it computes the ciphertext itself as $c^* \leftarrow \mathsf{Enc}(\mathsf{apk}, \rho_S)$. Finally, $\mathcal{B}_2$ simulates the proof π^* using the NIZK simulator and responds to the VESign query with the pair $\omega^* := (c^*, \pi^*)$. Upon receiving a Resolve query $(\mathsf{apk}, \mathsf{pk}, (t, \rho_U), \omega)$, $\mathcal{B}_2$ checks if there exists some $k \in \mathcal{P}[\mathsf{apk}]$, and if not it returns $\perp$ to $\mathcal{A}$ (note that this is consistent with the functionality of the Resolve oracle, which returns $\perp$ for queries involving an apk not obtained from the AKGen oracle). Otherwise, $\mathcal{B}_2$ parses $\omega := (c, \pi)$, queries its Dec oracle with the input (k, c) to obtain ρ_S and returns it to $\mathcal{A}$ as the response to the Resolve query.

We note that $\mathcal{B}_2$ perfectly simulates the OMMK-Opacity game to $\mathcal{A}$ with the help of its own IND-CCA challenger. More precisely, if the multi-user and multi-challenge IND-CCA challenger of PKE chose $b = 1$, then we are exactly in $\mathcal{H}_1$, and if it chose $b = 0$, then we are in $\mathcal{H}_2$. Therefore, if $\mathcal{A}$ can distinguish between the two hybrids with non-negligible advantage, then $\mathcal{B}_2$ can break the multi-user and multi-challenge IND-CCA security. □

All that remains is to show that the adversary $\mathcal{A}$ has negligible advantage in hybrid $\mathcal{H}_2$. We can bound the advantage of $\mathcal{A}$ by constructing a reduction $\mathcal{B}_3$ to the one-more unforgeability of the blind signature scheme. $\mathcal{B}_3$ proceeds as in hybrid $\mathcal{H}_2$, and for answering AKGen and Resolve queries it honestly generates PKE key pairs, except for answering VESign queries it does not sample a signing key pair $(\mathsf{sk}, \mathsf{pk})$ via the $\Sigma_B.\mathsf{KGen}$ algorithm, and instead obtains the pk from the one-more unforgeability challenger. Since $\mathcal{B}_3$ does not have the corresponding sk, whenever $\mathcal{A}$ submits a VESign query $(\mathsf{apk}, (t, \rho_U))$, if apk was obtained from AKGen, then $\mathcal{B}_3$ proceeds as in hybrid $\mathcal{H}_2$, where it computes $c \leftarrow \mathsf{Enc}(\mathsf{apk}, 0^{|\rho_S|})$, simulates π, forwards $\omega := (c, \pi)$ to $\mathcal{A}$. In the other case, i.e., when apk was *not* obtained from AKGen oracle, then $\mathcal{B}_3$ forwards (t, ρ_U) to the Signer oracle that is provided by the one-more unforgeability challenger to obtain ρ_S. Then, $\mathcal{B}_3$ computes $c \leftarrow \mathsf{Enc}(\mathsf{apk}, \rho_S)$, simulates π and outputs $\omega := (c, \pi)$ to $\mathcal{A}$. Upon receiving a Resolve query $(\mathsf{apk}, \mathsf{pk}, (t, \rho_U), \omega)$, if apk was not obtained from the AKGen oracle, then $\mathcal{B}_3$ returns $\perp$, otherwise, it calls its Signer oracle with the input (t, ρ_U) to obtain ρ_S, and returns this ρ_S as output of Resolve query to $\mathcal{A}$ (note that this is consistent since $\mathcal{A}$ can only query Resolve oracle with apk obtained from the AKGen oracle).

We note that $\mathcal{B}_3$ perfectly simulates the OMMK-Opacity game to $\mathcal{A}$ with the help of its one-more unforgeability challenger. Moreover, we have that whenever $\mathcal{A}$ outputs the forgery $\{t^*, (m_i^*, \sigma_{i^*})\}_{i \in [Q_S + Q_R + 1]}$, where Q_S is the number of queries to VESign with apk *not* obtained from the AKGen oracle, and Q_R is the number of Resolve queries with an apk obtained from the AKGen oracle, then $\mathcal{B}_3$ can forward the same set $\{t^*, (m_i^*, \sigma_{i^*})\}_{i \in [Q_S + Q_R + 1]}$ as its own forgery for the one-more unforgeability game. During the reduction $\mathcal{B}_3$ makes at most $Q := Q_S + Q_R$ queries to its own Signer oracle, hence, this constitutes a valid forgery for the one-more unforgeability game. We note that by assumption this happens only with negligible probability.

This completes the proof of Lemma C.6. □

Lemma C.7. *If* NIZK *is knowledge sound, then the* VEBS *scheme, described in Figure 24, satisfies CK-Extractability.*

Proof. We prove this lemma by constructing an adversary $\mathcal{B}$ against the knowledge soundness of NIZK. $\mathcal{B}$ simulates the CK-Extractability game to $\mathcal{A}$, it first receives a crs^* from the knowledge

soundness challenger of NIZK and sets $\mathsf{crs} := \mathsf{crs}^*$. Upon $\mathcal{A}$ outputting $(\mathsf{ask}^*, \mathsf{apk}^*, \mathsf{pk}^*, \mathsf{st}^*, (t, \rho_U), m^*, \omega^* := (c^*, \pi^*))$, $\mathcal{B}$ outputs the pair $(x^* := (c^*, \mathsf{pk}^*, \mathsf{apk}^*, (t, \rho_U)), \pi^*)$ to the knowledge soundness challenger of NIZK.

We observe that $\mathcal{B}$ perfectly simulates the extractability game to $\mathcal{A}$, and if $\mathcal{A}$ wins the extractability game, it holds that $\mathsf{VEVer}(\mathsf{pk}^*, \mathsf{apk}^*, (t, \rho_U), \omega^*) = \mathsf{NIZK.V}(\mathsf{crs}, x^*, \pi^*) = 1$ and $\mathsf{Vf}(\mathsf{pk}^*, t^*, m^*, \sigma) = 0$, where $\sigma \leftarrow \mathsf{Derive}(\mathsf{st}^*, t^*, \rho_S)$ for some $\rho_S \leftarrow \mathsf{Resolve}(\mathsf{ask}^*, \mathsf{pk}^*, \omega^*)$. The last condition implies (by the correctness of PKE and correctness of blind signatures) that $(x^*, w^*) \notin R$, where the witness $w^* \leftarrow \mathcal{E}_2(\mathsf{crs}, \tau_s, x^*, \pi^*)$, which gives us the winning condition against the knowledge soundness game. Hence, it follows that if $\mathcal{A}$ wins the CK-Extractability game with non-negligible advantage, then $\mathcal{B}$ also wins the knowledge soundness with non-negligible advantage. □

Lemma C.8. *The* VEBS *scheme, described in Figure 24, satisfies resolution duplication.*

Proof. In order to show that our construction satisfies resolution duplication we need to show that it satisfies resolution independence, that Resolve is a DPT algorithm and the existence of another DPT algorithm Extract that can simulate the Resolve algorithm given the secret key and the randomness of VESign.

For resolution independence observe that our VESign algorithm computes a fresh signer message ρ_S and encrypts it under apk, which gets decrypted during the Resolve algorithm. Hence, it trivially follows that the distributions of $\left[\mathsf{Signer}(\mathsf{sk}, t, \rho_U\right]$ and $\left[\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, (t, \rho_U), \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, (t, \rho_U)))\right]$ are identical. Moreover, it is clear that our Resolve algorithm is a DPT algorithm, since it just runs the decryption algorithm of the underlying PKE, which is in turn a DPT algorithm. Lastly, we can define the Extract algorithm as follows[6]

$$\mathsf{Extract}(t, \mathsf{sk}, \rho_U; r) := \Sigma_B.\mathsf{Signer}(\mathsf{sk}, (t, \rho_U); r).$$

We note that since the output of Σ_B.Signer gets encrypted inside the VESign algorithm, and later decrypted and output by the Resolve algorithm, it follows that

$$\mathsf{Extract}(t, \mathsf{sk}, \rho_U; r) = \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, (t, \rho_U), \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, (t, \rho_U); r)).$$

□

This completes the proof of Theorem C.1. □

C.3 Generic VEBS Construction for NIBS

The generic VEBS construction follows along the lines of the one given in Appendix C.2, except that we replace the blind signature scheme with a NIBS scheme, as shown in Figure 25. For our construction, we assume that the public parameters for the NIZKand NIBS, (crs, pp) are CRS-in-the-sky, as in [54].

[6]To be more precise, r is the randomness of the VESign algorithm, which can be interpreted as $r := (r_0, r_1, r_2)$ for our construction, such that r_0 is the randomness used by the Σ_B.Signer algorithm, r_1 is the randomness of the PKE.Enc algorithm, and r_2 is the randomness of the NIZK.P algorithm.

```
AKGen(pp)
1 : (sk_PKE, pk_PKE) ← PKE.KGen(1^λ)
2 : return (ask := sk_PKE, apk := pk_PKE)
Resolve(ask, pk, m̂, ω)
1 : parse ω := (nonce, c, π)
2 : psig := Dec(ask, c)
3 : return psig
VESign(sk, apk, m̂)
1 : parse (rpk, t) := m̂
2 : (psig, nonce) ← Issue(pp, sk, rpk, t)
3 : c ← Enc(apk, psig; r)
4 : stmt := (pp, nonce, c, pk, apk, rpk, t)
5 : wit := (psig, r, sk)
6 : π ← NIZK.P(crs, stmt, wit)
7 : return (nonce, c, π)
VEVer(pk, apk, m̂, ω)
1 : parse (rpk, t) := m̂
2 : parse ω := (nonce, c, π)
3 : stmt := (pp, nonce, c, pk, apk, rpk, t)
4 : return NIZK.V(crs, stmt, π)
```

Figure 25: Generic VEBS Construction for NIBS.

Instantiations. We can instantiate our generic NIBS construction analogous to the VEBS instantiation given in Appendix C.2, except that for a non-post-quantum instantiation we can use the recent NIBS construction based on structure preserving signatures given in [54], and for a post-quantum secure instantiation we can use the lattice-based NIBS given in [5].

Correctness and security. The correctness and security of our construction is established with the following theorem.

Theorem C.2. *Let Σ_{NIBS} be a secure NIBS scheme,* PKE *be a multi-user and multi-challenge IND-CCA secure scheme, and* NIZK *be a NIZK argument system, then the* VEBS *scheme, described in Figure 25, is secure.*

Proof. In order to prove Theorem C.2, we need to show that our generic construction satisfies completeness, OMMK-Opacity, CK-Extractability (strong) and resolution independence/duplication as described in Figure 24, is secure.

Since the construction in Figure 25 is nearly identical to the VEBS construction from Appendix C.2, the proofs of the properties that are common to both VEBS, such as completeness, one-more multi-key opacity, extractability under chosen keys and resolution independence/duplication follow analogously.

□

D Adaptor Signatures vs Verifiably Encrypted Signatures

In this section, we analyze the relationship between adaptor signatures and verifiable encrypted signatures. To do so, we use a VES construction using black-box adaptor signature, which we detail

$\mathsf{AKGen}(1^\lambda)$	$\mathsf{VEVer}(\mathsf{pk}, \mathsf{apk}, m, \omega)$
$(Y, y) \leftarrow \mathsf{GenR}(1^\lambda)$	**parse** $\tilde{\sigma} \coloneqq \omega$
return $(\mathsf{ask} := y, \mathsf{apk} := Y)$	**return** $\mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma})$
$\mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m)$	$\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \omega)$
$\tilde{\sigma} \leftarrow \mathsf{PreSign}(m, \mathsf{sk}, \mathsf{apk})$	**parse** $\tilde{\sigma} \coloneqq \omega$
return $\omega := \tilde{\sigma}$	**return** $\mathsf{Adapt}(\tilde{\sigma}, y)$

Figure 26: VES instantiation from AS.

in Figure 26. In Appendix D.1, we discuss why such construction fails to satisfy resolution duplication and opacity. In Appendix D.2, we hypothesize that extractability is the key difference between AS and VES. Therefore, we define non-extractable adaptor signature, i.e., an adaptor signature for which the extract algorithm cannot exist. We use this new theoretical notion to show that a non-extractable adaptor signature satisfies VES. Furthermore, we show that VES also satisfies the security notions of a non-extractable adaptor signature.

D.1 Standard AS and VES

Theorem D.1. *Let Σ_{AS} be an adaptor signature scheme defined with respect to a signature scheme Σ and a hard relation R. If Σ_{AS} is correct, then the VES construction given in Figure 26 does not satisfy resolution duplication.*

Proof. Assume towards contradiction that VES satisfies resolution duplication (see Definition A.13). Then, it means that there exists a DPT algorithm Derive defined as

$$\begin{aligned}\mathsf{Derive}\mathsf{sk}, m; r) &= \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m; r)) \\ &= \mathsf{Adapt}(\mathsf{PreSign}(\mathsf{sk}, m, Y; r), y).\end{aligned}$$

Using such a Derive algorithm, we can construct an adversary $\mathcal{A}$ against the hardness of the relation R as follows (note $\mathcal{A}$ is given as input the challenge statement Y):

$\mathcal{A}(Y)$
Sample a key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$.
Sample an arbitrary message m.
Compute $\tilde{\sigma} \leftarrow \mathsf{VESign}(\mathsf{sk}, Y, m; r)$, for randomness $r \in \{0,1\}^*$.
Compute $\sigma \coloneqq \mathsf{Derive}(\mathsf{sk}, m; r)$.
Compute and output $y \coloneqq \mathsf{Ext}(\sigma, \tilde{\sigma}, Y)$.

We have that y returned by $\mathcal{A}$ is indeed a witness of Y, i.e., $(Y, y) \in R$, due to the correctness of the underlying adaptor signature scheme Σ_{AS}. Hence, $\mathcal{A}$ breaks the hardness of R with probability 1. □

Theorem D.2. *Let Σ_{AS} be an adaptor signature scheme defined with respect to a signature scheme Σ and a hard relation R. If Σ_{AS} is correct, then the VES construction given in Figure 26 does not satisfy opacity.*

Proof. Assume that Σ_{AS} is correct. We can construct an adversary $\mathcal{A}$ that trivially wins the opacity game with the following steps:

- $\mathcal{A}$ initializes the challenger and obtains $(\mathsf{pk}, \mathsf{apk})$.
- $\mathcal{A}$ samples two messages, m and m^*.
- $\mathcal{A}$ makes a query to VESign for message m and obtains ω.
- $\mathcal{A}$ makes a query to Resolve for message m and ω and obtains σ.
- $\mathcal{A}$ computes $\mathsf{Ext}(\sigma, \omega, \mathsf{apk})$ and obtains ask. Note that since AS is correct, ask is the secret key corresponding to apk.
- $\mathcal{A}$ makes a query to VESign for message m^* and obtains ω^*. Note that this query does not add m^* to Q_{R}.
- $\mathcal{A}$ computes $\mathsf{Adapt}(\omega^*, \mathsf{ask})$ to obtain σ^*. Note that σ^* is a valid signature for message m^* and verification key pk since we assume that AS is correct.
- $\mathcal{A}$ forwards the tuple (m^*, σ^*) to the challenger and wins the game with probability 1.

□

D.2 Non-extractable Adaptor Signature and Verifiably Encrypted Signature

Since we showed with Theorems D.1 and D.2 that algorithm Ext conflicts with the security notions of VES, we introduce the notion of a non extractable adaptor signature scheme. A non extractable adaptor signature scheme (NE-AS) is an adaptor signature for which extractability does not exist. We define this primitive as follows:

Definition D.1 (Non-extractable adaptor signature scheme). *An NE-AS w.r.t. a hard relation R and a signature scheme $\Sigma = (\mathsf{KGen}, \mathsf{Sig}, \mathsf{Vf})$ consists of three algorithms $\Sigma_{AS}^{NE} = (\mathsf{PreSign}, \mathsf{Adapt}, \mathsf{PreVf})$ defined as:*

- $\underline{\mathsf{PreSign}(\mathsf{sk}, m, Y)}$*: is a* PPT *algorithm that on input a secret key* sk*, message $m \in \{0,1\}^*$ and statement $Y \in L$, outputs a pre-signature $\tilde{\sigma}$.*
- $\underline{\mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma})}$*: is a* DPT *algorithm that on input a public key* pk*, message $m \in \{0,1\}^*$, statement $Y \in L$ and pre-signature $\tilde{\sigma}$, outputs a bit b.*
- $\underline{\mathsf{Adapt}(\tilde{\sigma}, y)}$*: is a* DPT *algorithm that on input a pre-signature $\tilde{\sigma}$ and witness y, outputs a signature σ.*

A NE-AS satisfies the following security properties: aEUF-CMA'(Definition D.2), adaptability (Definition A.35), non-extractability(Definition D.3) and unlinkability(Definition A.38).

The notion aEUF-CMA′ is similar to full extractability Definition A.36, but removing the condition regarding algorithm Ext.

Definition D.2 (aEUF-CMA′ security). *An NE-AS scheme Σ_{AS}^{NE} is* aEUF-CMA′ *secure if for every* PPT *adversary $\mathcal{A}$ there exists a negligible function* negl *such that*

$$\Pr[\mathsf{aSigForge}_{\mathcal{A},\Sigma_{AS}}(\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where the experiment $\mathsf{aSigForge}_{\mathcal{A},\Sigma_{AS}}$ *is defined as follows:*

```
aSigForge_{A,Σ_AS}(λ)
1 : Q_M := ∅, Q_C := ∅
2 : (sk, pk) ← KGen(1^λ)
3 : (m, σ) ← A^{O_Sign, O_PreSign}(pk)
4 : return (m ∉ Q_M ∧ Vf(pk, m, σ) = 1)

O_Sign(m)
1 : σ ← Sig(sk, m)
2 : Q_M := Q_M ∪ {m}
3 : return σ

O_PreSign(m, Y)
1 : σ̃ ← PreSign(sk, m, Y)
2 : if Y ∉ Q_C :
3 :     Q_M := Q_M ∪ {m}
4 : return σ̃

NewY()
1 : (Y, y) ← GenR(1^λ)
2 : Q_C := Q_C ∪ {Y}
3 : return Y
```

Definition D.3 (Non-extractability). *An NE-AS scheme Σ_{AS}^{NE} satisfies non-extractability if for any $\lambda \in \mathbb{N}$, any key pair* $(\mathsf{pk}, \mathsf{sk}) \in \mathsf{KGen}$*, any message m, any statement $Y \in R$, any presignature $\tilde{\sigma}$ such that* $\mathsf{PreVf}(\mathsf{pk}, m, Y, \tilde{\sigma}) = 1$*, any signature σ such that* $\mathsf{Vf}(\mathsf{pk}, m, \sigma) = 1$ *and any* PPT *algorithm $\mathcal{E}$, the following holds:*

$$\Pr\left[(Y, y') \in R \mid y' \leftarrow \mathcal{E}(\tilde{\sigma}, \sigma, Y)\right] \leq \mathsf{negl}(\lambda)$$

D.3 VES and NE-AS are equivalent

D.3.1 VES from NE-AS is secure.

Theorem D.3 (MK-Opacity). *Assume the NE-AS scheme offers non extractability, unlinkability and aEUF-CMA'. Then, our construction offers MK-Opacity.*

Proof. We prove this theorem with a series of hybrids, in which we introduce small changes to the original game and prove that each change provides a negligible advantage to the adversary.

Hybrid $\mathcal{H}_0$: : This game, corresponds to the original game for MK-Opacity defined in Definition 3.1 and expanded with the interactions described in our implementation.

Hybrid $\mathcal{H}_1$: : This game, works exactly as $\mathcal{H}_0$ but with highlighted gray lines. If the message m^* was not queried in oracle VESign with an encryption key obtained with oracle AKGen, the game aborts.

Hybrid $\mathcal{H}_2$: : This game, works exactly as $\mathcal{H}_0$ but with highlighted gray lines. The challenger searches for the ω and adjudicator's keys provided for message m^*. For each tuple of $((\omega, \sigma, \mathsf{apk}))$, the challenger computes PPT algorithm $\mathcal{E}$ to obtain ask'. If one of those ask' is a decryption key corresponding to the encryption key used with ω, the game aborts.

Hybrid $\mathcal{H}_3$: : This game, works exactly as $\mathcal{H}_2$ but with highlighted gray lines. To respond to queries for oracle Resolve, the challenger now keeps track of the previous queries. If the pair (ω, m) was not queried before, the challenger responds with $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ and then stores $R[(\omega, m, \mathsf{apk})] := \sigma$.

Claim D.1. *For any* PPT *adversary $\mathcal{A}$,* $|\Pr[\mathcal{H}_0 = 1] - \Pr[\mathcal{H}_1 = 1]| \leq \mathsf{negl}(\lambda)$.

Proof. In order to prove this claim, we need to show that an adversary that wins $\mathcal{H}_0$ can be used by $\mathcal{B}$ to win $\mathcal{H}_1$ and that an adversary that wins $\mathcal{H}_1$ can be used to win $\mathcal{H}_0$. We show first how to win $\mathcal{H}_1$ with an adversary that wins $\mathcal{H}_0$

```
H_0
1 : Q_S := Q_R := ∅, Q_AK := []
2 : (sk, pk) ← KGen(1^λ)
3 : (m*, σ*) ← A^O(pk)
4 : b_0 := m* ∉ {Q_S, Q_R}
5 : b_1 := Vf(pk, m*, σ*)
6 : return b_0 ∧ b_1

O_VESign(m, apk)
1 : σ̃ ← PreSign(sk, m, Y)
2 : if Q_AK[apk] = ⊥
3 :     Q_S := Q_S ∪ (m)
4 : return ω := σ̃

O_AKGen()
1 : (Y, y) ← GenR(1^λ)
2 : Q_AK[Y] := y
3 : return Y

O_Resolve(ω, m, apk)
1 : if Q_AK[apk] = ⊥ abort
2 : if PreVf(pk, m, apk, ω) = 0 abort
3 : σ ← Adapt(ω, Q_AK[apk])
4 : Q_R := Q_R ∪ {m}
5 : return σ
```

Figure 27: Original game for MK-Opacity **expanded with our NE-AS implementation.**

```
H_1
1 : Q_S := Q_R := ∅, Q_AK := [], V := []
2 : (sk, pk) ← KGen(1^λ)
3 : (m*, σ*) ← A^O(pk)
4 : if V[m*] = ⊥ abort
5 : b_0 := m* ∉ {Q_S, Q_R}
6 : b_1 := Vf(pk, m*, σ*)
7 : return b_0 ∧ b_1

O_VESign(m, apk)
1 : σ̃ ← PreSign(sk, m, Y)
2 : if Q_AK[apk] = ⊥
3 :     Q_S := Q_S ∪ (m)
4 : else
5 :     V[m] := V[m] ∪ (σ̃, apk)
6 : return ω := σ̃

O_AKGen()
1 : (Y, y) ← GenR(1^λ)
2 : Q_AK[Y] := y
3 : return Y

O_Resolve(ω, m, apk)
1 : if Q_AK[apk] = ⊥ abort
2 : if PreVf(pk, m, apk, ω) = 0 abort
3 : σ ← Adapt(ω, Q_AK[apk])
4 : Q_R := Q_R ∪ {m}
5 : return σ
```

Figure 28: Game $\mathcal{H}_1$: This game, works exactly as $\mathcal{H}_0$ but with highlighted gray lines. If the message m^* was not queried in oracle VESign **with an adjudicator's key obtained with oracle** AKGen**, the game aborts.**

Assume that $\mathcal{A}$ wins $\mathcal{H}_0$ with no negligible probability. Then $\mathcal{B}$ can use $\mathcal{A}$ to win $\mathcal{H}_1$ with the same probability with the following steps:

- $\mathcal{B}$ initializes the challenger of $\mathcal{H}_1$ and simulates the challenger of $\mathcal{H}_0$. Receives from the challenger of $\mathcal{H}_1$ the verification key of the signature scheme pk.
- $\mathcal{B}$ invokes $\mathcal{A}$ on input pk and obtains the tuple (m^*, σ^*).
- $\mathcal{B}$ checks if m^* was queried in $\mathcal{O}$VESign and $m^* \notin \{Q_S, Q_R\}$. If the message was queried, then $\mathcal{B}$ simply forwards the tuple (m^*, σ^*) to the challenger. If the message was not queried, $\mathcal{B}$ requests a fresh adjudicator's key from $\mathcal{O}$AKGen to the challenger and the submits a query to oracle $\mathcal{O}$VESign for m^* and

$\mathcal{H}_2$

1 : $Q_S \coloneqq Q_R \coloneqq \emptyset, Q_{AK} \coloneqq [\,], V \coloneqq [\,]$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
3 : $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk})$
4 : **if** $V[m^*] = \perp$ **abort**
5 : **if** $\exists(\tilde{\sigma}, \mathsf{apk}) \in V[m^*]\ s.t.$
6 : $\mathsf{ask}' \leftarrow \mathcal{E}(\tilde{\sigma}, \sigma, \mathsf{apk})$,
7 : $(\mathsf{ask}', \mathsf{apk}) \in \mathsf{AKGen}$ **abort**
8 : $b_0 \coloneqq m^* \notin \{Q_S, Q_R\}$
9 : $b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}, m^*, \sigma^*)$
10 : **return** $b_0 \wedge b_1$

$\mathcal{O}\mathsf{VESign}(m, \mathsf{apk})$

1 : $\tilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, Y)$
2 : **if** $Q_{AK}[\mathsf{apk}] = \perp$
3 : $\quad Q_S \coloneqq Q_S \cup (m)$
4 : **else**
5 : $\quad V[m] \coloneqq V[m] \cup (\tilde{\sigma}, \mathsf{apk})$
6 : **return** $\omega \coloneqq \tilde{\sigma}$

$\mathcal{O}\mathsf{AKGen}()$

1 : $(Y, y) \leftarrow \mathsf{GenR}(1^\lambda)$
2 : $Q_{AK}[Y] \coloneqq y$
3 : **return** Y

$\mathcal{O}\mathsf{Resolve}(\omega, m, \mathsf{apk})$

1 : **if** $Q_{AK}[\mathsf{apk}] = \perp$ **abort**
2 : **if** $\mathsf{PreVf}(\mathsf{pk}, m, \mathsf{apk}, \omega) = 0$ **abort**
3 : $\sigma \leftarrow \mathsf{Adapt}(\omega, Q_{AK}[\mathsf{apk}])$
4 : $Q_R \coloneqq Q_R \cup \{m\}$
5 : **return** σ

Figure 29: Game $\mathcal{H}_2$: This game, works exactly as $\mathcal{H}_1$ but with highlighted gray lines. The challenger searches for the ω and adjudicator's keys provided for message m^*. For each tuple of $((\tilde{\sigma}, \sigma, \mathsf{apk}))$, the challenger computes PPT algorithm $\mathcal{E}$ to obtain ask'. If one of those ask' is a decryption key corresponding to the encryption key used with ω, the game aborts.

the fresh adjudicator's key. Then, $\mathcal{B}$ submits the tuple (m^*, σ^*) to the challenger.

Regarding the oracles, $\mathcal{B}$ simply forwards all queries to the corresponding oracle of the challenger. This ensures that the memory of all oracles is synchronized, and that the exact same queries that $\mathcal{A}$ makes in $\mathcal{H}_0$ are also replicated by $\mathcal{B}$ for $\mathcal{H}_1$.

Note that the only difference between games is whether or not m^* is one of the messages queried by $\mathcal{A}$ to $\mathcal{O}\mathsf{VESign}$ with an adjudicator key obtained with $\mathcal{O}\mathsf{AKGen}$. Since such queries do not modify Q_S, $\mathcal{B}$ simply needs to check if $\mathcal{A}$ did such query. If m^* was not queried to VESign with an apk obtained from oracle AKGen, $\mathcal{B}$ simply makes that query to their challenger. If $\mathcal{A}$ wins $\mathcal{H}_0$ with non negligible probability, $\mathcal{B}$ wins $\mathcal{H}_1$ as well.

Now, it is left to show that an adversary that wins $\mathcal{H}_1$ also wins $\mathcal{H}_0$. Note that we mount the adversary $\mathcal{B}$ in the same way as before. Different to the previous case, $\mathcal{A}$ already selected a message that was queried in VESign. It is easy to see that the forgery from $\mathcal{A}$ that wins $\mathcal{H}_1$ also wins $\mathcal{H}_0$.

Therefore, we can conclude that $|\Pr[\mathcal{H}_0 = 1] - \Pr[\mathcal{H}_1 = 1]| \leq \mathsf{negl}(\lambda)$. □

Claim D.2. *Let* Bad_1 *be the event that* $\mathcal{H}_2$ *aborts on the gray line 7. Assume that the NE-AS is non extractable. Then* $\Pr[\mathsf{Bad}_1(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume by contradiction that there exists a PPT adversary $\mathcal{A}$ and a PPT algorithm $\mathcal{E}$ such that $\Pr[\mathcal{H}_2(\lambda) = 1] > \mathsf{negl}(\lambda)$. Then, we can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break non-extractability with the following steps:

- $\mathcal{B}$ computes $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$ and invokes $\mathcal{A}$ on input pk.
- $\mathcal{A}$ returns the tuple (m^*, σ^*).
- $\mathcal{B}$ does not abort on line 4 since we assume that $\mathcal{A}$ aborts on line 7.
- $\mathcal{B}$ since $V[m^*] \neq \perp$, there must be at least one tuple $(\tilde{\sigma}, \mathsf{apk})$. $\mathcal{B}$ computes $\mathcal{E}$ on $(\tilde{\sigma}, \sigma^*, \mathsf{apk})$ for each $(\tilde{\sigma}, \mathsf{apk})$ tuple in $V[m^*]$. If for one of these tuples ask' produced by $\mathcal{E}$ is a valid decryption key for its corresponding apk, $\mathcal{B}$ forwards the tuple $(\mathsf{pk}, m^*, \mathsf{apk}, \tilde{\sigma}, \sigma)$ to the challenger of Non-Extractability, together with the details for PPT algorithm $\mathcal{E}$. If no tuple $(\tilde{\sigma}, \mathsf{apk})$ satisfies this condition, $\mathcal{B}$ selects one of the tuples at random and forwards the required information to the challenger.

Regarding the oracles, $\mathcal{B}$ knows sk and generates the adjudicator's keys, so can simulate faithfully all of them to $\mathcal{A}$. Now, if $\mathcal{A}$ is able to abort on the highlighted gray lines, this means that for at least one of the tuples $(\tilde{\sigma}, \mathsf{apk})$ recorded in $V[m^*]$, the output of the PPT algorithm $\mathcal{E}$ is a valid decryption key for apk. Therefore, if $\mathcal{B}$ forwards the tuple $(\mathsf{pk}, m^*, \mathsf{apk}, \tilde{\sigma}, \sigma)$ to the challenger with the specification of PPT algorithm $\mathcal{E}$, it will win the non-extractability game with the same probability that $\mathcal{A}$ is able to abort. However, we assume that the probability that such a PPT algorithm $\mathcal{E}$ exists is negligible, therefore the probability that $\mathcal{A}$ aborts on the highlighted gray lines is also negligible and this claim has been proven. □

Claim D.3. *Let* Bad_2 *be the event that:*

$$\left| \begin{array}{l} \Pr[\mathcal{H}_2(\lambda) = 1] \\ -\Pr[\mathcal{H}_3(\lambda) = 1] \end{array} \right| > \mathsf{negl}(\lambda)$$

Assume that the NE-AS is unlinkable. Then $\Pr[\mathsf{Bad}_2(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume by contradiction that $\Pr[\mathsf{Bad}_2(\lambda)] > \mathsf{negl}(\lambda)$, then there exists PPT distinguisher $\mathcal{D}$ such that:

$$\Pr\left[b = b^* \middle| \begin{array}{l} b \leftarrow^{\$} \{0,1\} \\ \mathcal{H}_{b+2}(\lambda) \\ b^* \leftarrow \mathcal{D}() \end{array} \right] > \frac{1}{2} + \mathsf{negl}(\lambda)$$

We can construct adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break NE-AS unlinkability with the following steps:

- $\mathcal{B}$ initializes the challenger and obtains pk.
- $\mathcal{B}$ invokes $\mathcal{A}$ on input pk and obtains the tuple (m^*, σ^*).
- For one of the queries done by $\mathcal{A}$ to $\mathcal{O}\mathsf{Resolve}$, $\mathcal{B}$ forwards the message m and the corresponding pair (apk, ask) to the challenger. The challenger replies with a signature, which $\mathcal{B}$ relays to $\mathcal{A}$.
- $\mathcal{D}$ sends a bit to $\mathcal{B}$, who forwards this bit to the challenger.

Regarding the oracles, $\mathcal{B}$ creates the adjudicator's keys, so it can compute all of the Resolve queries normally. The only exception is the query in which instead of computing Adapt, it will forward the query to the challenger, who will either forward an adapted signature or a fresh signature. Regarding queries to $\mathcal{O}\mathsf{VESign}$, $\mathcal{B}$ simply relays the queries to oracle $\mathcal{O}\mathsf{PreSign}$ from the challenger.

The only difference between $\mathcal{H}_2$ and $\mathcal{H}_3$ is the distribution of the responses given by the challenger in oracle Resolve. In particular, in the game described above, the only difference is the one query that is forwarded to the challenger, in which the challenger

$\mathcal{H}_3$

$Q_S \coloneqq Q_R \coloneqq \emptyset$
$Q_{AK} \coloneqq [\,], V \coloneqq [\,], R \coloneqq [\,]$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KGen}(1^\lambda)$
$(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk})$
if $V[m^*] = \perp$ **abort**
if $\exists(\tilde{\sigma}, \mathsf{apk}) \in V[m^*]$ *s.t.*
$\mathsf{ask}' \leftarrow \mathcal{E}(\tilde{\sigma}, \sigma, \mathsf{apk})$,
$(\mathsf{ask}', \mathsf{apk}) \in \mathsf{AKGen}$ **abort**
$b_0 \coloneqq m^* \notin \{Q_S, Q_R\}$
$b_1 \coloneqq \mathsf{Vf}(\mathsf{pk}, m^*, \sigma^*)$
return $b_0 \wedge b_1$

$\mathcal{O}\mathsf{VESign}(m, \mathsf{apk})$

$\tilde{\sigma} \leftarrow \mathsf{PreSign}(\mathsf{sk}, m, Y)$
if $Q_{AK}[\mathsf{apk}] = \perp$
  $Q_S \coloneqq Q_S \cup (m)$
else
  $V[m] \coloneqq V[m] \cup (\tilde{\sigma}, \mathsf{apk})$
return $\omega \coloneqq \tilde{\sigma}$

$\mathcal{O}\mathsf{AKGen}()$

$(Y, y) \leftarrow \mathsf{GenR}(1^\lambda)$
$Q_{AK}[Y] \coloneqq y$
return Y

$\mathcal{O}\mathsf{Resolve}(\omega, m, \mathsf{apk})$

if $Q_{AK}[\mathsf{apk}] = \perp$ **abort**
if $\mathsf{PreVf}(\mathsf{pk}, m, \mathsf{apk}, \omega) = 0$ **abort**
$R[\omega, m, \mathsf{apk}] = \perp$
  $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
  $R[\omega, m, \mathsf{apk}] \coloneqq \sigma$
else
  $\sigma \coloneqq R[\omega, m, \mathsf{apk}]$
$Q_R \coloneqq Q_R \cup \{m\}$
return σ

Figure 30: Game $\mathcal{H}_3$: This game, works exactly as $\mathcal{H}_2$ but with highlighted gray lines. To respond to queries for oracle Resolve**, the challenger now keeps track of the previous queries. If the pair (ω, m) was not queried before, the challenger responds with $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$ and then stores $R[(\omega, m, \mathsf{apk})] \coloneqq \sigma$.**

either responds with a fresh signature or an adapted one. Hence, if there is a distinguisher $\mathcal{D}$ that is able to identify the difference between these two games, we can use it to differentiate between the distributions of adapted and fresh signatures. However, this contradicts our assumption that NE-AS satisfies unlinkability, so $\mathcal{D}$ does not exit and this claim has been proven. □

Claim D.4. *Assume that the NE-AS is* aEUF-CMA′ *secure. Then* $\Pr[\mathcal{H}_3(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.

Proof. Assume by contradiction that there exists a PPT adversary $\mathcal{A}$ that wins $\mathcal{H}_3$ with no negligible probability. Then, we can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break aEUF-CMA′ security with the following steps:

- $\mathcal{B}$ initializes the challenger and obtains pk.
- $\mathcal{B}$ invokes $\mathcal{A}$ on input pk and obtains the tuple (m^*, σ^*).
- $\mathcal{B}$ forwards the tuple (m^*, σ^*) to the challenger.

Regarding the oracles, $\mathcal{B}$ does the following. For oracle AKGen, forwards the queries to oracle NewY, assigning $\mathsf{apk} \coloneqq Y$. For oracle VESign, $\mathcal{B}$ forwards the query to $\mathcal{O}$PreSign. Note that the memory of both oracles is synchronized. For oracle Resolve, $\mathcal{B}$ forwards the execution of Sign as a query to $\mathcal{O}$Sign. This ensures that the memory of both oracles is synchronized.

Now, if $\mathcal{A}$ wins $\mathcal{H}_3$, this means that the message m^* is not in Q_R nor Q_S and that the signature is valid. These are the same conditions required to win aEUF-CMA′. Note that since the memories of both games are synchronized, if $\mathcal{A}$ satisfies the winning conditions in $\mathcal{H}_3$, $\mathcal{B}$ also does in aEUF-CMA′. However, we assume that the probability of winning aEUF-CMA′ is negligible, so $\mathcal{A}$ must not exist. This claim has been proven. □

This completes the proof of Theorem D.3. □

Theorem D.4. *Let Σ_{AS}^{NE} be an adaptor signature scheme defined with respect to a signature scheme Σ and a hard relation R. If Σ_{AS}^{NE} is unlinkable, then the VES construction* VES *given in Figure 26 satisfies resolution independence.*

Proof. Assume towards contradiction that there exists an adversary $\mathcal{A}$ that breaks resolution independence. This means that $\mathcal{A}$ distinguishes between the distributions of

$$[\mathsf{Sign}(\mathsf{sk}, m)] \text{ and } [\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m))],$$

We can use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that wins AS unlikability with the following steps:

- $\mathcal{B}$ initializes the challenger of AS unlinkability and receives the verification key pk.
- $\mathcal{B}$ samples a message m, computes $(\mathsf{ask}, \mathsf{apk}) \leftarrow \mathsf{AKGen}(1^\lambda)$ and sends $(m, (\mathsf{ask}, \mathsf{apk}))$ to the challenger.
- The challenger replies with σ_b. $\mathcal{B}$ has to reply with 0 if the signature received was adapted, or 1 if the signature was computed with Sign. $\mathcal{B}$ forwards σ_b to $\mathcal{A}$, who replies if σ_b belongs to the Sign distribution or the Resolve-VESign distribution. Then, $\mathcal{B}$ replies with the appropriate bit.

It is easy to see that if $\mathcal{A}$ distinguishes between the distributions of

$$[\mathsf{Sign}(\mathsf{sk}, m)] \text{ and } [\mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m, \mathsf{VESign}(\mathsf{sk}, \mathsf{apk}, m))],$$

with probability $\epsilon \geq \mathsf{negl}(n)$, the probability that $\mathcal{B}$ wins the AS unlinkability is also $\epsilon \geq \mathsf{negl}(n)$. However, this contradicts our assumption that AS satisfies unlinkability, so adversary $\mathcal{A}$ does not exist and this theorem has been proven. □

Theorem D.5. *Let Σ_{AS}^{NE} be an adaptor signature scheme defined with respect to a signature scheme Σ and a hard relation R. If Σ_{AS}^{NE} satisfies pre-signature adaptability, then the VES construction given in Figure 26 satisfies CK-Extractability.*

Proof. Assume by contradiction that there exists an adversary $\mathcal{A}$ that wins against the CK-Extractability game. We can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to win pre-signature adaptability with the following steps:

- $\mathcal{B}$ invokes $\mathcal{A}$ on input 1^λ to receive the tuple $(\mathsf{ask}^*, \mathsf{apk}^*, \mathsf{pk}^*, m^*, \omega^*)$.
- $\mathcal{B}$ assigns $(Y, y) \coloneqq (\mathsf{apk}^*, \mathsf{ask}^*)$, $\tilde{\sigma} \coloneqq \omega^*$ and forwards the tuple $(\mathsf{pk}^*, m^*, (Y, y), \tilde{\sigma})$ to the challenger.

Now, since $\mathcal{A}$ wins CK-Extractability, this means that the signature obtained by running Resolve on ω^* with ask^*, m^* and pk^* is not valid, but ω^* is a valid encrypted signature. Therefore, such tuple will provide an example in which a pre-signature that verifies ($\tilde{\sigma} \coloneqq \omega^*$) is adapted with the appropriate witness ($y \coloneqq \mathsf{ask}^*$), but provides an invalid signature. This contradicts our assumption that AS pre-signature adaptability holds, so adversary $\mathcal{A}$ does not exist and this claim has been proven. □

```
PreSign(sk, m, Y)
1 : parse sk := (sk', pk)
2 : ω ← VESign(sk', Y, m)
3 : return σ̃ := (m, pk, ω)
Adapt(σ̃, y)
1 : parse σ̃ := (m, pk, ω)
2 : return Resolve(y, pk, m, ω)
PreVf(pk, m, Y, σ̃)
1 : parse σ̃ := (m*, pk*, ω)
2 : assert m* = m ∧ pk* = pk
3 : return VEVer(pk, Y, m, ω)
```

Figure 31: NE-AS instantiation from VES.

D.3.2 NE-AS from VES is secure. We have shown that if we instantiate VES with a NE-AS, we have a secure VES. Now we show that if we instantiate NE-AS with VES, as our black-box construction in Figure 31, we get a secure NE-AS.

Theorem D.6 (aEUF-CMA′)**.** *If VES satisfies MK-Opacity, then the construction of non-extractable adaptor signatures, given in Figure 31, is* aEUF-CMA′ *secure.*

Proof. Assume towards contradiction that there exists a PPT adversary $\mathcal{A}$ such that $\Pr\left[\mathsf{aSigForge}_{\mathcal{A},\Sigma_{AS}}(\lambda) = 1\right] > \mathsf{negl}(\lambda)$. Then, we can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to break the MK-Opacity with the following steps:

- $\mathcal{B}$ initializes the MK-Opacity challenger and obtains pk.
- $\mathcal{B}$ invokes $\mathcal{A}$ on input pk and obtains the tuple (m^*, σ^*).
- $\mathcal{B}$ forwards the tuple (m^*, σ^*) to the challenger.

To simulate the NewY oracle, $\mathcal{B}$ simply forwards the query to its own AKGen oracle, assigns the response $Y := \mathsf{apk}$ and outputs Y to the adversary $\mathcal{A}$. Note that this simulation ensures that the adjudicator's keys and statements requested to MK-Opacity and aEUF-CMA′ are synchronized.

To simulate a query m to the $\mathcal{O}$Sign oracle, $\mathcal{B}$ samples an instance of the hard relation $(Y', y') \leftarrow \mathsf{GenR}(1^\lambda)$ and sends the query $(\mathsf{apk} := Y', m)$ to its VESign oracle. The MK-Opacity challenger replies with ω, for which $\mathcal{B}$ computes $\sigma \leftarrow \mathsf{Resolve}(y', \mathsf{pk}, m, \omega)$, and forwards σ to the adversary $\mathcal{A}$. Note that messages m queried to the $\mathcal{O}$Sign oracle are stored in the internal list Q_M (see Definition D.2). Similarly, queries to the oracle VESign with an apk not obtained from the oracle AKGen are also stored in the list Q_S (see Definition 3.1). Therefore, this simulation ensures that the list Q_M of aEUF-CMA′ game and list Q_S of MK-Opacity are synchronized.

To simulate a query (m, Y) to the $\mathcal{O}$PreSign oracle, $\mathcal{B}$ simply forwards the query $(\mathsf{apk} := Y, m)$ to the VESign oracle. The MK-Opacity challenger replies with some ω, for which $\mathcal{B}$ defines $\tilde{\sigma} := (m, \mathsf{pk}, \omega)$ and outputs $\tilde{\sigma}$ to the adversary $\mathcal{A}$. Now, if Y was obtained via a query to the NewY oracle, we have that the message m will not be appended to the lists Q_M in aEUF-CMA′ or Q_S in MK-Opacity, since the oracles NewY and AKGen are synchronized. However, if Y was sampled by the adversary, then the message m will be appended to Q_M in aEUF-CMA′ and to Q_S in MK-Opacity.

Now, if $\mathcal{A}$ wins $\mathsf{aSigForge}_{\mathcal{A},\Sigma_{AS}}$, then this means that the message m^* has not been appended to Q_M and that the signature is a valid forgery. However, these are the same conditions required to win the MK-Opacity game. Note that since the internal lists Q_M and Q_S of both games are synchronized, if $\mathcal{A}$ satisfies the winning conditions in $\mathsf{aSigForge}_{\mathcal{A},\Sigma_{AS}}$, then $\mathcal{B}$ also does in MK-Opacity. However, this contradicts our assumption that the probability of winning MK-Opacity is negligible, hence, the probability that $\mathcal{A}$ wins aEUF-CMA′ must also be negligible. □

Theorem D.7 (Adaptability)**.** *If VES satisfies CK-Extractability, then the construction of non-extractable adaptor signatures, given in Figure 31, is adaptable.*

Proof. Assume towards contradiction that there exists an adversary $\mathcal{A}$ that can break adaptability, then we can construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to win CK-Extractability with the following steps:

- $\mathcal{B}$ initializes the extractability challenger.
- $\mathcal{B}$ invokes $\mathcal{A}$, which responds with a statement and witness pair (Y, y), a key pair $(\mathsf{pk}, \mathsf{sk})$, a message m and a pre-signature $\tilde{\sigma}$.
- $\mathcal{B}$ extracts ω from the pre-signature $\tilde{\sigma}$.
- $\mathcal{B}$ assigns $\mathsf{apk} := Y$ and $\mathsf{ask} := y$.
- $\mathcal{B}$ checks that $(Y, \sigma) \in R$.
- $\mathcal{B}$ forwards the tuple $(\mathsf{ask}, \mathsf{apk}, \mathsf{pk}, m, \omega)$ to the CK-Extractability challenger.

Since by assumption we have that $\mathcal{A}$ breaks adaptability this means that the pre-signature $\tilde{\sigma}$ is valid, but the adapted signature σ is not valid and the statement and witness are in the relation. Note that these three conditions are the same to win CK-Extractability. Hence, the tuple $(\mathsf{ask}, \mathsf{apk}, \mathsf{pk}, m, \omega)$ constitutes a valid forgery against CK-Extractability. □

Theorem D.8 (Non-Extractability)**.** *If VES satisfies MK-Opacity, then the construction of non-extractable adaptor signatures, given in Figure 31, is non-extractable.*

Proof. Assume towards contradiction that there exists a PPT algorithm $\mathcal{E}$ such that non-extractability is not satisfied, i.e.,

$$\Pr\left[(Y, y') \in R \mid y' \leftarrow \mathcal{E}(\tilde{\sigma}, \sigma, Y)\right] > \mathsf{negl}(\lambda).$$

We can construct an adversary $\mathcal{A}$ that trivially wins the MK-Opacity game with the following steps:

- $\mathcal{A}$ initializes the MK-Opacity challenger and obtains pk.
- $\mathcal{A}$ samples two messages m and m^*.
- $\mathcal{A}$ queries oracle AKGen to obtain an adjudicator key apk.
- $\mathcal{A}$ makes a query (apk, m) to its VESign oracle to obtain ω.
- $\mathcal{A}$ makes a query $(\mathsf{apk}, \mathsf{pk}, m, \omega)$ to its Resolve oracle to obtain a signature σ.
- $\mathcal{A}$ computes $\mathcal{E}(\omega, \sigma, \mathsf{apk})$ and obtains ask. Note that since $\mathcal{E}$ breaks non-extractability, we have that the pair $(\mathsf{ask}, \mathsf{apk})$ is in the support of AKGen.
- $\mathcal{A}$ makes a query (apk, m^*) to its VESign oracle to obtain ω^*. Note that this query does not add m^* to the list Q_S maintained by the MK-Opacity challenger.

- $\mathcal{A}$ computes $\sigma^* \leftarrow \mathsf{Resolve}(\mathsf{ask}, \mathsf{pk}, m^*, \omega^*)$. Note that σ^* is a valid signature for the message m^* under the key pk.
- $\mathcal{A}$ forwards the pair (m^*, σ^*) to the MK-Opacity challenger.

It is easy to see that $\mathcal{A}$ wins the MK-Opacity game with the same advantage that $\mathcal{E}$ breaks non-extractability. However, this contradicts our assumption that MK-Opacity holds, so algorithm $\mathcal{E}$ does not exist and non-extractability holds. □

Theorem D.9 (Unlinkability)**.** *If VES is resolution independent, then the construction given in Figure 31, achieves unlinkability.*

Proof. Assume towards contradiction that there exists an adversary $\mathcal{A}$ that breaks unlinkability. This means that adversary $\mathcal{A}$ distinguishes between the distributions of the signatures produced using algorithm Sign and the signatures produced using algorithms PreSign and then Adapt. Note that, as described in Figure 31, algorithm PreSign is implemented as algorithm VESign, while algorithm Adapt is implemented as algorithm Resolve. Therefore, adversary $\mathcal{A}$ distinguishes between the distributions of the signatures produced using algorithm Sign and the signatures produced using algorithms VESign and then Resolve. However, this contradicts our assumption that VES is resolution independent. Therefore, such an adversary $\mathcal{A}$ does not exist and Theorem D.9 has been proven. □